

PLACEMENT READY

Interview Preparation Master

A consolidated Java full-stack, SQL, DSA, project, aptitude, and HR study bank

12 source PDFs	195 original pages	Company-wise notes
Java + Spring	SQL + DSA	React + HR

Prepared from the PDFs and placement messages supplied in this chat. Source documents are preserved unchanged in the appendix.

Contents

Contents	2
Quick alphabetical index	3
How to use this master file	5
High-frequency priority map	5
Project and architecture interview	6
Project story and ownership	6
Architecture and end-to-end flow	6
Reliability and scale scenarios	6
Java and object-oriented programming	7
Core Java	7
Strings and collections	7
Concurrency	7
Spring Boot, REST, and security	8
Spring and REST	8
JWT and Spring Security	8
Database and SQL	9
Design and fundamentals	9
Queries to write and dry-run	9
Optimization and consistency	9
React and JavaScript	10
React	10
JavaScript	10
DSA and coding practice	11
Strings and arrays	11
Linked structures, trees, and algorithms	11
Puzzles and aptitude	12
Frequently reported puzzles	12
Company-wise interview map	13
HR and behavioral preparation	14
Questions and scenarios	14
Four-week revision plan	15
Original source library	16

Quick alphabetical index

Use this index for fast navigation. Open the PDF bookmark panel to jump to a section or source, or use Ctrl+F with the listed search term.

Topic	Go to	Topic	Go to
Anagram	DSA and coding practice - Strings and arrays	JWT / OAuth / FilterChain	Spring Boot - JWT and Spring Security
Aptitude	Puzzles and aptitude; C3IT appendix	localStorage / sessionStorage	React and JavaScript - JavaScript
Architecture	Project and architecture interview	Locking / transactions	Database optimization; project scale scenarios
ArrayList / LinkedList / Vector	Java - Strings and collections	Microservices	Project - Architecture and end-to-end flow
Authentication / authorization	Spring Boot - JWT and Spring Security	MongoDB / MySQL / NoSQL	Database and SQL - Design and fundamentals
Booking concurrency	Project - Reliability and scale scenarios	NetSPI	Company-wise map; NetSPI appendix
C3IT	Company-wise map; C3IT appendix	Node event loop	Java - Concurrency
Cerillion	Project, database, React, security, DSA; Interview Questions appendix	Normalization	Database and SQL; Prescient company map
CI/CD and EC2 deployment	Project - Architecture and end-to-end flow	OOP	Java and object-oriented programming
Collections	Java - Strings and collections	Optimistic / pessimistic locking	Database - Optimization and consistency
Comparator / Comparable	Java - Strings and collections; Oxane appendix	Oxane Partners	Company-wise map
Concurrency / multithreading	Java - Concurrency	Pagination / filters	React and JavaScript - React
Context API / Redux	React and JavaScript - React	Pattern Technologies	Company-wise map; Pattern appendix
Deadlock / race condition	Java - Concurrency	Payment gateway	Project - Reliability and scale scenarios
Design patterns / Singleton	Java - Core Java	PharmaAce	Company-wise interview map
Docker	Project deployment questions; PharmaAce company map	Prescient Technologies	Company-wise interview map
DSA top 100	DSA and coding practice; Top 100 DSA appendix	Project explanation	Project and architecture interview
Entity / ORM / JPA	Spring and REST; database appendices	Puzzles	Puzzles and aptitude
Exusia	Company-wise map; HR scenarios	React hooks / useEffect / useMemo	React and JavaScript - React
FinIQ	Company-wise interview map	REST API	Spring Boot - Spring and REST
Garbage collection	Java - Core Java	SQL question bank	Database and SQL; SQL appendix
HashMap / HashSet	Java - Strings and collections	Spring annotations	Spring Boot - Spring and REST
Hoisting / closures	React and JavaScript - JavaScript	Spring Security	Spring Boot - JWT and Spring Security
HQL / SQL	Database and SQL - Design and fundamentals	Stored procedures / triggers	Database and SQL
HTTP status codes	Spring Boot - Spring and REST	Strings	Java collections; DSA strings; Java coding appendices
HR questions	HR and behavioral preparation	T-Systems	Company-wise interview map
Indexes / EXPLAIN	Database and SQL - Optimization and consistency	Thinkbridge	Company-wise map; Thinkbridge appendix
Java coding programs	DSA section; Java program appendices	Two Sum / subarray	DSA and coding practice - Strings and arrays
Java core	Java and OOP; Core and Advanced Java appendix	Vaipratech	Company-wise map; Vaipratech appendix

Topic	Go to	Topic	Go to
JavaScript equality	React and JavaScript - JavaScript	Virtual DOM / reconciliation	React and JavaScript - React
Joins / aggregation	Database and SQL - Queries to write and dry-run	Window functions / rank	Database and SQL - Queries to write and dry-run

How to use this master file

Start with the priority map, then prepare your own project story. Use the categorized checklist for active recall. After that, practise the company-wise sets and finally consult the original appendices for full context and worked examples.

- Answer concept questions in 60-90 seconds: definition, why it matters, one example, one trade-off.
- Answer project questions with evidence: problem, your action, technical choice, measurable result.
- For coding: clarify constraints, give brute force, optimize, state complexity, dry-run, test edge cases.
- For SQL: restate expected columns, identify tables and relationships, build joins, then group/filter.
- Do not claim Docker, Kubernetes, AI/ML, warehousing, or another technology unless you can handle follow-up questions.

High-frequency priority map

Priority	Topics	Expected evidence
1 - Must master	Project flow; Java OOP/strings/collections; Spring REST/security/JWT; SQL joins/grouping; basic DSA	Explain + code + project example
2 - Strong working level	React hooks/state; concurrency; transactions/locking; DB optimization; deployment flow	Scenario + trade-off
3 - Practise repeatedly	Puzzles, aptitude, rapid-fire HR, handwritten dry runs	Clear reasoning under time pressure

Project and architecture interview

Project story and ownership

1. Give a concise introduction connecting your background, strongest skills, and target role.
2. Explain your CDAC project and final-year project: problem, users, features, stack, and outcome.
3. What was your exact role? Which modules, APIs, screens, queries, or fixes did you own?
4. Describe a difficult bug, conflict, or delivery risk and how you handled it.
5. What would you redesign if traffic, data volume, or team size increased tenfold?
6. Where did you use OOP, collections, multithreading, or design patterns in the project?
7. What tables and domain models did you create? Explain their relationships.
8. What frontend and backend rating would you give yourself, and what evidence supports it?

Architecture and end-to-end flow

1. Draw the folder structure, layers, key modules, and dependency direction.
2. Draw the system architecture and database schema.
3. Walk through one major user flow from browser action to persisted data and response.
4. Write a REST API with full Controller -> Service -> Repository/DAO flow.
5. Explain your use-case diagram, actors, system boundary, and interactions.
6. Which architecture did the project follow, and why was it suitable?
7. Explain the microservices architecture: services, gateway, discovery, configuration, and communication.
8. How were configurations handled across environments: profiles, environment variables, config server, CI/CD?
9. How did you deploy to EC2? Explain build, artifact/image, network/security, database, process, domain, and verification.
10. Explain client-side rendering, server-side rendering, and what Next.js changes.

Reliability and scale scenarios

1. Only one seat remains and many users try to book it. Prevent overselling using a transaction plus optimistic or pessimistic locking.
2. When integrating a payment gateway, at what point is a seat reserved? Handle pending, success, failure, timeout, retry, idempotency, and webhook verification.
3. A payment test fails one day before a client presentation. How do you isolate the bug, communicate risk, and provide a safe fallback?
4. Database queries became slow as data grew. How do you measure, inspect EXPLAIN, index, refactor, paginate, cache, and verify?
5. How would you add a new feature without breaking the current schema and APIs?

Java and object-oriented programming

Core Java

1. Explain encapsulation, abstraction, inheritance, and polymorphism with code and real project examples.
2. Method overloading vs overriding. Can overloading depend only on return type?
3. What are static and instance methods? Can static methods be overloaded or overridden?
4. Why is main static? Explain static and non-static initialization order.
5. Explain final for variables, methods, and classes; predict outputs involving final.
6. Interface vs abstract class; what changed for interfaces in Java 8 and later?
7. Explain boxing/unboxing, access modifiers, constructors, super, and variable shadowing.
8. Explain the Java compilation and execution process: source, bytecode, JVM, JIT.
9. How are local variables allocated? When is heap memory used?
10. What is garbage collection? Can `System.gc()` force immediate collection?
11. Explain Singleton, its use cases, lazy/eager creation, and thread-safe implementation.
12. Explain SOLID and other design principles with a small example.

Strings and collections

1. Why are Strings immutable? Explain the String pool and `==` versus `equals()`.
2. String vs StringBuilder vs StringBuffer; make StringBuilder access thread-safe.
3. Explain Java Collections and why Map is not a subtype of Collection.
4. ArrayList vs LinkedList; ArrayList vs Vector; Stack vs Queue.
5. Map vs HashMap; HashMap vs Hashtable; HashMap vs ConcurrentHashMap; SortedMap.
6. Explain HashMap internals, collision handling, resizing, load factor, and null handling.
7. Can an object be a HashMap key? State the equals/hashCode contract and immutability concern.
8. Comparator vs Comparable; write multilevel sorting for objects.
9. Use `getOrDefault` to calculate character frequency.
10. Convert Employee records into a map: employee name -> list of departments.
11. Streams, lambda expressions, functional interfaces, and the functional parameter accepted by `map()`.

Concurrency

1. Process vs thread; Thread class vs Runnable; override `run()` and start execution.
2. Explain thread lifecycle and thread safety.
3. What is a race condition? Have you used multithreading in your project?
4. Explain deadlock and prevention/avoidance techniques.
5. Synchronous vs asynchronous work and suitable use cases.
6. How does a single-threaded Node.js runtime handle concurrent I/O? Explain the event loop and worker threads/processes.

Spring Boot, REST, and security

Spring and REST

1. What is inversion of control and dependency injection? What is a Spring bean?
2. Differentiate `@Component`, `@Service`, `@Repository`, and `@RestController`. What is their default scope?
3. Explain `@SpringBootApplication` and its component annotations.
4. Explain common Spring annotations and application listeners.
5. What is REST? Explain resources, verbs, representations, statelessness, and an API from your project.
6. Write a notification API where the user chooses email or SMS; choose a clean extensible design.
7. How does Spring Boot connect to a database? Explain `DataSource`, JPA, entities, repositories, and configuration.
8. Entity Framework and ORM: explain what an ORM does, benefits, N+1 risk, and when native SQL is justified.
9. Explain `@Transactional`, rollback behavior, propagation, isolation, and self-invocation caveats.
10. HTTP codes: 200/201/204, 400, 401, 403, 404, 409, 422, and 500.

JWT and Spring Security

1. Authentication vs authorization with a project example.
2. Why use JWT? Explain header, payload, signature, claims, expiry, and refresh strategy.
3. Trace token validation on frontend and backend.
4. Which signing algorithm/protocol did you use? Explain secret/public-key verification and key handling.
5. Explain Spring Security architecture and the `FilterChain` used in your project.
6. Basic Authentication vs bearer JWT vs OAuth 2.0; do not treat OAuth as a token format.
7. Where are tokens stored? Compare memory, secure cookies, `localStorage`, and `sessionStorage` with XSS/CSRF trade-offs.
8. Explain roles and permissions in your application.

Database and SQL

Design and fundamentals

1. Explain your database schema, keys, cardinalities, constraints, and indexes.
2. Can a table have two primary keys? Can a primary-key column contain NULL or duplicates?
3. Composite primary key vs composite unique key; candidate key vs unique key.
4. Model products and orders correctly using an order_items junction table.
5. Normalize a supplied table through 1NF, 2NF, and 3NF; state when denormalization helps.
6. SQL vs NoSQL; MySQL vs MongoDB; when should each be used?
7. Why use both MySQL and MongoDB? Identify transaction/integrity needs versus flexible/document use cases.
8. HQL/JPQL vs SQL; where should each be used?
9. Stored procedure vs function; loops available in stored programs.
10. What is data reconciliation? How would you detect and resolve differences safely?

Queries to write and dry-run

1. Product name and total quantity ordered from product, order, and order-item tables.
2. LEFT JOIN plus GROUP BY plus COALESCE/IFNULL for rows with no matches.
3. Find duplicate values and duplicate groups in a table.
4. Self-join Employee to return each employee and manager; show CTO when manager is NULL.
5. Second-highest and third-highest salary, including tie behavior.
6. Use aggregate functions, filters, joins, concatenation, and HAVING.
7. Explain INNER, LEFT, RIGHT, FULL, CROSS, and SELF JOIN.
8. Window functions; RANK vs DENSE_RANK vs ROW_NUMBER.
9. DELETE vs TRUNCATE vs DROP.
10. Create an appropriate trigger and discuss audit, validation, recursion, and hidden-side-effect risks.

Optimization and consistency

1. Use EXPLAIN/EXPLAIN ANALYZE to inspect access paths, scans, join order, and row estimates.
2. Choose composite-index column order from equality, range, sorting, selectivity, and covering needs.
3. Avoid SELECT *, non-sargable predicates, unnecessary DISTINCT, N+1 queries, and unbounded result sets.
4. Optimistic vs pessimistic locking; failure/retry behavior and suitable workloads.
5. Transactions, isolation levels, dirty/non-repeatable/phantom reads, and atomic booking updates.

React and JavaScript

React

1. Is React a library or framework? Explain the difference.
2. Components, props, state, and props drilling.
3. How do you manage global data? Compare Context API and Redux.
4. Explain hooks, useEffect, dependency arrays, cleanup, and common infinite-loop/stale-closure mistakes.
5. Explain useMemo and when memoization costs more than it saves.
6. React component lifecycle and its mapping to hooks.
7. Virtual DOM, reconciliation, and the diffing algorithm. When is it beneficial?
8. How do you trigger/manual rendering, and what actually causes a component to re-render?
9. Frontend pagination: client vs server pagination, API contract, page/cursor choice.
10. Filters: controlled inputs, debouncing, query parameters, cancellation, and server filtering.
11. Form validation: schema-based checks, server validation, field errors, and accessible UX.
12. Frontend optimization: code splitting, lazy loading, bundle analysis, caching, memoization, and avoiding unnecessary renders.
13. Debug a table tag that looks different by inspecting Network responses and comparing JSON values before reading backend code.
14. Which React libraries did you use, and why? What is npm and which dependencies did you add?

JavaScript

1. == versus === in JavaScript; == versus equals() in Java.
2. undefined vs null with examples.
3. Explain lexical scope and closures with a practical example.
4. Explain hoisting for var, let, const, function declarations, and the temporal dead zone.
5. JavaScript vs Java: type system, runtime, concurrency, object model, and common use cases.
6. localStorage vs sessionStorage: lifetime, scope, capacity, and security concerns.

DSA and coding practice

Interview pattern: first explain brute force, then optimize. State time and space complexity, dry-run with an example, and test empty, single-element, duplicate, negative, and overflow cases.

Strings and arrays

1. Check whether two strings are anagrams using sorting and then a frequency approach.
2. Palindrome check for string and number.
3. Reverse a string; reverse it without built-in methods or even `length()`.
4. Reverse a number and handle overflow/sign.
5. Pangram check.
6. Character-frequency map using `HashMap.getDefault()`.
7. Normalize email addresses: remove dots before `@`; after `+` ignore characters until `@`; count occurrences.
8. Decode a Caesar-shifted sentence by deriving the shift from a known original word.
9. Transform/remove repeated characters from a specified character set as described in NetSPI/Aumni variants.
10. Count special characters that are outside paired delimiter sections.
11. Two Sum: $O(n^2)$ brute force followed by $O(n)$ `HashMap`.
12. Move all zeroes to the end in-place.
13. Return a subarray with a target sum; discuss positive-only sliding window vs prefix-sum `HashMap`.
14. Reverse an array in-place.
15. Maximum consecutive ones and missing number.

Linked structures, trees, and algorithms

1. Delete a specified node from a linked list; clarify head/tail and whether only the node is provided.
2. Detect a cycle in a linked list using Floyd's algorithm.
3. Find the intersection point of two linked lists.
4. Find maximum depth/height of a binary tree and produce its mirror.
5. Implement Stack with push, pop, peek, and $O(1)$ `getMinValue`.
6. Explain and implement bubble sort, quicksort, merge sort, and heapsort with complexity.
7. Merge intervals.
8. Maximize non-overlapping meetings in one room.
9. What is a graph? Which representation and traversal would you choose?
10. Find the sum of both diagonals in a 4x4 matrix without double-counting the center in odd dimensions.
11. Optimized prime-number check up to $\sqrt{n}$.
12. Pattern-printing and dry-run questions.

Puzzles and aptitude

Frequently reported puzzles

1. Five interlocked rings: determine the minimum cuts required to separate all rings; clarify the exact chain arrangement.
2. A flower patch doubles daily and fills the pond on day 30. When was it one-quarter full?
3. Three switches in one room control one bulb in another room. Identify the switch with one visit.
4. An auto has three working tyres and two spares for 100 km. Minimize swaps so all five cover equal distance.
5. Use 5 L and 3 L beakers to measure exactly 4 L.
6. An 80 m pole: monkey climbs 5 m, slips 2 m in alternating minutes. Account for no slip after reaching the top.
7. 15 resources finish in 6 days. Estimate time with 20 resources under constant productivity.
8. Find the angle between hour and minute hands under stated conditions.
9. Nine coins: eight weigh 1 g, one weighs 2 g. Find the heavier coin in minimum balance weighings.
10. Juice-transfer puzzle: compare cross-contamination amounts after equal-volume transfers.
11. A car loses one tyre and later the tyre is found without screws. Complete the journey safely using available wheel screws.
12. Cycle/candle/L-shape/nine-stick puzzles: ask for exact diagrams and constraints before answering.
13. Geometry: isosceles-triangle/square area and concentric-circle region.
14. Work, partnership, time-speed-distance, age, and resource-rate aptitude.

Company-wise interview map

Company	Reported focus
Cerillion	Project structure/schema/flow; microservices; MySQL vs MongoDB; HQL; SQL optimization/triggers; React pagination/filters/validation; token flow; anagram; ring puzzle.
Pattern Technologies	SQL joins/grouping; product-order schema; OOP; min-stack; linked-list deletion; npm/React debugging; clock angle; Two Sum/anagram; JWT; project tables/models; HR and director rounds.
FinIQ	Test, GD on Planned Career vs Flexible Career; overloading and exceptions; memory/GC; JWT/basic auth; rapid-fire collections, sorting, graph, Spring Security, project and HR.
PharmaAce	Keys, duplicates, joins, concat, SQL vs NoSQL, windows; React hooks/context/memo/effect/reconciliation; JS closure/hoisting; Java GC/compilation/OOP; EC2/Docker; managerial fit.
Exusia	Scenario round: payment integration failure before client demo; learning an unfamiliar technology and delivering within one week; layered counter-questions.
Prescient Technologies	Deep OOP, Singleton, multithreading, thread-safe StringBuilder, HashMap/HashSet/string/regex logic, project DB design, normalization, schema change and optimization, React/Spring basics, DSA, puzzles, aptitude, HR gaps.
Oxane Partners	Strings and immutability/pool, collections and concurrent maps, REST design, SQL aggregates/self-join, Spring DB connection and @Transactional, reverse array, cycle detection, Comparable/Comparator.
T-Systems	Project contribution, full REST flow, JWT/filter chain/OAuth, palindrome/reverse array, HTTP codes, HashMap/HashSet internals, Spring annotations, conflict scenario; HR on AI/LLMs/company/location.
NetSPI	MCQ sections plus string coding; maximize meetings, mirror tree, merge intervals; see full appendix.
Thinkbridge	Screening plus paper/notepad DSA. Present problem understanding, examples, edge cases, algorithm, code, output, complexity, and diagram.
C3IT	Coding with pseudocode approval, aptitude/reading comprehension, OOP/SOLID/design, adaptability, client readiness, technical and HR rounds.
Vaiprtech	Project/Spring beans and scope, multithreading, prior coding review, email normalization and counts, streams/lambdas/functional interfaces.

HR and behavioral preparation

Questions and scenarios

1. Tell me about yourself in 60-90 seconds.
2. Why CDAC? Describe your journey and what changed after the program.
3. Why this company, role, and location? What do you know about the company?
4. Explain low engineering/WPT/web-programming marks without excuses.
5. Explain an education/career gap or a revoked offer honestly and constructively.
6. Describe your family background, hobbies, and a non-technical achievement.
7. Why are you a good team player? Give a concrete example.
8. Describe a conflict and how you handled it.
9. Will you work in support, DevOps, databases, or a new technology if assigned?
10. Rate your aptitude/problem-solving/front-end/back-end skill and identify the next improvement step.
11. Which technology are you genuinely strongest in? Expect deep follow-ups.
12. What AI tools and LLM concepts do you know? Have you used generative AI responsibly in a project?
13. What thoughtful questions will you ask about team success, mentorship, product direction, and company growth?

Presentation matters: pause, clarify assumptions, think aloud, and keep a calm, friendly tone. For gaps or weak scores, give a short fact-based explanation, what you learned, and evidence of improvement.

Four-week revision plan

Time	Focus	Deliverable
Week 1	Project story + Java core	Architecture/schema diagrams; OOP; strings; collections; 10 basic coding problems
Week 2	Spring + SQL	REST flow; JWT/security; transactions; joins/aggregates/windows; 15 query drills
Week 3	React + DSA	Hooks/state/reconciliation; arrays/strings/lists/trees/sorting; complexity and dry runs
Week 4	Mocks + company sets	Two technical mocks, one HR mock, puzzles/aptitude, weak-topic repair, concise notes

- Daily: 30 min concept recall, 45 min coding/SQL, 30 min project/HR speaking practice.
- Twice weekly: solve on paper without IDE, then verify and record missed edge cases.
- Weekly: update a one-page weakness log and retest only the weak items.

Original source library

The following source PDFs are appended in full and unchanged. Use the PDF bookmarks to jump directly to each source. The consolidated guide above removes obvious repetition; the appendices preserve original wording, code, diagrams, and context.

No.	Source	Pages	Purpose
1	Interview Questions	4	Company coding questions, DSA links, Cerillion test topics
2	Important Java Programs for QA/SDET	36	Worked Java programs
3	Core and Advanced Java Notes	16	Java concept revision notes
4	C3IT Interview and Test	5	Test, aptitude, technical, and HR experience
5	Vaipratech Interview Questions	2	Java, Spring, streams, email-normalization coding
6	NetSPI Interview Experience	3	Online test and technical rounds
7	Thinkbridge Interview Experience	3	Screening and DSA round format
8	Pattern Technologies Interview Experience	4	Online test and technical interview
9	Complete Java/Spring Boot Interview Guide	15	Scanned 2025 preparation guide
10	Top 100 DSA Interview Questions	9	Curated DSA practice list
11	SQL Interview Question Bank	68	Beginner through advanced SQL questions
12	Java Coding Interview Questions	30	Java coding questions with sample answers

SOURCE 01

Interview Questions

Company coding questions, DSA links, Cerillion test topics

DSA problems

Pattern

You are given an input string encoded using alphabet shifting, either to the right or to the left by a fixed distance. Another input will be given for the original word from the input string. Your task is to decode the input string and return the original string.

A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z.

Eg.

If original string is : Testing is Boring! (This is not given)

Input will be : String s = Uftujoh jt Cpsjph! and string originalWord = "Boring";

Expected output: Testing is Boring!

(alphabet is shifted to right by 1, this difference can be anything, negative as well)

Using originalWord we have to find out the shift and return the entire original string.

Aumni & NetSpi (Exactly same question was asked in the coding round for both companies!)

Given a string, capitalize any consecutive occurrences of characters 'abeioust' that appear two or more times consecutively. Return the modified string.

For example, for the input string "Testing is boring!", the modified string is "TESTIng IS BOring".

NetSPI (interview)

Codes:

<https://www.geeksforgeeks.org/merging-intervals/>

<https://www.geeksforgeeks.org/find-the-maximum-depth-or-height-of-a-tree/>

<https://www.geeksforgeeks.org/move-zeroes-end-array/>

<https://www.geeksforgeeks.org/check-if-pair-with-given-sum-exists-in-array/>

Puzzles:

<https://www.youtube.com/watch?v=Q1B5lgxIPrU>

<https://www.careercup.com/question?id=5145121580384256>

Cumulus (Interview)

<https://www.geeksforgeeks.org/write-a-function-to-get-the-intersection-point-of-two-linked-lists/>

<https://www.geeksforgeeks.org/find-the-missing-number/>

Cerillion

Written test

Basic MCQs on java, javascript, html, aptitude, MySQL

Three queries based on joins, group by, third highest salary

Codes:

Javascript: write a function to return sum of all the elements of given array

Java: Given a string containing '"', '|' characters, and other characters, write a function to count the number of '"' characters that are not enclosed by two '|' characters.

Eg: sdf*|df*dsf|df*das*|sdf**f|

Expected output: 3

sdf*|df*dsf|df*das*|sdfdf|

Highlighted parts are enclosed between two '|'s

Java: Given a string, determine if it is a valid string, which means it contains matching pairs of parentheses, brackets, and curly braces. For example, the string 'aaa(bbb{ccc[aaa]bbb})ccc' is valid because each opening parenthesis, bracket, or curly brace has a matching closing pair. However, the string '((aaab)' is invalid because the parentheses are not correctly matched.

Interview

<https://www.geeksforgeeks.org/java-program-count-occurrences-character/>

Core Java

Latest version of java? Which one did you use in your project?

Oops concepts

Explain OOPs pillars that helped you in your project. Use cases were expected.

Abstract class, interface, multiple inheritance

Name collision when we implement two or more interfaces and they have methods with the same signatures.

Method overloading, method overriding

Upcasting, downcasting

Default methods in interface

Exception handling, try-catch-finally blocks, System.out()

What will be the output:

```
Try{
x = 1.0 / 0.0
}
Catch(RuntimeException rte)
{
sop("Runtime exception caught");
}
catch(exception e)
{
sop("Exception caught");
}
```

What will be the output:

```
System.out.println(Math.min(Double.MIN_VALUE, 0.0));
```

Threading: Why is there Runnable interface when there is already Thread class?

How is synchronisation achieved in java? Use case for synchronisation and situation where it can be inefficient.

Collection:

How do Set and HashMap work internally in Java?

What is their initial capacity and what happens when it is exhausted?

Time complexity to add an element in ArrayList, LinkedList, Set.

Is HashMap thread safe? How to make it thread safe?

Advanced java project related:

Explain the flow of a REST request in a Spring Boot application, starting from the React client to the backend DispatcherServlet, through filters (for JWT authentication/authorization), and finally to the Database.

Annotations used in the project

@Transactional use case, is it required every time? Is it method level or class level?

Where was it really required in your project, use case?

What is n+1 problem in hibernate? Did you face it in your project?

What is eager and lazy fetching? Use case for both?

What approach did you use to create tables in the database? (code-first approach with Hibernate as the ORM)

Which tools did you use for testing your project?

MySQL

Types of joins and use cases.

Stored procedures, triggers.

Consider implementing an after insert trigger on a table that adds the same entry to the same table in the trigger. What will be the result of such an implementation?

Where vs having

Group by, order by

MySQL functions, line functions, group functions examples

Drop vs delete vs truncate

DDL vs DML commands

Use of indexes? What happens internally when we create an index on a column? How to select columns to create indexes?

How do you configure a many-to-many entity association mapping?

How to rename a column?

GIT:

How would you approach making changes to the code on the main branch? Write the Git commands you would use.

SDLC:

Explain the steps of the Software Development Life Cycle (SDLC) in the context of your project, from requirement gathering to deployment.

SOURCE 02

Important Java Programs for QA/SDET

Worked Java programs

IMP JAVA Programs for QA/SDET Interview

1.) Java program to Find Odd or Even number

```
import java.util.Scanner;

public class OddEven {
    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter any number: ");
        int number = scanner.nextInt();

        if (number % 2 == 0) {
            System.out.println(number + " is even.");
        } else {
            System.out.println(number + " is odd.");
        }
    }
}
```

2.) Java program to find Prime number

```
import java.util.Scanner;

public class PrimeNumber {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();

        if (isPrime(number)) {
            System.out.println(number + " is a prime number.");
        } else {
            System.out.println(number + " is not a prime number.");
        }
    }

    public static boolean isPrime(int num) {
        for (int i = 2; i <= num / 2; i++) {
            //try each number by using %
            if (num % i == 0) {
                return false;
            }
        }
        return true;
    }
}
```

3.) Java program to find Fibonacci series upto a given number range

```
import java.util.Scanner;

public class PrimeNumber {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("enter number of terms");
        int number = 6;
        int first = 0, second = 1, next;
        System.out.println("Fibonacci series is ");
        for ( int i = 0; i<=number; i++)
        {
            System.out.println(first + "");
            next = second+first;
            first = second;
            second = next;
        }
    }
}
```

Output: 0 1 1 2 3 5 8

4.) Java program to swap two numbers without using third variable

```
import java.util.Scanner;

public class SwapNumbers {
    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter the first number: ");
        int a = 5,
        System.out.print("Enter the second number: ");
        int b = 10;
        System.out.println("Before swapping: a = " + a + ", b = " + b);

        a = a + b;
        b = a - b;
        a = a - b;
        System.out.println("After swapping: a = " + a + ", b = " + b);

    }
}
```

Output: After Swapping: a = 10 , b = 5

5.) Java program to Find Factorial on given Number

```
import java.util.Scanner;

public class FactorialNumber {

    public static void main(String[] args) {
        int factorial =1;
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter any number ");
        int number = 5;

        for (int i = 1; i <= number; i++){
            factorial = factorial * i;
        }
        System.out.println("Factorial number is :" +factorial);
    }
}
```

Input: 5!

Output: 5! = 5*4*3*2*1 = 120

6.) Java program to Reverse Number

```
import java.util.Scanner;

public class ReverseNumber {

    public static void main(String[] args) {
        int no, rev=0,r,a;
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter any number : ");
        no = scanner.nextInt();
        a = no;
        while(no>0)
        {
            r = no%10;
            rev = rev*10+r;
            no=no/10;
        }

        System.out.println("Reverse : " +rev);
    }
}
```

Input: 15786

Output: 68751

7.) Java program to find **Armstrong Number**

```
import java.util.Scanner;
public class ArmstrongNumber {

    public static void main(String[] args) {
        int arm=0, a,b,c,d,no;
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter any number : ");
        no = scanner.nextInt();
        d = no;
        while(no>0)
        {
            a = no%10;
            no = no/10;
            arm =arm+a*a*a;
        }
        if(arm==d){
            System.out.println("Armstrong number");
        }
        else{
            System.out.println("Not Armstrong number");
        }
    }
}
```

8.) Java program to find **number of digits** in given number

```
import java.util.Scanner;
public class NumberOfDigits {

    public static void main(String[] args) {
        int no = 0, a = 0;
        Scanner scanner = new Scanner(System.in);
        System.out.println("Enter any number : ");
        no = scanner.nextInt();
        if(no<0)
        {
            no = no * -1;
        }
        else if (no==0) {
            no=1;
        }
        while(no>0)
        {
            no=no/10;
            a++;
        }
        System.out.println("Number of digits in given number is :"+a);
    }
}
```

9.) Java program to find **Palindrome number**

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int number = scanner.nextInt();

        if (isPalindrome(number)) {
            System.out.println(number + " is a palindrome.");
        } else {
            System.out.println(number + " is not a palindrome.");
        }
    }

    public static boolean isPalindrome(int num) {
        int originalNumber = num;
        int reversedNumber = 0;

        while (num != 0) {
            int digit = num % 10;
            reversedNumber = reversedNumber * 10 + digit;
            num = num / 10;
        }

        return originalNumber == reversedNumber;
    }
}
```

Enter a number: 1001

1001 is a palindrome.

10.) Java program to calculate the sum of digits of a number

```
public class Main {
    public static void main(String[] args) {
        int number = 12345;

        int sumOfDigits = calculateSumOfDigits(number);

        System.out.println("Sum of digits of " + number + " is: " +
sumOfDigits);
    }

    public static int calculateSumOfDigits(int number) {
        int sum = 0;
        while (number > 0) {
            int digit = number % 10; // Extract the last digit
            sum = sum + digit; // Add the digit to sum
            number = number / 10; // Remove the last digit from number
        }
        return sum;
    }
}
```

Output:

Sum of digits of 12345 is: 15

Strings

1.) Java program to reverse a string

```
import java.util.Scanner;
public class Test {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();
        char ch;
        String nstr = "";
        for (int i = 0; i < input.length(); i++) {
            ch = input.charAt(i);
            nstr = ch + nstr;
        }
        System.out.println("Reversed String is : " + nstr);
    }
}
```

2.) Java program to reverse each word of a given string

```
public static void main(String[] args) {
    reverseEachWordOfString("Java is good programming langauges");
}

static void reverseEachWordOfString(String inputString)
{
    String[] words = inputString.split(" ");

    String reverseString = "";
    for (int i = 0; i < words.length; i++) {
        String word = words[i];
        String nstr = "";
        char ch;
        for (int j = 0; j < word.length(); j++) {
            ch = word.charAt(j);
            nstr = ch + nstr;
        }
        reverseString = reverseString + nstr + " ";
    }

    System.out.println(inputString);
    System.out.println(reverseString);
}
```

Input: Java is good programming langauges

Output: avaJ si doog gnimmargorp seguagnal

3.) Java program to find duplicate characters in a string

```
import java.util.HashMap;
import java.util.Set;

public class Main {

    public static void main(String[] args) {
        duplicateCharacterCount("Learn Java Programming");
    }

    static void duplicateCharacterCount(String inputString) {

        HashMap<Character, Integer> charCountMap = new HashMap<>();
        char[] strArray = inputString.toCharArray();
        for (char c : strArray) {
            if (charCountMap.containsKey(c)) {
                charCountMap.put(c, charCountMap.get(c) + 1);
            } else {
                charCountMap.put(c, 1);
            }
        }

        Set<Character> charsInString = charCountMap.keySet();
        System.out.println("Duplicate Characters in : " + inputString);

        for (Character ch : charsInString) {
            if (charCountMap.get(ch) > 1) {
                System.out.println(ch + " : " + charCountMap.get(ch));
            }
        }
    }
}
```

Duplicate Characters in : Learn Java Programming

a : 4

g : 2

m : 2

n : 2

r : 3

4.) Java program to count Occurrences of Each Character in String

```
import java.util.HashMap;

public class Main {

    public static void main(String[] args) {
        CharacterCount("Test Automation Java Automation");
    }

    static void CharacterCount(String inputString) {
        HashMap<String,Integer> charCountMap = new HashMap<>();
        for(String s : inputString.split(" "))
        {
            if(charCountMap.containsKey(s))
            {
                charCountMap.put(s, charCountMap.get(s)+1);
            }
            else
            {
                charCountMap.put(s,1);
            }
        }
        System.out.println("Count of Characters in a given string : " +
            charCountMap);
    }
}
```

Count of Characters in a given string : {Java=1, Automation=2, Test=1}

5.) Java program to count the number of words in a string

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Enter the String");
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        int count = 1;

        for (int i = 0; i < s.length() - 1; i++) {
            if ((s.charAt(i) == ' ') && (s.charAt(i + 1) != ' ')) {
                count++;
            }
        }
        System.out.println("Number of words in a string: " +count);    }
}
```

Enter the String: Welcome to Java World
Number of words in a string: 4

6.) Java program to find all permutations of a given string

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        String str = "abc";
        permute(str, "");
    }

    static void permute(String str, String prefix) {
        if (str.length() == 0) {
            System.out.println(prefix);
        } else {
            for (int i = 0; i < str.length(); i++) {
                String rem = str.substring(0,i) + str.substring(i+1);
                permute(rem,prefix + str.charAt(i));
            }
        }
    }
}
```

abc

acb

bac

bca

cab

cba

7.) Java program to find if a string is Palindrome

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        String str = "madam";
        System.out.println(isPalindrome(str));
    }

    static boolean isPalindrome(String str) {
        int start = 0;
        int end = str.length() - 1;

        while (start < end) {
            if (str.charAt(start) != str.charAt(end)) {
                return false;
            }
            start++;
            end--;
        }
        return true;
    }
}
```

8.) Java program to determine if Two Strings are Anagrams

```
public class Main {

    public static void main(String[] args) {
        String str1 = "listen";
        String str2 = "silent";
        System.out.println(areAnagrams(str1, str2));
    }

    static boolean areAnagrams(String str1, String str2) {
        if(str1.length() != str2.length())
        {
            return false;
        }

        int[] charCount = new int[256];
        for( int i = 0; i < str1.length(); i++)
        {
            charCount[str1.charAt(i)]++;
            charCount[str2.charAt(i)]--;
        }

        for ( int count : charCount)
        {
            if ( count !=0 )
            {
                return false;
            }
        }
        return true;
    }
}
```

9.) Java program to Count Vowels and Consonants in a given string

```
public class Main {
    public static void main(String[] args) {
        String str = "Hello World";
        VowelConsonantCount(str);
    }

    static void VowelConsonantCount(String str) {
        int vowels = 0, consonants = 0;
        str = str.toLowerCase();
        for (char c : str.toCharArray()) {
            if (c >= 'a' && c <= 'z') {
                if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
                {
                    vowels++;
                } else {
                    consonants++;
                }
            }
        }
        System.out.println("Vowels : " + vowels);
        System.out.println("Consonants : " + consonants);
    }
}
```

Vowels : 3

Consonants : 7

10.) Java program to print unique characters

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();

        System.out.println("Unique characters in \"" + input + "\"");
        printUniqueCharacters(input);
    }

    public static void printUniqueCharacters(String str) {
        // Assume ASCII characters (0-127), use boolean array to track
        character occurrences
        boolean[] unique = new boolean[128];

        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            if (!unique[ch]) {
                unique[ch] = true;
                System.out.print(ch + " ");
            }
        }
    }
}
```

Enter a string: **Java Automation**

Unique characters in "Java Automation":

J a v A u t o m i n

11.) Java program to print even indexed characters

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();

        System.out.println("Even indexed characters in \"" + input + "\"");
        printEvenIndexedCharacters(input);
    }

    public static void printEvenIndexedCharacters(String str) {
        for (int i = 0; i < str.length(); i++) {
            if (i % 2 == 0) {
                System.out.print(str.charAt(i));
            }
        }
    }
}
```

Enter a string: Automation

Even indexed characters in "Automation":

Atmto

12.) Java program to remove space from a given string

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string with spaces: ");
        String input = scanner.nextLine();

        String stringWithoutSpaces = removeSpaces(input);
        System.out.println("String without spaces: " +
stringWithoutSpaces);
    }

    public static String removeSpaces(String str) {
        StringBuilder result = new StringBuilder();
        for (int i = 0; i < str.length(); i++) {
            if (str.charAt(i) != ' ') {
                result.append(str.charAt(i));
            }
        }
        return result.toString();
    }
}
```

```
Enter a string with spaces: Welcome to Java World
String without spaces: WelcometoJavaWorld
```

13.) Java program to print each letter twice from a given string

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();

        String doubledString = doubleCharacters(input);
        System.out.println("Doubled characters: " + doubledString);
    }

    public static String doubleCharacters(String str) {

        StringBuilder doubled = new StringBuilder();
        for (int i = 0; i < str.length(); i++) {
            char ch = str.charAt(i);
            doubled.append(ch).append(ch); // Append each character
        }
        return doubled.toString();
    }
}
```

```
Enter a string: hello
Doubled characters: hheelllloo
```

14.) Java program to swap two string without using 3rd variable

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter first string: ");
        String str1 = scanner.nextLine();
        System.out.print("Enter second string: ");
        String str2 = scanner.nextLine();

        System.out.println("Before swapping: str1 = " + str1 + ",
str2 = " + str2);

        // Swapping without using a third variable
        str1 = str1 + str2; // Concatenate str1 and str2 and
store in str1
        str2 = str1.substring(0, str1.length() - str2.length());
        // Extract the initial part (original str1) from the concatenated
string
        str1 = str1.substring(str2.length()); // Extract the
remaining part (original str2) from the concatenated string

        System.out.println("After swapping: str1 = " + str1 + ",
str2 = " + str2);
    }
}
```

Enter first string: Hello

Enter second string: World

Before swapping: str1 = **Hello**, str2 = **World**

After swapping: str1 = **World**, str2 = **Hello**

15.) Java program to gives Output: a2b2c3d2 for the Input String Str = "aabbccdd"

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();

        String output = getCharacterCount(input);
        System.out.println("Output: " + output);
    }

    public static String getCharacterCount(String str) {
        StringBuilder result = new StringBuilder();
        int count = 1;

        for (int i = 0; i < str.length(); i++) {
            // If the next character is the same, increase the count
            if (i + 1 < str.length() && str.charAt(i) == str.charAt(i
+ 1)) {
                count++;
            } else {
                // Append the character and its count to the result
                result.append(str.charAt(i)).append(count);
                count = 1; // Reset the count
            }
        }

        return result.toString();
    }
}
```

Enter a string: aabbccdd

Output: a2b2c3d2

16.) Java program to gives two Output: “abcde”, “ABCDE” for the Input String Str = “aBACbcEDed”

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();
        System.out.println("Original String is: "+ input);
        separateCharacters(input);
    }

    public static void separateCharacters(String input)
    {
        StringBuilder lowerCase = new StringBuilder();
        StringBuilder upperCase = new StringBuilder();

        for(char ch : input.toCharArray())
        {
            if(Character.isLowerCase(ch))
            {
                lowerCase.append(ch);
            }
            else
            {
                upperCase.append(ch);
            }
        }
        System.out.println("Output in lowercase: "+lowerCase);
        System.out.println("Output in uppercase "+upperCase);
    }
}
```

Enter a string: **aBACbcEDed**

Output in lowercase: abcde

Output in uppercase: ABCDE

17.) Java program to gives two Output: “Subburaj”, “123” for the Input String Str = “Subbu123raj”

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter a string: ");
        String input = scanner.nextLine();
        System.out.println("Original String is: "+ input);
        separateAlphaAndNumeric(input);
    }

    public static void separateAlphaAndNumeric(String input)
    {
        StringBuilder alphaPart = new StringBuilder();
        StringBuilder numericPart = new StringBuilder();

        for(char ch : input.toCharArray())
        {
            if(Character.isLetter(ch))
            {
                alphaPart.append(ch);
            }
            else if (Character.isDigit(ch))
            {
                numericPart.append(ch);
            }
        }

        System.out.println("Output in Alpha: "+alphaPart.toString());
        System.out.println("Output in Numeric: "+numericPart.toString());
    }
}
```

Enter a string: **Subbu123raj**

Output in lowercase: Subburaj

Output in uppercase: 123

18.) Java program to gives Output: "32412120000" for the Input String Str = "32400121200"

```
public class Main {
    public static void main(String[] args) {
        String input = "32400121200";
        String output = rearrangeDigits(input);
        System.out.println("Output: " + output);
    }

    public static String rearrangeDigits(String input) {
        // Split the input into parts: digits and non-digits
        StringBuilder digits = new StringBuilder();
        StringBuilder nonDigits = new StringBuilder();

        for (char c : input.toCharArray()) {
            if (Character.isDigit(c)) {
                digits.append(c);
            } else {
                nonDigits.append(c);
            }
        }

        // Concatenate non-digits followed by digits
        return digits.toString() + nonDigits.toString();
    }
}
```

Output: 32412120000

19.) Java program to gives Output: "00003241212" for the Input String Str = "32400121200"

```
public class Main {
    public static void main(String[] args) {
        String input = "32400121200";
        String formattedOutput = String.format("%011d",
Long.parseLong(input));
        System.out.println("Formatted output: " + formattedOutput);
    }
}
```

Formatted output: 00003241212

20.) Java program to find the longest without repeating characters

```
import java.util.HashSet;

public class Main {
    public static void main(String[] args) {
        String s1 = "abcabcbb"; // Expected: "abc", length 3
        String s2 = "bbbbbb";   // Expected: "b", length 1
        String s3 = "pwwkew";   // Expected: "wke", length 3
        String s4 = "";         // Expected: "", length 0

        System.out.println("Longest substring without repeating
characters in s1: " + lengthOfLongestSubstring(s1)); // Output: 3
        System.out.println("Longest substring without repeating
characters in s2: " + lengthOfLongestSubstring(s2)); // Output: 1
        System.out.println("Longest substring without repeating
characters in s3: " + lengthOfLongestSubstring(s3)); // Output: 3
        System.out.println("Longest substring without repeating
characters in s4: " + lengthOfLongestSubstring(s4)); // Output: 0
    }

    public static int lengthOfLongestSubstring(String s) {
        HashSet<Character> set = new HashSet<>();
        int maxLength = 0;
        int start = 0;
        int end = 0;

        while (end < s.length()) {
            char currentChar = s.charAt(end);
            if (!set.contains(currentChar)) {
                set.add(currentChar);
                maxLength = Math.max(maxLength, end - start + 1);
                end++;
            } else {
                set.remove(s.charAt(start));
                start++;
            }
        }

        return maxLength;
    }
}
```

Arrays

1.) Find **common elements** between two arrays

```
import java.util.HashSet;
import java.util.Set;

public class CommonElements {
    public static void main(String[] args) {
        int[] array1 = {1, 2, 3, 4, 5};
        int[] array2 = {4, 5, 6, 7, 8};

        Set<Integer> commonElements = findCommonElements(array1,
array2);

        System.out.println("Common elements: " + commonElements);
    }

    public static Set<Integer> findCommonElements(int[] array1,
int[] array2) {
        Set<Integer> set1 = new HashSet<>();
        Set<Integer> commonSet = new HashSet<>();

        // Add elements of the first array to the set
        for (int num : array1) {
            set1.add(num);
        }

        // Check for common elements in the second array
        for (int num : array2) {
            if (set1.contains(num)) {
                commonSet.add(num);
            }
        }

        return commonSet;
    }
}
```

**Input: array1 = {1,2,3,4,5} and
array2 = {4,5,6,7,8}**

Output: Common elements: [4, 5]

2.) Find first and last element of ArrayList

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> arrayList = new ArrayList<>();
        arrayList.add("Apple");
        arrayList.add("Banana");
        arrayList.add("Cherry");
        arrayList.add("Date");
        arrayList.add("Elderberry");

        if (!arrayList.isEmpty()) {
            String firstElement = arrayList.get(0);
            String lastElement = arrayList.get(arrayList.size() - 1);

            System.out.println("First element: " + firstElement);
            System.out.println("Last element: " + lastElement);
        } else {
            System.out.println("The ArrayList is empty.");
        }
    }
}
```

Output:

First element: Apple

Last element: Elderberry

3.) Sort an array without using in-built method

```
public class Main {
    public static void main(String[] args) {
        int[] array = {5, 2, 9, 1, 6};

        selectionSort(array);

        System.out.println("Sorted array:");
        for (int num : array) {
            System.out.print(num + " ");
        }

        public static void selectionSort(int[] array) {
            int n = array.length;
            for (int i = 0; i < n - 1; i++) {
                int minIndex = i;
                for (int j = i + 1; j < n; j++) {
                    if (array[j] < array[minIndex]) {
                        minIndex = j;
                    }
                }

                // Swap array[i] and array[minIndex]
                int temp = array[i];
                array[i] = array[minIndex];
                array[minIndex] = temp;
            }
        }
    }
}
```

Output:

Sorted array:

1 2 5 6 9

4.) Remove duplicates from an Array

```
import java.util.HashSet;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        int[] array = {5, 2, 9, 1, 6, 2, 5};

        int[] uniqueArray = removeDuplicates(array);

        System.out.println("Array with duplicates removed:");
        for (int num : uniqueArray) {
            System.out.print(num + " ");
        }
    }

    public static int[] removeDuplicates(int[] array) {
        Set<Integer> set = new HashSet<>();
        for (int num : array) {
            set.add(num);
        }

        int[] result = new int[set.size()];
        int i = 0;
        for (int num : set) {
            result[i++] = num;
        }

        return result;
    }
}
```

Output:

Array with duplicates removed:

1 2 5 6 9

5.) Remove duplicates from an ArrayList

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.Set;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> arrayList = new ArrayList<>();
        arrayList.add(5);
        arrayList.add(2);
        arrayList.add(9);
        arrayList.add(1);
        arrayList.add(6);
        arrayList.add(2);
        arrayList.add(5);

        ArrayList<Integer> uniqueList =
removeDuplicates(arrayList);

        System.out.println("ArrayList with duplicates
removed:");
        for (int num : uniqueList) {
            System.out.print(num + " ");
        }
    }

    public static ArrayList<Integer>
removeDuplicates(ArrayList<Integer> list) {
        Set<Integer> set = new HashSet<>(list);
        return new ArrayList<>(set);
    }
}
```

Output:

ArrayList with duplicates removed:

1 2 5 6 9

6.) Find the missing number in an Array

```
public class Main {  
    public static void main(String[] args) {  
        int[] array = {1, 2, 4, 5, 6}; // Missing number is 3  
        int missingNumber = findMissingNumber(array);  
        System.out.println("The missing number is: " + missingNumber);  
    }  
  
    public static int findMissingNumber(int[] array) {  
        int n = array.length + 1; // Since one number is missing, the length  
should be n+1  
        int totalSum = n * (n + 1) / 2; // Sum of first n natural numbers  
  
        int arraySum = 0;  
        for (int num : array) {  
            arraySum += num;  
        }  
        return totalSum - arraySum;  
    }  
}
```

Output:

The missing number is: 3

7.) Find the largest and smallest element in an Array

```
public class Main {  
    public static void main(String[] args) {  
        int[] array = {5, 2, 9, 1, 6, 3};  
  
        int[] result = findLargestAndSmallest(array);  
  
        System.out.println("Smallest element: " + result[0]);  
        System.out.println("Largest element: " + result[1]);  
    }  
  
    public static int[] findLargestAndSmallest(int[] array) {  
        if (array == null || array.length == 0) {  
            throw new IllegalArgumentException("Array must not be null or  
empty");  
        }  
  
        int smallest = array[0];  
        int largest = array[0];  
  
        for (int num : array) {  
            if (num < smallest) {  
                smallest = num;  
            }  
            if (num > largest) {  
                largest = num;  
            }  
        }  
        return new int[]{smallest, largest};  
    }  
}
```

Output:

Smallest element: 1

Largest element: 9

8.) Search element in an Array

```
public class Main {  
    public static void main(String[] args) {  
        int[] array = {5, 2, 9, 1, 6, 3};  
        int target = 6;  
  
        int index = linearSearch(array, target);  
  
        if (index != -1) {  
            System.out.println("Element " + target + " found at index: " +  
index);  
        } else {  
            System.out.println("Element " + target + " not found in the  
array.");  
        }  
    }  
  
    public static int linearSearch(int[] array, int target) {  
        for (int i = 0; i < array.length; i++) {  
            if (array[i] == target) {  
                return i; // Element found, return index  
            }  
        }  
        return -1; // Element not found  
    }  
}
```

Output:

Element 6 found at index: 4

Element 10 not found in the array

9.) Array consists of integers and special characters, sum only integers

```
public class Main {  
    public static void main(String[] args) {  
        String[] array = {"5", "2", "9", "a", "1", "6", "#", "3"};  
  
        int sum = sumIntegers(array);  
  
        System.out.println("Sum of integers in the array: " + sum);  
    }  
  
    public static int sumIntegers(String[] array) {  
        int sum = 0;  
        for (String element : array) {  
            try {  
                int num = Integer.parseInt(element);  
                sum += num;  
            } catch (NumberFormatException e) {  
                // Ignore non-integer elements  
            }  
        }  
        return sum;  
    }  
}
```

Output:

Sum of integers in the array: 26

10.) Find Minimum and Maximum from an Array

```
public class Main {
    public static void main(String[] args) {
        int[] array = {5, 2, 9, 1, 6, 3};

        // Find maximum and minimum
        int max = findMaximum(array);
        int min = findMinimum(array);

        // Print the results
        System.out.println("Minimum value in the array: " + min);
        System.out.println("Maximum value in the array: " + max);
    }

    public static int findMaximum(int[] array) {
        if (array.length == 0) {
            throw new IllegalArgumentException("Array must not be empty");
        }
        int max = array[0]; // Initialize max to the first element
        for (int i = 1; i < array.length; i++) {
            if (array[i] > max) {
                max = array[i]; // Update max if current element is larger
            }
        }
        return max;
    }

    public static int findMinimum(int[] array) {
        if (array.length == 0) {
            throw new IllegalArgumentException("Array must not be empty");
        }
        int min = array[0]; // Initialize min to the first element
        for (int i = 1; i < array.length; i++) {
            if (array[i] < min) {
                min = array[i]; // Update min if current element is smaller
            }
        }
        return min;
    }
}
```

Output:

Minimum value in the array: 1

Maximum value in the array: 9

11.) Java program to count Odd and Even number from given array

Input: {1,2,3,4,5,6,7,8,9}

```
public class Main {
    public static void main(String[] args) {
        int[] array = {1, 2, 3, 4, 5, 6, 7, 8, 9};

        int[] count = countOddAndEven(array);

        System.out.println("Even numbers count: " + count[1]);
        System.out.println("Odd numbers count: " + count[0]);
    }

    public static int[] countOddAndEven(int[] array) {
        int[] count = new int[2]; // Index 0 for odd count, Index 1 for
even count

        for (int num : array) {
            if (num % 2 == 0) {
                count[1]++; // Increment even count
            } else {
                count[0]++; // Increment odd count
            }
        }
        return count;
    }
}
```

Output:

Even numbers count: 4

Odd numbers count: 5

12.) Java program – input array was given [1,1,2,2,3,4,5,5,6,6], Output – [3,4]

```
import java.util.HashMap;
import java.util.Map;
import java.util.ArrayList;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        int[] array = {1, 1, 2, 2, 3, 4, 5, 5, 6, 6};
        List<Integer> result = findNonRepeatedElements(array);
        System.out.println("Non-repeated elements: " + result);
    }

    public static List<Integer> findNonRepeatedElements(int[]
array) {

        // Step 1: Count occurrences of each element using a
        HashMap
        Map<Integer, Integer> countMap = new HashMap<>();
        for (int num : array) {
            countMap.put(num, countMap.getOrDefault(num, 0) + 1);
        }

        // Step 2: Identify elements with count equal to 1 (non-
        repeated)
        List<Integer> nonRepeatedElements = new ArrayList<>();
        for (Map.Entry<Integer, Integer> entry :
countMap.entrySet()) {
            if (entry.getValue() == 1) {
                nonRepeatedElements.add(entry.getKey());
            }
        }
        return nonRepeatedElements;
    }
}
```

Output :

Non-repeated elements: [3, 4]

Java program to implement **hashCode** and **equals**

```
import java.util.Objects;

public class Student {
    private int id;
    private String name;

    // Constructor
    public Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

    // Getters and setters (omitted for brevity)

    // hashCode method
    @Override
    public int hashCode() {
        return Objects.hash(id, name);
    }

    // equals method
    @Override
    public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (obj == null || getClass() != obj.getClass())
            return false;
        Student student = (Student) obj;
        return id == student.id && Objects.equals(name, student.name);
    }

    public static void main(String[] args) {
        // Creating objects of Student class
        Student student1 = new Student(1, "Alice");
        Student student2 = new Student(2, "Bob");
        Student student3 = new Student(1, "Alice");

        // Testing equals method
        System.out.println("student1.equals(student2): " +
student1.equals(student2)); // Output: false
        System.out.println("student1.equals(student3): " +
student1.equals(student3)); // Output: true

        // Testing hashCode method
        System.out.println("HashCode of student1: " + student1.hashCode());
        System.out.println("HashCode of student2: " + student2.hashCode());
        System.out.println("HashCode of student3: " + student3.hashCode());
    }
}
```

SOURCE 03

Core and Advanced Java Notes

Java concept revision notes

Core & Advanced Java Notes

✓ 1. OOP Concepts

Java is based on Object-Oriented Programming.
The main pillars are:

- **Encapsulation:** Bind data with code (class).
 - **Inheritance:** Reuse code from parent class.
 - **Polymorphism:** Same method, different behaviors.
 - **Abstraction:** Hide internal details, show only essential features.
-

✓ 2. Data Types and Variables

Java supports **primitive** (int, char, boolean, etc.) and **non-primitive** (String, arrays) data types.

Variables are containers to store data.

Java is strongly typed — you must declare the type.

Example: `int age = 25;`

Use **type casting** to convert between types.

✓ 3. Control Flow Statements

Control statements manage program flow.

- **if / if-else:** Conditional execution.
 - **switch:** Multi-branch control.
 - **for / while / do-while:** Looping structures.
 - **break:** Exit loop early, **continue:** skip current iteration.
-

✓ 4. Arrays and Strings

An **array** is a fixed-size collection of elements of the same type.

Syntax: `int[] arr = new int[5];`

A **String** is a sequence of characters and is immutable.
Use methods like `.length()`, `.charAt()`, `.equals()`, etc.
Use `.equals()` to compare values, not `==`.

✓ 5. Constructors

Constructors are special methods to initialize objects.
They have the same name as the class and no return type.
You can overload constructors with different parameters.
Java provides a **default constructor** if none is written.
Called automatically when you create an object.

✓ 6. Inheritance

Inheritance allows one class to reuse another class's code using `extends`.
The **child class** inherits fields and methods of the **parent class**.
Helps with code reusability and method overriding.
Java supports **single**, **multilevel**, and **hierarchical** inheritance.
Use `super` to access parent methods or constructors.

✓ 7. Polymorphism

Polymorphism means “many forms.”

- **Compile-time (overloading)**: Same method name, different parameters.
 - **Runtime (overriding)**: Subclass modifies parent method.
Enhances flexibility and readability.
Helps in dynamic method dispatch (object-based behavior).
-

✓ 8. Abstraction

Abstraction hides complex internal logic and shows only important features.

Achieved using **abstract classes** and **interfaces**.

An abstract class may contain both abstract and regular methods.

Interfaces provide full abstraction (Java 7 and earlier).

Encourages clean and maintainable code.

✓ 9. Encapsulation

Encapsulation binds data and code into a single unit (class).

Use **private** access for fields and **public getters/setters** for access.

Protects data from unauthorized access.

Improves modularity and maintainability.

Supports data hiding and secure programming.

✓ 10. Exception Handling

Java uses exceptions to handle runtime errors gracefully.

Keywords: try, catch, finally, throw, throws.

- **Checked exceptions:** handled at compile-time.
 - **Unchecked exceptions:** occur at runtime.
Avoids program crashes and allows error recovery.
-

✓ 11. Multithreading

Java supports multithreading for concurrent execution.

Create threads using Thread class or Runnable interface.

Thread methods: start(), run(), sleep(), join().

Use **synchronization** to avoid data conflicts.

Improves performance in multi-core systems.

✓ 12. Java Collections Framework

Collections are used to store and manipulate groups of data.

Includes interfaces like List, Set, Map, and classes like ArrayList, HashSet, HashMap.

Collections replace arrays for dynamic data handling.

Supports searching, sorting, inserting, and deleting data efficiently.

All classes are in java.util package.

✓ 13. Wrapper Classes

Wrapper classes convert primitives to objects: `int` → `Integer`, `char` → `Character`, etc.

Useful in collections that require objects.

Supports **autoboxing** (primitive to object) and **unboxing** (object to primitive).

Each primitive has a corresponding wrapper.

Example: `Integer num = 10;`

✓ 14. File Handling (I/O)

Java uses classes like `File`, `FileReader`, `BufferedReader` for file input.

Use `FileWriter`, `BufferedWriter` for writing.

I/O classes are in `java.io` package.

Always handle file exceptions with try-catch.

Supports reading text, binary files, and streams.

✓ 15. Java 8 Features

Java 8 introduced modern programming features:

- **Lambda expressions:** Anonymous methods
- **Stream API:** Functional-style data processing

- **Method references:** Shorter syntax to call methods
 - **Default methods:** Method body in interfaces
Boosts code readability and conciseness.
-

✓ 16. Access Modifiers

Java uses access modifiers to control visibility.

- `private`: within class
 - (default): within package
 - `protected`: within package and subclasses
 - `public`: everywhere
- Helps with data hiding and design security.
-

✓ 17. Static Keyword

`static` belongs to the class, not the object.

Used for variables, methods, and blocks.

Accessed using `ClassName.staticMember`.

Useful for utility methods and constants.

Reduces memory usage when shared across instances.

✓ 18. final Keyword

Used to declare constants, prevent method overriding or inheritance.

- `final` variable: value cannot change
 - `final` method: cannot be overridden
 - `final` class: cannot be extended
- Improves security and design stability.
-

✓ 19. this and super Keyword

- **this:** Refers to the current object. Used to avoid variable name conflicts.
 - **super:** Refers to the parent class. Used to access parent members.
Helpful in constructor chaining and method overriding.
-

✅ 20. Packages and Imports

Packages are used to group related classes.

Use package to define, and import to access them.

Helps in avoiding class name conflicts.

Java has built-in packages like java.util, java.io, etc.

You can create your own packages for better organization.

21. Interfaces vs Abstract Classes

Both are used to achieve abstraction in Java.

Interface: All methods are abstract by default (till Java 7).

Abstract class: Can have both abstract and non-abstract methods.

A class can **implement multiple interfaces** but can **extend only one abstract class**.

Use interfaces when multiple inheritance is needed.

✅ 22. Method Overloading and Overriding

- **Overloading:** Same method name, different parameters (compile-time polymorphism).
- **Overriding:** Subclass changes the behavior of superclass method (runtime polymorphism).
Overloading happens in the **same class**, overriding in **child-parent relationship**.
Java uses @Override annotation for clarity.

✓ 23. Anonymous Inner Class

An inner class without a name, defined and instantiated at the same time.

Mostly used in **event handling** and **functional interfaces**.

Declared using:

```
Runnable r = new Runnable() { public void run() { ... } };
```

✓ 24. Enum in Java

Enums are special classes to define constants.

They are more powerful than final static variables.

You can define methods and constructors in enums.

Example:

```
enum Day { MON, TUE, WED }
```

✓ 25. Varargs (Variable Arguments)

Allows a method to accept **0 or more** arguments.

Declared with ... syntax:

```
void print(String... names) { }
```

Internally treated as an array. Only **one varargs** allowed per method.

✓ 26. Serialization and Deserialization

Serialization converts an object to a byte stream to save to a file or send over a network.

Use ObjectOutputStream for serialization, ObjectInputStream for

deserialization.

Class must implement Serializable interface.

transient keyword is used to skip fields during serialization.

✓ 27. Singleton Design Pattern

Restricts the instantiation of a class to **only one object**.

Used for logging, configuration classes, etc.

Typical implementation:

```
private static Singleton instance;  
public static Singleton getInstance() { ... }
```

✓ 28. Immutable Class

An immutable class's state (data) cannot be changed once created.

String is the best example.

To make a class immutable:

- Make fields final and private
 - Don't provide setters
 - Return copies of mutable objects
-

✓ 29. Java Annotations

Metadata that provides information to the compiler or runtime.

Examples: @Override, @Deprecated, @SuppressWarnings

Used heavily in frameworks like Spring, Hibernate.

You can create your own custom annotations.

✓ 30. Functional Interfaces

An interface with **only one abstract method**.
Used with lambda expressions and method references.
Examples: Runnable, Comparable, Predicate<T>.
Declared with @FunctionalInterface annotation.

✓ 31. Lambda Expressions

Introduced in Java 8 to make code concise.
Syntax: (a, b) -> a + b
Used to implement functional interfaces.
Replaces anonymous inner classes in many cases.

✓ 32. Method References

A shorter way to refer to a method.
Syntax: ClassName::methodName
Example: System.out::println
Used with streams and lambda expressions.

✓ 33. Streams API

Processes collections in a **functional and pipeline** style.
Supports operations like filter(), map(), collect(), forEach()
Introduced in Java 8, great for working with large data sets.

✓ 34. Default and Static Methods in Interfaces

From Java 8 onwards:

- **default:** Methods with body inside interface.

- **static:** Belong to the interface, not instance.
Helps in adding new methods to interfaces without breaking existing code.
-

✓ 35. JVM Architecture

JVM components:

- **Class Loader**
 - **Memory Area (Heap, Stack, etc.)**
 - **Execution Engine**
 - **Garbage Collector**
Responsible for executing Java bytecode on any machine.
-

✓ 36. Garbage Collection

Java handles memory automatically using the **Garbage Collector (GC)**.

It removes unreferenced objects to free up memory.

You can request GC using `System.gc()`, but it's not guaranteed.

Helps avoid memory leaks and crashes.

✓ 37. JIT Compiler

Just-In-Time (JIT) compiler is part of JVM.

It compiles bytecode into native machine code at runtime.

Improves performance by reducing interpretation overhead.

✓ 38. ClassLoader in Java

Loads classes into memory during runtime.

Types: **Bootstrap**, **Extension**, **Application** ClassLoader.

Enables dynamic loading of classes.

✓ 39. Transient Keyword

Used to prevent fields from being serialized.

Transient variables are skipped during object serialization.

Useful for sensitive data like passwords.

✓ 40. Volatile Keyword

Tells JVM that a variable may be changed by different threads.

Ensures visibility of changes to variables across threads.

Used in multithreaded environments.

41. Reflection API

The Reflection API allows you to inspect and modify classes, methods, and fields at runtime.

Used in frameworks (like Spring, Hibernate) for dependency injection and object inspection.

Example:

```
Class<?> c = Class.forName("MyClass");  
Method[] methods = c.getDeclaredMethods();
```

✓ 42. Java Memory Model (JMM)

Defines how threads interact through memory and what behaviors are allowed in multithreading.

Ensures **visibility**, **ordering**, and **atomicity** of operations.

Important for understanding volatile, synchronized, and **thread safety**.

✓ 43. Thread Lifecycle

States: New → Runnable → Running → Blocked/Waiting → Terminated.

Each thread goes through these states managed by the JVM.

Understanding lifecycle helps debug performance and concurrency issues.

✓ 44. Thread Safety and Synchronization

Ensures that shared data is accessed by only one thread at a time.

Use synchronized, ReentrantLock, and atomic classes.

Avoids **race conditions** and inconsistent data.

✓ 45. Executor Framework

Provides thread pool management (ExecutorService) for better multithreading.

Avoids manually creating threads.

Supports Callable, Future, and task scheduling.

✓ 46. Java Generics

Allows you to write **type-safe**, reusable code.

Example: List<String> instead of List.

Supports compile-time checks and eliminates type casting.

✓ 47. Wildcard in Generics (?)

Used when the exact type is unknown.

- `<? extends T>`: accepts T or its subclasses.
 - `<? super T>`: accepts T or its superclasses.
- Enhances flexibility in generic methods/classes.
-

✓ 48. Java Modules (Java 9+)

Modularizes large Java applications.

Introduced `module-info.java` for defining dependencies.

Improves encapsulation and reduces startup time.

✓ 49. Optional Class (Java 8)

A container for possibly-null values to avoid `NullPointerException`.

Methods: `isPresent()`, `ifPresent()`, `orElse()`, `map()`.

Example:

```
Optional<String> name = Optional.ofNullable(null);
```

✓ 50. Comparable vs Comparator

Both are used to sort objects.

- `Comparable`: implemented in the class itself (`compareTo()`).
 - `Comparator`: external custom sorting (`compare()`).
- Allows sorting by multiple fields dynamically.
-

✓ 51. Annotations Processing (APT)

Used to process custom annotations at compile time.
Helps generate code, configuration files, etc.
Frameworks like Lombok and Dagger use it heavily.

✓ 52. Java Records (Java 14+)

A compact syntax for immutable data classes.
Auto-generates constructors, getters, toString(), equals(), hashCode().
Example:

```
record Person(String name, int age) { }
```

✓ 53. Pattern Matching (Java 16+)

Simplifies instanceof checks and type casting.
Example:

```
if (obj instanceof String s) {  
    System.out.println(s.length());  
}
```

✓ 54. Sealed Classes (Java 17)

Restricts which classes can extend or implement a class/interface.
Improves control over class hierarchies.
Example:

```
sealed class Shape permits Circle, Square { }
```

✓ 55. Memory Leaks in Java

Memory that is no longer needed but not collected by GC.
Common in long-lived collections, listeners, static fields.
Use profiling tools like **VisualVM** or **JProfiler** to detect leaks.

✓ 56. `Class.forName` vs `newInstance`

- `Class.forName()` loads class at runtime.
- `.newInstance()` creates object using default constructor.
Useful for dynamic loading in frameworks and JDBC drivers.

✓ 57. Java `WeakReference` & `SoftReference`

Used in caching and memory-sensitive applications.

- **`WeakReference`**: collected eagerly
- **`SoftReference`**: collected only when memory is low
Helps in implementing custom memory management strategies.

✓ 58. Proxy Classes (Dynamic Proxy)

Creates objects at runtime that implement interfaces.

Used in **AOP (Aspect-Oriented Programming)**, like Spring.

Implemented via `java.lang.reflect.Proxy`.

✓ 59. JDBC Architecture

Used to connect Java to relational databases.

Consists of `DriverManager`, `Connection`, `Statement`, `ResultSet`.

Supports prepared statements and batch processing.

60. Java Native Interface (JNI)

Allows Java to call C/C++ native code.

Used in performance-sensitive operations and legacy systems.

Requires writing native libraries using javah and .dll or .so files.

SOURCE 04

C3IT Interview and Test

Test, aptitude, technical, and HR experience

C3IT

Test – One coding Question + Aptitude test (25 questions) + 4 Reading Comprehensions (3 questions in each)

Coding Round – In that we were told to write the pseudo code and then the code which you will be writing on your ide. First you have to get checked with your written pseudo code and code on the paper if that is going to be right then you are allowed to do the coding on the laptop. After that you have to make a pdf file of your pseudo code and outputs.

Question –

Print this star pattern taking the length and width. (not really necessary to give two inputs I took one only)

```
* * * * *
* * *   * * *
* * *   * * *
* *     * *
*       *
* *     * *
* * *   * * *
* * * * *
* * * * *
* * * * *
```

Aptitude round Questions (total 25 questions)

1. If one tyre of a car suddenly gets stolen and after sometime u find the tyre without the screws how will u make ur journey complete?
2. An orange colored glass has Orange juice and white colored glass has Apple juice both of equal volumes. 50ml of the orange juice is taken and poured into the white glass. After that similarly, 50ml from the white glass is poured into the orange glass. Of the two quantities, the amount of apple juice in the orange glass and the amount of orange juice in the white glass, which one is greater and by how much?
3. There are 9 coins. 8 coins weigh 1 gram and 1 coin weighs 2 grams. How will you find out the heavier coin in a minimum number of weighings and how many weighings will be needed? (This question was asked in two companies from me)

The topics from which there were questions – Age Problems, Time Speed Distance, Puzzle questions(majorly this were asked in the aptitude test)

Interview Round-1

1. Introduce yourself.
2. What you know about OOP.
3. What is Encapsulation, Abstraction
4. What is interface
5. Boxing and Unboxing
6. Solid Principles
7. Design Principles
8. Why you have less marks in CCEE
9. How much time will you take to learn a new module if we will provide you with the learning material, Will you be able to deal with foreign client in 6 months.

Interview Round - 2

1. Is your code 100% working
2. What you will do to optimize this code
3. Why you took only one input and do you think its ok to do so
4. Real life example of Polymorphism.
5. What is interface and abstract class.
6. Give an example of real life with respect to Function Overloading.
7. What is CSS, What are different selectors in CSS
8. What is Javascript and JQuery, What is the punch line of JQuery
9. What is Bootstrap how you will define it

Vervotech

(In this I was not selected after 2nd round of interview but adding questions from this also, as I think it will be really helpful)

Test – 3 Coding Questions, 20 Aptitude Question

(All to be done together not like what we had in c3it)

Coding – Q1. Star Pattern

```
*  
**  
***  
****  
*****
```

Q2. The total number of counts for each character occurring in a string.

Example: apoorva

a: 2, p: 1, o:2, r:1, v:1

Q3. Number of combinations for the input of 2 digit number.

For 1: there is no character assigned.



Example: input given – 23 Output – {ad, ae, af, bd, be, bf, cd, ce, cf}

Interview Round-1

1. Introduce yourself
2. What was your role in the project
3. Draw the ER diagram for it.
4. Asked a situational based question based on my project which I was not able to answer properly.
5. What is abstract class can you make an instance of it can you write a constructor in it.
6. What are react components and there types.
7. Difference between the interface and abstract class.
8. Gave a database query to me.

Col1 of table t1

1

0

1

0

0

I had to write a query that can change the 1-> 0 and 0->1.

9. What is indexing
10. Write the code for this star pattern

```

    *
  * *
 * * *
* * * *
* * * * *
```

11. What are searching algorithms what is binary search its time complexity.
12. Why we use Hashset
13. Why you have less marks in this subjects...

Interview Round -2

1. Introduce yourself
2. How was your experience of doing cdac
3. What mistakes you have made why less marks etc.

4. About project what was your role what your group partners did. Is your project deployed
5. Another situational based question was asked regarding the project
6. Questions on OOP and abstract class interfaces.
7. How many coding questions you have solved give a count.
8. Gave me that puzzle of 9 balls 8 have same weight 1 is of less weight minimum iterations to get the less weighted ball.
9. The code for finding prime numbers from a given range of numbers.

Note: There are so many questions I am not able to recall now specially of the aptitude test questions. But I hope this much is enough to give you an idea of interviews and tests.

My experience

This were my 2 company processes which I shared and both were of .net I felt that basically they were checking the logic and the basic concepts you had or not.

PUZZLES were asked majorly. And basic coding questions.

Some extra tips: Just believe in yourself, Don't get frustrated with the long waiting hours to get to an interview, just be calm and as Atul Sir says enjoy the placement process.

All the best EVERYONE.



- Apoorva Vairagi

SOURCE 05

Vaipratech Interview Questions

Java, Spring, streams, email-normalization coding

Vaipratech

Interview Round-1

1. Tell me about yourself
2. What is the aim of your project? What tech stack did you use?
3. Which part of your project did you work on?
4. What is Bean, Service component and RestController?
5. What is the default behaviour of Bean and Service component?
6. Do you know about multi-threading?
7. Are you comfortable with Data Structures?
8. He asked about the code I wrote in the previous round.
 - a. Why did your code pass only 6/10 testcases?
 - b. What changes would you've done if you had more time to complete?

Next, he gave me a problem statement and asked me to write the code for a function named `validEmail(String email)`

Q1) You are given invalid email as input and have to convert it into valid email. If the email has a '.' or a '+' then it's an invalid email.

i) If it has a '.' then remove it,

ii) If it has a '+' then truncate everything that comes after it until '@'

Example: Input – 'janak.dac@gmail.com' Output – 'janakdac@gmail.com'

Input – 'janak+dac@gmail.com' Output – 'janak@gmail.com'

Input – 'ja.na.+kdac@gmail.com' Output – 'jana@gmail.com'

Now, explain the code logic

Q2) Now you are given a list of invalid emails and have to convert them into valid emails by utilising the previous code and display them with their number of occurrences. Write and explain code using hashmap data structure.

Interview Round-2

1. What is Stream and Lambda expressions?

2. What is a functional interface?
Does the map() method of stream api take a functional interface as a parameter?
3. Write a code using stream and lambda expression:
 - a. To find the maximum number from a list of numbers.
 - b. To find the average of the list of numbers.
4. What is dependency injection?
Where have you used it in your project?
5. Write and explain the code to reverse a Singly Linked List
6. What is REST?

Then he asked me to write a POST API which carries a path variable 'productName'. Fetch all the data of this product from the database and display it on the frontend in a table format.

Basically had to write a whole RestController component with necessary annotations and autowire required components like JPA repository.

- Explain what is JPA repository?
- What is ResponseEntity?
- DAO vs DTO
- What is ModelMapper?
- Where do you store your database username, password, url?
Answer: application.properties file
- Role of application.properties file

SOURCE 06

NetSPI Interview Experience

Online test and technical rounds

NETSPI Interview Experience

Round 1: Online Test

Total 6 Sections – 5 based on MCQs | 1 Coding Question

1. Aptitude
2. Communication
3. Networking
4. Pattern Puzzles
5. MySQL
6. Coding Problem based on String

Problem Statement:

Given a string, if 2 or more subsequences of ['a', 'b', 'e', 't', 'i', 'o'] are present, remove them and return the new string.

Sample Input:

"testing is a boring job"

Sample Output:

"sng is a ring j"

Round 2: Technical Interview – 1

Introduction:

I began with a brief introduction about myself and then proceeded to ask three DSA (Data Structures and Algorithms) questions.

1) Maximize Meeting in a Room (max-meetings): [meeting-room](#)

- I provided the logic of the program on paper and then coded it on the interviewer's PC.
- Although not all test cases passed, he was mainly interested in assessing my logical thinking abilities.

2) Binary Tree – Mirror (mirror-tree): [mirror-tree](#)

- I first wrote the logic on paper and then successfully coded it. All test cases passed for this question.

3) Merge Intervals (merge-intervals): [merge-intervals](#)

- I explained the logic of the problem on paper but did not code it.
- The interviewer might have been interested in my understanding of the problem-solving approach rather than the coding implementation.

Java:

In the Java section, the interviewer focused on the following topics:

1. OOPS (Object-Oriented Programming):
 - Definitions and examples were discussed
2. Exceptions:
 - I was asked about the types of exceptions and their handling mechanisms.
3. Global Exceptions:
 - Further discussions were held regarding global exceptions in Java.
4. Interfaces vs Abstract Classes:
 - Differences between interfaces and abstract classes, when to use them.

JavaScript:

- Define JavaScript and its runtime environment.
- Single-threaded or multithreaded nature.
- Difference between Promises and Async Await.
- Callback hell.
- Closure concept in JavaScript.

React:

- Predicting output based on a given code snippet.

Round 3: Technical Interview – 2

1. Project Discussion:
 - Explain your project and your specific contributions to it.
2. Scalability:
 - Strategies for scaling the project.
 - Handling a large number of users efficiently.
3. Error Handling:
 - What if the application crashes while sending emails to a list of users.
4. Database Optimization:
 - Techniques for faster searching in a table with millions of records.

- Management of a large database with millions of records.

Round 4: Technical + HR

Technical Questions:

1. Real DOM vs Virtual DOM in React?
2. Explain session, cookies, localStorage.
3. Difference between sessionStorage and localStorage.
4. Synchronization between tabs: If one tab signs in, how to reflect the sign-in status in other tabs?
5. Coding Question: Two Sum --- [Two-Sum](#)

HR Questions:

1. Puzzle 1: Using two hourglasses (measuring 7 and 4 minutes), how to measure 9 minutes?
2. Given 81 balls where 80 weigh the same and 1 is heavier, with a weighing machine, what's the minimum number of comparisons to find the heavy ball?
3. What unique qualities do you possess?
4. What do you consider toxic in a company culture?

SOURCE 07

Thinkbridge Interview Experience

Screening and DSA round format

Name : Anurag, Company : Thinkbridge

Round 1 : Screening

Asked quants, logical reasoning and picture puzzle questions in 40 min test at E-vision. I submitted the test in 20 minutes.

Round 2 : DSA round

- There were 4 sets (A,B,C,D) of paper each having 1 easy, 1 medium and 1 hard level question. I had set C. Though I found those questions in my set to be easy to moderate level only.
- They wanted to test approach and problem solving ability. I had to type it in Notepad without use of IDE. There was flexibility for document format though they mentioned few points to be present which I covered. So my format for the document was :

1. Problem statement
2. Understanding problem statement and summarizing it in couple of line
3. Few examples to support
4. Few more examples for edge cases
5. Writing the step by step algorithm
6. Now writing the whole code in Java (Though they expected pseudo code)
7. Expected code output
8. Diagrammatic explanation (Which I couldn't do due to lack of time and I mentioned it too)

I followed this approach and the technical lead who took my interview complemented me that presentation and approach for edge case was good. Most probably it helped my interview to be scheduled at start only.

Set C questions :

Easy

1. For encoding an even-length binary string into a sequence of A, T, C, and G, we iterate from left to right and replace the characters as follows 00 is replaced with A 01 is replaced with T 10 is replaced with C 11 is replaced with G Given a binary string S of length N (N is even), find the encoded sequence.

Input Format : First line will contain T, number of test cases. Then the test cases follow. Each test case contains two lines of input. First line contains a single integer N, the length of the sequence. Second line contains binary string S of length N.

Output Format : For each test case, output in a single line the encoded sequence.

Medium 2. Chef has 33 bags that she wants to take on a flight. They weigh A, B, and C kgs respectively. She wants to check-in exactly two of these bags and carry the remaining one bag with her. The airline restrictions says that the total sum of the weights of the bags that are checked-in cannot exceed D kgs and the weight of the bag which is carried cannot exceed E kgs. Find if Chef can take all the three bags on the flight.

Input Format : The first line of the input contains a single integer T denoting the number of test cases. The description of T test cases follows. Each test case contains a single line of input, five space separated

integers A,B,C,D,E.

Output Format : For each test case, output in a single line answer "YES" if Chef can take all the three bags. You may print each character of the string in uppercase or lowercase (for example, the strings "yEs", "yes", "Yes" and "YES" will all be treated as identical).

Subtasks Subtask #1 (100 points): original constraints Sample 1: Input Output 3 1 1 15 5 8 7 6 15 5 8 5 7 15 6 YES NO YES

Hard 3. Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer or transmitted across a network connection link to be reconstructed later in the same or another computer environment. Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Input: A binary tree. Output: Output should be a serialised string and the same binary tree, reconstructed from the serialized string.

Example 1: Input: root = [1,2,3,null, null,4,5] Output: [1,2,3, null, null, 4,5] Example 2: Input: root = [] Output: []

- Personally, I found these question to be easy and similar to questions that were frequently asked in CodeChef weekly contests earlier and so I completed typing everything in Notepad everything in 90 minutes and mailed them pdf.

Round 3 : Technical Interview

- It lasted for about 1 hour and started with my personal introduction and projects I had done at college
- I was asked about the part of project I had worked on. How I approached to design and develop it.
- Then I was asked to run my project on my laptop and the tech lead who took my interview, she tested its UI and asked me to use swagger and postman. Also asked me a few questions on docker and AWS deployment and how I did it.
- Talked out security aspect, JWT, cookies and gave me few situations how I would handle them in my project if required
- I wasn't asked about OOPs but since I mentioned I used SOLID principles while development, asked me how I did it and asked me questions on my backend code
- Had a discussion over server side caching and how I did it in my project and pointed out the vulnerabilities in my project that I needed to be addressed
- Then I was asked about database schema of my project and she asked many situational questions like what I have to add new fields, which table to modify, what should be relations and then asked about triggers that I used
- On DB, I was asked to write join queries on her laptop for given tables and theory about views, stored procedures and DML operations, alias, delete without any clause and truncate, indexes and types and twister questions around it
- Then I was asked basic theory questions on Data structures like Stack, Queue, Tree and Recursion and a question of Valid Parenthesis on paper
- Then at last, we had a discussion over Software Development Life Cycle and how they at their company follows Sprints.

- I also asked her about a few points from their JD and had a friendly chat over day to day work
- Finally I was asked how long I am willing to work since there is no bond on which I said as long as they give me good projects and enough growth opportunities since it is a startup company which started 10 years ago on which she said if I do exceptional, might get opportunity to lead after a year. She had been working since 5 years in that company and told me about her experience in short when she was fresher and had given interview.

Round 4 : HR round

- I was asked to give a 10 minutes HR round and the HR asked me basic HR questions and she took some notes out of it.

This was my 1st interview experience on 14 th March in CCPP. I had lower number of calls and this was my 5th call in CCPP due to lower score in ccee but got selected in it. I took her outlook / feedback on my project. I had answered about 70 to 80 % of the questions correctly and the interviewer was also supportive as she gave me hints as and when needed and it was a good experience.

SOURCE 08

Pattern Technologies Interview Experience

Online test and technical interview

Interview Experience

Pattern Technologies (On-Campus)

Ameya Thakur, B.Tech'23, PGDAC Sept 2023,

Part 1: The Online Test

It was a MCQ based examination with One Coding Problem. There were 5 Sections :

1. Communication

2. Critical Thinking

Communication and Critical Thinking questions were easy and could be answered with basic common sense.

3. Software Engineer

Software Engineer section consisted objective conceptual questions on Data Structures and Algorithms.

4. Coding: Intermediate-level algorithms

Coding problem named Ciphertext was asked. Similar problem:- <https://www.geeksforgeeks.org/caesar-cipher-in-cryptography/>

Only two test cases were visible for the problem. However, there were more hidden test cases which were difficult to think of. So, after submitting, 40% of my total test cases were passed.

5. MySQL.

MySQL questions were based upon the Database queries. Some common concepts :- Joins, group by, not null;



Part 2: Technical Interview I

Interview started with the 'tell me about what you have done so far' question and I answered it explaining about how my interest was culminated towards software programming, the projects I worked upon and the internship that I did. Next after that, the interviewer asked me, why am I appearing for the support role. Though I was more interested in the software developer's role, I mentioned that I want to explore the things in IT world, and for that, I am ready for the Support role as well. After this point, my actual interview was started.

Question 1:

You have given 3 business entities users, articles, and the comments. Users can post articles and the articles can be commented upon. He clarified that he is not much bothered about who is commenting on the articles. He asked me to design the schema for above business requirement and establish the relations if any.

Question. 2

If you want to find out count of comments on all articles, what query will you write.

To make the things clear, I asked whether there is a user entity involvement in this operation or not and he came back as "let's involve the user as well in this". So, the expected columns in the output were :

user_name | article_name | comments_count

I performed the join on three columns based upon the foreign keys and used group by article_name and user_name.

He gave me the couple of hints and I managed to reach till the correct answer.

He also asked me the reason why we need to have group by clause for all the columns mentioned in the select statement. I tried to explain him the logic behind it.

Question 3.

Third question was the extension of the second question.

Suppose, if we want to find out top 5 users of which the articles have the most number of comments, what query will you go for.

The answer was similar to the answer of the second question with the little changes.

Question 4.

Write algorithm to find out angle between two hands of the clock and the constraints were

1. The time will be given as in 24-hour clock.
2. always need to find the shortest angle.
3. Not to use any fixed formula.

Interviewer was not expecting the movement of hour hand when some minutes are passed which he didn't mention in the question. So, it's important to clarify the doubts regarding the question and to discuss your approach in the initial stages. I was going in a complicated way to solve this question and he brought me on the track and gave me the hint to the approach. I wrote the pseudo code based upon that and then started thinking about the constraints. After solving the complete question, he asked me about the different approaches in his mind and their correctness to solve the problem.

These were the 4 major technical problems that was asked to me in the first technical interview. Apart from this, there were some non-technical questions such as, which technology did you learn by yourself recently? For my surprise, at the end, the interviewer conveyed me that he is forwarding me to an another technical interview for the role of Software Developer.

Part 3: Technical Interview II

It was partially based upon the projects I worked upon, one DSA question and rest were the straightforward conceptual questions.

Project based questions : In my introduction, I mentioned about my previous projects and the interviewer was keen to know more about them. Some of the questions asked which I remember :

1. How did you publish the applications on Play Store? Which build tool did you use?
2. Have you published any cross-platform application?
3. Do you know the platforms and technologies with which we can build the applications that can run on Android as well as IOS with the same code?
4. Why did you consider building native Android Applications instead of the web apps.
5. How does the microservices communicate with each other.
6. CDAC project discussion – what role did you play? What is Authentication? What is Authorization? How security works in Spring boot?

7. Have you heard about web workers?

8. When the large-amount of payload comes from the response, how do you manage it on front-end to avoid the inefficiency in rendering?

- I knew the answer in case of the Android Development which I explained to him.

9. DSA Problem –

Problem Statement. - There is a List<Company<List<Car{

```
Id: "",  
type: "",  
name:"""  
}>>>
```

and you have to convert it to List<type<List<Car>>>

Explanation :- There is a list of Company objects which holds the list of car objects that contain the field called the type of the car. Now, we have to segregate the cars based upon their types and return the list of types of cars.

I managed to solve this problem using Hashmap. Interviewer was expecting the solution using Ecma 6 functions which I was not able to recall.

10. Later on he started touching upon the frontend topics. Some of them were

Promises – what problem does the promise solve, what is the solution to avoid the nesting in promise

Async-await – what are they? How they work?

React features – Declarative Vs Imperative, etc.

11. Do you know about the deadlocks in DBMS?

12. Do you have any questions?

- I asked the interviewer about the job role and his role in the company.

Part 4: HR Interview

1. Introduce yourself.

2. your Family Background.

3. Can you show me your projects which you have mentioned in the resume?

4. Why did you choose IT despite being a chemical engineer.

5. What is your biggest achievement so far?

6. What was your biggest challenge so far?

7. Are you comfortable to work in the US shift?

8. Do you have any question for me?

- I asked about the name 'Pattern'. Does it have any relevance with the philosophy or the goal of the company?

HR was happy with my question as she said she had the same question when she joined the company and she answered me how the person who kept the name to the company answered her!

Preparation -

1. Practicing coding problems on coding platforms helped me to clear the coding rounds.

2. Nilesh Sir's Interview questions discussion sessions and his notes were pretty helpful to clear the concepts for the interviews.

3. Conceptual understanding and ability to explain the part which you have done in your project definitely helps.

4. I had taken up the overview of the part which I haven't implemented in our group project such as Spring Security so that I could have something to speak if the question comes.

All the Best!

For any help regarding the interview preparation, you can contact me at amey36thakur@gmail.com

P.S.

Its okay to have less calls. Its okay not to get shortlisted every time. But don't let it affect your interview preparation. Be confident about yourself, and that one call will make everything well.

SOURCE 09

Complete Java/Spring Boot Interview Guide

Scanned 2025 preparation guide

Complete Java/Spring Boot Interview Preparation Guide 2025

Study Plan Overview

- **Beginner Level:** 2-3 weeks preparation
 - **Intermediate Level:** 3-4 weeks preparation
 - **Advanced Level:** 4-6 weeks preparation
-

LEVEL 1: BEGINNER (0-2 Years Experience)

Core Java Fundamentals

Basic Concepts

1. What is the difference between JDK, JRE, and JVM?
2. What are the main features of Java (OOP principles)?
3. What are primitive data types in Java?
4. What is autoboxing and unboxing?
5. What are access modifiers in Java (private, protected, public, default)?
6. What is a package in Java?
7. What is a static keyword? Static variables and methods?
8. What is a Constructor in Java? Types of constructors?
9. Can we use static methods in a Constructor?

OOP Concepts

10. What is inheritance? Types of inheritance in Java?
11. What is polymorphism? Give examples
12. What is encapsulation and abstraction?
13. What is method overloading vs method overriding?
14. When to use Interface vs Abstract Class?
15. What is a Marker Interface? Why use it?

String Handling

16. What is the difference between String, StringBuilder, and StringBuffer?
17. Difference between creating String with literal and new operator

- 18. What is String pool and how does it work?
- 19. What are the advantages of String pool?
- 20. Why String is immutable?

Basic Collections

- 21. What is a Collection in Java?
- 22. Difference between Array and ArrayList?
- 23. What is the difference between ArrayList and LinkedList?
- 24. What is a HashMap? Basic operations?
- 25. What is the difference between HashSet and TreeSet?
- 26. What is an Iterator?
- 27. What is the difference between List and Set?
- 28. Array vs List
- 29. Set vs List

Exception Handling

- 30. What is an Exception? Types of exceptions?
- 31. What is the difference between checked and unchecked exceptions?
- 32. What is try-catch-finally block?
- 33. Can we use try without catch?
- 34. What is throw vs throws?
- 35. What are the use cases of creating user-defined exceptions?
- 36. How to handle user-defined exception?
- 37. What is a NullPointerException & how to prevent it?
- 38. What is a ClassCastException?
- 39. What is Error vs Exception?

Basic Control Structures

- 40. Explain the difference between break and continue statements
- 41. What is the difference between == and .equals()?

Spring Boot Basics

Core Concepts

- 42. What is Spring Boot?
- 43. What are the advantages of Spring Boot?
- 44. What is @SpringBootApplication annotation?
- 45. What is the difference between @Component, @Service, and @Repository?
- 46. What is dependency injection?
- 47. What is @Autowired?
- 48. What are Spring Boot starters?

- 49. What is application.properties file?
- 50. Difference between Spring and Spring Boot?
- 51. What is IOC container in Spring?

REST API Basics

- 52. What is HTTP? Common HTTP methods?
 - 53. What are HTTP status codes (200, 404, 500)?
 - 54. What is REST API?
 - 55. What is JSON?
 - 56. What is @RestController vs @Controller?
 - 57. What is the process of creating a REST API (with example)?
-

LEVEL 2: INTERMEDIATE (2-4 Years Experience)

Advanced Core Java

Collections Deep Dive

- 58. Contract between hashCode() and equals() methods?
- 59. Explain the internal working of HashMap in Java
- 60. What happens on a HashMap collision?
- 61. What is the load factor in HashMap?
- 62. Difference between HashMap, LinkedHashMap, and TreeMap?
- 63. Difference between HashMap, LinkedHashMap, and ConcurrentHashMap
- 64. How can you convert a HashMap into an ArrayList?
- 65. What are the differences between Comparable and Comparator?
- 66. What is a ConcurrentHashMap? How does it work?
- 67. What's the difference between Hashtable and ConcurrentHashMap?
- 68. What is the difference between Vector and ArrayList?
- 69. HashMap vs Hashtable
- 70. ArrayList vs LinkedList

Java 8 Features

- 71. What is a Functional Interface?
- 72. Explain Java 8 features (Lambdas, Streams, Optional)
- 73. Difference between map() vs flatMap() in Java 8?
- 74. What is Stream API and its advantages?
- 75. What is Stream pipeline?
- 76. Difference between intermediate and terminal operators in Stream API
- 77. Using Stream API, find the 2nd highest salary from employee objects list

- 78. Use of Stream API in projects
- 79. Lambda expressions vs Anonymous classes

Advanced OOP

- 80. What is a Functional Interface?
- 81. What are Sealed Classes (Java 17)?
- 82. What is Fail-Fast Iteration & how to handle it?
- 83. What is ConcurrentModificationException? Fail safe and fail fast iterators?
- 84. Final, finally and finalize()?
- 85. Autoboxing vs Unboxing
- 86. What is Cloneable? Deep clones vs shallow clones
- 87. Shallow Copy vs Deep Copy

Memory Management

- 88. Explain Java memory management
- 89. What is garbage collection in Java? Types of GC?
- 90. What is the finalize() method?
- 91. Stack vs Heap Memory Allocation
- 92. How to handle OutOfMemoryException?

Multithreading Basics

- 93. What is multithreading in Java? Thread lifecycle?
- 94. What is synchronization in Java? synchronized keyword?
- 95. What is the difference between wait() and sleep()?
- 96. What is the volatile keyword?
- 97. What is ThreadLocal?
- 98. What is a deadlock in Java? How can it be avoided?
- 99. Thread vs Runnable
- 100. What is the difference between Runnable and Callable?
- 101. Synchronized methods vs Synchronized blocks

Spring Boot Intermediate

Advanced Configuration

- 102. How does Spring Boot auto-configuration work?
- 103. What is the role of @Configuration and @Bean?
- 104. How do profiles work in Spring Boot (@Profile use case)?
- 105. What is @Transactional, and what's the benefit?
- 106. How do you write a global exception handler in Spring Boot?
- 107. Difference between application.properties and application.yml?
- 108. What is the use of CommandLineRunner and ApplicationRunner?

- 109. How does the embedded server (Tomcat) work in Spring Boot?
- 110. How do you override default auto-configurations?
- 111. How do you handle circular dependencies in Spring Boot?

Dependency Injection

- 112. What is constructor injection vs field injection vs setter injection?
- 113. What is `@Qualifier` and `@Primary` annotation?
- 114. If both `@Qualifier` and `@Primary` are used, which one will take precedence?
- 115. How to avoid bean creation failures in dependency injection?
- 116. `@Autowired` vs `@Qualifier`
- 117. `@Primary` vs `@Qualifier`

REST API Advanced

- 118. `@RequestMapping` vs `@GetMapping`
- 119. `@PathVariable` vs `@RequestParam`
- 120. `@PostMapping` vs `@PutMapping`
- 121. PUT vs PATCH
- 122. `@ExceptionHandler` vs `@ControllerAdvice`

Spring Boot Annotations

- 123. `@ComponentScan` vs `@EnableAutoConfiguration`
- 124. `@Configuration` vs `@Bean`
- 125. `@Async` vs `@Scheduled`
- 126. `@Cacheable` vs `@CacheEvict`

Database & JPA Basics

- 127. What is ORM?
- 128. What is JPA? Difference between JPA and Hibernate?
- 129. What is an Entity in JPA?
- 130. What are JPA annotations (`@Entity`, `@Id`, `@GeneratedValue`)?
- 131. What is the difference between `persist()` and `merge()`?
- 132. What are JPA relationships (`@OneToMany`, `@ManyToOne`, etc.)?
- 133. What is JPQL?
- 134. What is lazy loading vs eager loading?
- 135. What is the difference between `save()` and `saveAndFlush()`?
- 136. In Hibernate, what is the difference between `get()` and `load()` methods?

Testing Basics

- 137. What is JUnit? Basic annotations?
- 138. What is `@Mockito`? When to use `@Mock`?

- 139. What is the difference between @Mock, @MockBean, and @Spy?
- 140. How do you test REST controllers using MockMvc?
- 141. What is @SpringBootTest vs @WebMvcTest?

Security Basics

- 142. What is Authentication vs Authorization?
 - 143. What is Spring Security?
 - 144. How do you secure REST APIs using Spring Security?
 - 145. What is BasicAuth vs JWT?
 - 146. What are HTTP status codes: 401, 403, 404, 500, 502, 503?
-

LEVEL 3: ADVANCED (4+ Years Experience)

JVM Internals & Performance

- 147. How does the JVM work internally? (Class loader, memory areas, GC)
- 148. What are the different types of class loaders in Java?
- 149. What are strong, weak, soft, and phantom references in Java?
- 150. How does Java handle memory leaks despite having garbage collector?
- 151. What are different GC algorithms? When to use which?
- 152. How do you tune JVM performance?
- 153. What is JIT compilation?
- 154. What are JVM memory areas (Heap, Stack, Method Area, PC Register)?
- 155. What is metaspace in Java 8+?
- 156. How do you analyze heap dumps and thread dumps?

Advanced Concurrency

- 157. What is the Executor framework in Java?
- 158. What is a Future in Java?
- 159. What is a ThreadPool and why should you use it?
- 160. What are the real differences between ReentrantLock and synchronized?
- 161. How does volatile differ from synchronized?
- 162. What's the difference between thread safety and atomicity?
- 163. When would you use CountdownLatch vs CyclicBarrier?
- 164. What is the difference between busy-waiting vs blocking vs non-blocking calls?
- 165. How does CompletableFuture work internally?
- 166. How do you avoid thread starvation?
- 167. How would you write a thread-safe singleton?
- 168. What is the Producer-Consumer pattern? Implement with BlockingQueue?
- 169. What is ForkJoinPool and how it differs from regular thread pools?

- 170. How do Java Stream operations work internally (parallel vs sequential)?
- 171. What is a live-lock and how is it different from deadlock?
- 172. What is the Java Memory Model (JMM)?
- 173. How does a Semaphore work? Real-life examples?
- 174. What is a BlockingQueue? Types and use cases?

Advanced Spring Boot

- 175. How does Spring manage bean lifecycle? What are the hooks?
- 176. What is a proxy in Spring? JDK vs CGLIB proxies?
- 177. What is the difference between @Autowired, @Inject, and @Resource?
- 178. Can you inject a prototype bean into a singleton? How?
- 179. What is the default scope of a bean in Spring?
- 180. How do you create custom auto-configuration?
- 181. What is Spring Boot Actuator? Important endpoints?
- 182. How do you implement custom health checks?
- 183. What is @ConditionalOnProperty, @ConditionalOnClass?
- 184. How do you implement custom metrics and monitoring?
- 185. What are the Scopes in Spring?
- 186. Prototype vs Request Scope?

Advanced AOP

- 187. What is AOP and how is it implemented in Spring Boot?
- 188. Explain @Before, @After, @Around, @AfterReturning, @AfterThrowing advices
- 189. What is a Pointcut and how do you define it?
- 190. What is a JoinPoint and what data can it provide?
- 191. How do you create custom annotations and intercept them using AOP?
- 192. Real-world use cases of AOP (logging, auditing, security, caching)

Advanced JPA & Database

- 193. What is the N+1 select problem in JPA? How to fix it?
- 194. What are entity graphs and how do they help?
- 195. When would you use @Query vs derived queries vs Criteria API?
- 196. How does optimistic locking work in JPA? @Version annotation?
- 197. What is pessimistic locking? When to use it?
- 198. How do you handle pagination and sorting in Spring Data JPA?
- 199. What are database transactions? ACID properties?
- 200. What is connection pooling? HikariCP configuration?
- 201. How do you implement database migrations with Flyway/Liquibase?
- 202. What is database indexing? Query optimization strategies?

Advanced Spring Security

- 203. How do you implement OAuth2 and JWT in Spring Boot?
- 204. What is a stateless session? How does JWT help?
- 205. How do you implement two-factor authentication (2FA)?
- 206. How does CSRF protection work in Spring?
- 207. What is the difference between pre-auth and post-auth filters?
- 208. How does SecurityFilterChain work in Spring Boot 3+?
- 209. What is UserDetailsService and how do you implement it?
- 210. How do you implement role-based access control (RBAC)?
- 211. How do you implement method-level security?
- 212. How do you secure microservices communication?
- 213. How to implement OAuth authentication in Spring Boot?

Microservices Architecture

- 214. What is the difference between monolithic and microservices architecture?
- 215. How do microservices communicate with each other?
- 216. What is RestTemplate? How to use it?
- 217. What is an API Gateway? Why is it used?
- 218. What is service discovery?
- 219. How do you implement resilience patterns (Retry, Circuit Breaker, Bulkhead)?
- 220. What is the Circuit Breaker pattern? Libraries like Hystrix/Resilience4j?
- 221. How do you implement API rate limiting?
- 222. What happens when one microservice becomes slow? Isolation strategies?
- 223. How do you handle versioning of REST APIs?
- 224. How do you implement distributed logging and tracing (ELK, Zipkin)?
- 225. What's the difference between sync and async communication?
- 226. What is eventual consistency? CAP theorem?
- 227. How do you implement saga pattern for distributed transactions?
- 228. How do you handle data consistency across microservices?
- 229. Microservices architecture vs Monolithic architecture
- 230. What is producer and consumer applications?

Apache Kafka

- 231. What are the core components of Kafka?
- 232. What is a Partition in Kafka? How does partitioning work?
- 233. What is a Kafka Topic, Producer, Consumer?
- 234. What are Consumer Groups? How does rebalancing work?
- 235. What if a Kafka consumer keeps retrying endlessly? Dead letter queue?
- 236. How do you ensure message ordering in Kafka?
- 237. What is Kafka Connect? Use cases?
- 238. How do you handle exactly-once delivery semantics?
- 239. What is Kafka Streams? When to use it?
- 240. How do you monitor Kafka performance?

- 241. Explain Kafka and how it handles real-time message processing

Advanced Testing

- 242. What's the difference between unit, integration, and E2E testing?
 - 243. How do you mock external REST APIs in tests?
 - 244. What are TestContainers? How to use with Spring Boot?
 - 245. How do you write parameterized tests in JUnit 5?
 - 246. What is the role of `@DataJpaTest`, `@WebMvcTest`, `@SpringBootTest`?
 - 247. How do you implement test data builders and object mothers?
 - 248. What is contract testing? Pact framework?
 - 249. How do you implement performance testing?
 - 250. What is mutation testing?
-

EXPERT LEVEL (5+ Years / Architect Level)

System Design & Architecture

- 251. How do you design a **scalable e-commerce** system?
- 252. How do you implement **distributed caching** (Redis, Hazelcast)?
- 253. What is **CQRS** pattern? When to use it?
- 254. How do you implement **event sourcing**?
- 255. What is **database sharding**? Strategies?
- 256. How do you design for **high availability**?
- 257. What is **load balancing**? Different strategies?
- 258. How do you implement **API Gateway** patterns?
- 259. What is **strangler fig pattern** for legacy system migration?
- 260. How do you design **monitoring and alerting** systems?
- 261. How does **Redis caching** work, and when should it be used?

DevOps & Cloud

- 262. How do you containerize a Spring Boot app with Docker?
- 263. What is Docker compose? Multi-stage Docker builds?
- 264. How do you create a Jenkins pipeline for Java apps?
- 265. What is the difference between blue-green and rolling deployments?
- 266. How do you implement zero-downtime deployment?
- 267. What is infrastructure as code? Terraform basics?
- 268. How do you handle secrets management (Vault, K8s secrets)?
- 269. What is monitoring and observability? Prometheus, Grafana setup?
- 270. JAR vs WAR files
- 271. Maven vs Gradle

- 272. Continuous Integration vs Continuous Deployment

Kubernetes & Cloud Native

- 273. What is Kubernetes? Pods, Services, Deployments?
- 274. How do you deploy Spring Boot apps on Kubernetes?
- 275. What is a sidecar container? Use cases?
- 276. How do you handle configuration in K8s (ConfigMaps, Secrets)?
- 277. What is service mesh? Istio basics?
- 278. How do you implement autoscaling in K8s?
- 279. What is ingress controller?
- 280. How do you implement health checks in K8s?
- 281. What is Helm? Chart management?
- 282. Cloud-native patterns for Java applications?

Performance & Troubleshooting

- 283. How do you profile Java applications?
- 284. How to debug memory leaks?
- 285. How to analyze thread dumps?
- 286. Gateway Timeout vs Service Unavailable - troubleshooting?
- 287. How do you optimize database queries?
- 288. How to handle high CPU usage in production?
- 289. What tools do you use for application monitoring?
- 290. How do you implement distributed tracing?
- 291. Service Not Found – how to troubleshoot?
- 292. How to debug locally & remotely?

MODERN JAVA FEATURES (2024-2025)

Java 21+ Features

- 293. What are Virtual Threads vs Platform Threads?
- 294. How do Virtual Threads improve performance in I/O-heavy applications?
- 295. When should you NOT use Virtual Threads?
- 296. How to create Virtual Threads using `Executors.newVirtualThreadPerTaskExecutor()`?
- 297. How do Record Patterns simplify switch expressions?
- 298. What are the benefits of pattern matching for instanceof?
- 299. What are `getFirst()` and `getLast()` methods in collections?
- 300. How do String Templates improve string building over concatenation?

Reactive Programming

- 301. What is reactive programming?
 - 302. Difference between Mono and Flux?
 - 303. How to handle backpressure in reactive streams?
 - 304. When to use reactive vs traditional Spring MVC?
 - 305. How to chain multiple asynchronous operations with `CompletableFuture`?
 - 306. Reactive programming vs Imperative programming
-



SCENARIO-BASED QUESTIONS

Performance Issues

- 307. Your Spring Boot application takes 45 seconds to start up in production, but only 5 seconds locally. How do you investigate?
- 308. After deploying to production, your REST APIs are responding in 5+ seconds. What's your debugging approach?
- 309. Your application works fine with 100 users but crashes with `OutOfMemoryError` with 1000 users. How do you solve this?
- 310. Your microservice occasionally returns 503 Service Unavailable during peak traffic. What could cause this?

Security Scenarios

- 311. Users get logged out every 30 minutes despite JWT tokens being valid for 24 hours. What's happening?
- 312. Your API is being hit by 10,000 requests per second from the same IP. How do you implement rate limiting?
- 313. You need to implement SSO for 20 different applications. How do you design this?

Database Issues

- 314. Your e-commerce app takes 30 seconds to load product catalog with 10,000 products. How do you optimize?
- 315. You're getting '`LazyInitializationException`' in production but not in tests. Why and how do you fix it?
- 316. Two users are trying to update the same product simultaneously. How do you handle this?

Microservices Challenges

317. You need to migrate a monolithic application with 2 million users to microservices. What's your strategy?
318. Service A depends on Service B, which depends on Service C. Service C is down. How do you prevent cascade failures?
319. You have an order that touches 5 different microservices. If one fails after others succeed, how do you handle this?
-

BEHAVIORAL & LEADERSHIP QUESTIONS

Technical Leadership

320. How do you evaluate new technologies for adoption?
321. Describe your approach to technical debt management
322. How do you handle disagreements in technical design?
323. What's your strategy for mentoring junior developers?

Problem Solving

324. Describe a complex technical problem you solved
325. How did your technical decisions impact business metrics?
326. Describe a time you prevented a major production issue
327. How do you prioritize technical improvements vs new features?

Communication & Collaboration

328. How do you explain technical concepts to non-technical stakeholders?
329. Describe a time you had to work with a difficult team member
330. How do you handle pressure and tight deadlines?
-

PREPARATION STRATEGY

Week 1-2: Core Java Mastery

- Focus on questions 1-100
- Practice basic coding problems
- Understand OOP principles thoroughly
- Master collections and exception handling

Week 3-4: Spring Boot Deep Dive

- Focus on questions 101-200
- Build sample REST APIs
- Practice dependency injection scenarios
- Understand Spring Security basics

Week 5-6: Advanced Concepts

- Focus on questions 201-300
- Study system design patterns
- Practice microservices scenarios
- Learn performance optimization

Week 7-8: Modern Features & Practice

- Study Java 21+ features
- Practice scenario-based questions
- Mock interviews with peers
- Review real project experiences

Final Week: Integration & Review

- Practice all levels together
 - Focus on weak areas
 - Prepare STAR format answers for behavioral questions
 - Review company-specific requirements
-



QUICK REFERENCE CHARTS

Key Annotations Comparison

Annotation	Purpose	Scope
@Component	Generic stereotype	Class level
@Service	Business logic	Class level
@Repository	Data access	Class level
@Controller	Web requests	Class level
@RestController	REST APIs	Class level

HTTP Status Codes

Code	Meaning	When to Use
200	OK	Successful request
201	Created	Resource created
400	Bad Request	Invalid input
401	Unauthorized	Authentication required
403	Forbidden	Access denied
404	Not Found	Resource doesn't exist
500	Internal Server Error	Server-side error

Collection Performance

Operation	ArrayList	LinkedList	HashMap
Add	O(1)	O(1)	O(1)
Get	O(1)	O(n)	O(1)
Remove	O(n)	O(1)	O(1)
Search	O(n)	O(n)	O(1)



FINAL TIPS

Before Interview

1. Research the company's tech stack
2. Prepare 2-3 detailed project examples
3. Practice coding on whiteboard/shared screen
4. Review recent Java/Spring releases

During Interview

1. Think out loud while solving problems
2. Ask clarifying questions
3. Explain trade-offs and alternatives
4. Show willingness to learn

After Interview

1. Send thank you email within 24 hours
 2. Address any gaps from the interview
 3. Follow up professionally
-

Good luck with your interview preparation! 🚀

Remember: Consistent practice and understanding concepts deeply is more important than memorizing answers.

SOURCE 10

Top 100 DSA Interview Questions

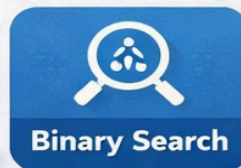
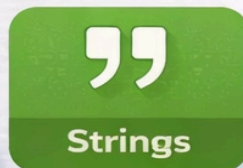
Curated DSA practice list

TOP 100

DSA INTERVIEW

QUESTIONS

- ✓ Most Asked Coding Questions
- ✓ Perfect for Freshers



- ✓ Covers Basics to Stacks & Queues
- ✓ Most Asked Coding Questions
- ✓ Perfect for Freshers

TOP 100 DSA INTERVIEW QUESTIONS

1. Basics (10)

1. Reverse a String
 2. Reverse a Number
 3. Palindrome Check (String & Number)
 4. Armstrong Number
 5. Count digits in a number
 6. GCD / HCF of two numbers
 7. Prime Number Check (Optimized)
 8. Print primes in range
 9. Swap two numbers
 10. Decimal to Binary
-

2. Arrays (25)

11. Two Sum
12. Move Zeroes to End
13. Maximum Consecutive Ones

14. Best Time to Buy & Sell Stock
15. Maximum Subarray (Kadane's Algorithm)
16. Rotate Array
17. Majority Element
18. Missing Number
19. Find Single Number
20. Find Duplicates in Array
21. Find Disappeared Numbers
22. Contains Duplicate
23. Rearrange Array by Sign
24. Merge Intervals
25. Insert Interval
26. Product of Array Except Self
27. Set Matrix Zeroes
28. Spiral Matrix
29. Merge Sorted Array
30. Next Permutation

- 31. Leaders in Array
 - 32. Trapping Rain Water
 - 33. Maximum Product Subarray
 - 34. Longest Consecutive Sequence
 - 35. Three Sum
-

3. Strings (20)

- 36. Valid Anagram
- 37. Reverse Words in String
- 38. Longest Common Prefix
- 39. Longest Palindrome (length)
- 40. Valid Parentheses
- 41. First Occurrence in String (strstr)
- 42. Count & Say
- 43. Roman to Integer
- 44. Integer to Roman

- 45. Longest Substring Without Repeating Characters
 - 46. Isomorphic Strings
 - 47. Minimum Window Substring
 - 48. Group Anagrams
 - 49. Palindrome Check
 - 50. String Compression
 - 51. Longest Palindromic Substring
 - 52. Remove Adjacent Duplicates
 - 53. Word Break
 - 54. Length of Last Word
 - 55. First Unique Character
-

4. Sliding Window & Two Pointers (15)

- 56. Maximum Sum Subarray (Size K)
- 57. First Negative in Window
- 58. Longest Substring Without Repeating Characters

59. Longest Repeating Character Replacement
 60. Minimum Window Substring
 61. Fruits Into Baskets
 62. Permutation in String
 63. Count Subarrays with Sum K
 64. Longest Subarray with Sum K
 65. Binary Subarrays with Sum
 66. Container With Most Water
 67. Trapping Rainwater (Two Pointer)
 68. Pair Sum (Two Pointer)
 69. Minimum Size Subarray Sum
 70. Longest Ones After K Flips
-

5. Binary Search & Sorting (10)

71. Binary Search
72. First & Last Position of Element

- 73. Search in Rotated Sorted Array
 - 74. Find Peak Element
 - 75. Find Minimum in Rotated Array
 - 76. Ceil & Floor in Sorted Array
 - 77. Search Insert Position
 - 78. Square Root using Binary Search
 - 79. Count Occurrences in Sorted Array
 - 80. Kth Largest Element in Array
-

6. Hashing & Prefix Sum (10)

- 81. Two Sum (Hashing Approach)
- 82. Subarray Sum Equals K
- 83. Longest Subarray with Zero Sum
- 84. Count Distinct Elements in Window
- 85. Group Anagrams
- 86. Longest Consecutive Sequence

- 87. First Repeating Element
 - 88. First Non-Repeating Character
 - 89. Count Pairs with Given Sum
 - 90. Subarray with Given XOR
-

7. Linked List (5)

- 91. Reverse Linked List
 - 92. Middle of Linked List
 - 93. Detect Cycle in Linked List
 - 94. Merge Two Sorted Lists
 - 95. Remove Nth Node from End
-

8. Stack & Queue (5)

- 96. Valid Parentheses
- 97. Next Greater Element
- 98. Min Stack

99. Implement Queue using Stack

100. Sliding Window Maximum

SOURCE 11

SQL Interview Question Bank

Beginner through advanced SQL questions

Basic Level (Beginner)

1.	What is SQL?
2.	What is a Database?
3.	What are the types of SQL commands?
4.	What is Primary Key?
5.	What is Foreign Key?
6.	What is UNIQUE Key?
7.	What is the difference between Primary Key and UNIQUE Key?
8.	What is NOT NULL constraint?
9.	What is Default Constraint?
10.	What is the difference between DELETE, TRUNCATE, and DROP?
11.	What is the difference between WHERE and HAVING?
12.	What are Joins in SQL?
13.	What is INNER JOIN?
14.	What is LEFT JOIN?
15.	What is RIGHT JOIN?
16.	What is FULL JOIN?
17.	What is Self Join?
18.	What is Cross Join?
19.	What is Union and Union All?
20.	What is the difference between UNION and UNION ALL?
21.	What is Normalization?
22.	What is Denormalization?
23.	What is the difference between CHAR and VARCHAR?
24.	What is the difference between SQL and MySQL?
25.	What is Auto Increment in SQL?

Intermediate Level

26.	What is Subquery?
27.	What is Nested Query?
28.	What is Correlated Subquery?
29.	What is Group By in SQL?
30.	What is the difference between Group By and Order By?
31.	What is the use of LIMIT in SQL?
32.	How to find the Second Highest Salary in SQL?
33.	How to find Duplicate Records in a table?
34.	What is CTE (Common Table Expression)?
35.	What is Temporary Table in SQL?
36.	What is Window Function in SQL?
37.	What is the difference between ROW_NUMBER(), RANK(), and DENSE_RANK()?
38.	What is CASE Statement in SQL?
39.	What is COALESCE in SQL?
40.	What is NVL Function in SQL?
41.	What is Indexing in SQL?
42.	What is Clustered Index?
43.	What is Non-Clustered Index?
44.	What is the difference between Clustered and Non-Clustered Index?
45.	What is View in SQL?
46.	What is the difference between View and Table?
47.	What is Stored Procedure?
48.	What is the difference between Function and Stored Procedure?
49.	What is Trigger in SQL?
50.	What is Cursor in SQL?

Advanced Level

51.	What is the ACID Property in SQL?
52.	What is a Transaction in SQL?
53.	What is the difference between COMMIT and ROLLBACK?
54.	What is Savepoint in SQL?
55.	What is the difference between IN and EXISTS?
56.	What is the difference between DELETE and TRUNCATE?
57.	What is Index Fragmentation?
58.	What is the difference between RANK() and DENSE_RANK()?
59.	How to fetch common records from two tables?
60.	What is the difference between UNION and JOIN?
61.	What is Pivot Table in SQL?
62.	What is Case Sensitivity in SQL?
63.	How to find the Nth Highest Salary?
64.	How to get First 3 Maximum Salaries?
65.	What is the difference between Drop, Delete, and Truncate?
66.	How to calculate Age from Date of Birth in SQL?
67.	What is Recursive Query in SQL?
68.	What is the difference between Temporary Table and CTE?
69.	How to find Odd and Even records in SQL?
70.	What is JSON in SQL?
71.	What is XML in SQL?
72.	How to handle NULL values in SQL?
73.	What is Dynamic SQL?
74.	How to calculate Percentage in SQL?
75.	How to find the Employees who earn more than their Manager?

Real-Time Scenarios

76.	How to find Duplicate Emails in the Employee Table?
77.	How to get the Highest Salary in each Department?
78.	How to find Employees joined in the last 3 months?
79.	How to Display the First 5 Records in SQL?
80.	How to find the Number of Employees in each Department?
81.	How to find the Last 3 Records in SQL?
82.	How to find Employees without Managers?
83.	How to find the First Name starting with 'A'?
84.	How to fetch Alternate Rows from a table?
85.	How to swap two columns in SQL?
86.	How to display the Duplicate Records with their Count?
87.	How to find the Highest Salary without using MAX()?
88.	How to fetch common records from two tables without JOIN?
89.	How to delete Duplicate Records from a table?
90.	How to find the Department with the highest Employee Count?

Optimization Techniques

91.	How to Optimize SQL Queries?
92.	What is Query Execution Plan?
93.	How to Improve Query Performance?
94.	What is Indexing?
95.	What is Table Partitioning?
96.	How to Avoid Deadlocks in SQL?
97.	What is the use of EXISTS in SQL?
98.	What is Query Optimization?
99.	What is the Difference Between Stored Procedure and Function in SQL?
100.	What is the difference between OLTP and OLAP in SQL?

Basic Level (Beginner)

1. What is SQL?

SQL (Structured Query Language) is a standard programming language used to interact with relational databases. It is used to store, retrieve, update, and delete data. SQL is also used to create and modify database structures such as tables, views, and indexes.

Example:

```
SELECT * FROM Employees;
```

This query retrieves all the records from the Employees table.

2. What is a Database?

A database is an organized collection of data that is stored and managed electronically. It allows users to efficiently store, retrieve, update, and manage data. Databases are used to handle large amounts of information in various applications such as websites, business systems, and applications.

Example:

A customer database in an e-commerce website may store customer details like name, email, contact number, and purchase history.

3. What are the types of SQL commands?

SQL commands are categorized into five types based on their functionality:

1. **DDL (Data Definition Language)** – Defines the structure of the database.
 - CREATE, ALTER, DROP, TRUNCATE
2. **DML (Data Manipulation Language)** – Manages data stored in the database.
 - SELECT, INSERT, UPDATE, DELETE
3. **DCL (Data Control Language)** – Controls access to the data.
 - GRANT, REVOKE
4. **TCL (Transaction Control Language)** – Manages transactions in the database.
 - COMMIT, ROLLBACK, SAVEPOINT
5. **DQL (Data Query Language)** – Retrieves data from the database.
 - SELECT

4. What is Primary Key?

A Primary Key is a column or a combination of columns in a table that uniquely identifies each row in that table. It does not allow NULL values and must always contain unique values.

Key Features:

- Uniquely identifies each record
- Cannot have duplicate values
- Cannot contain NULL values
- Only one primary key is allowed per table

Example:

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Age INT  
);
```

Here, EmployeeID is the primary key that uniquely identifies each employee.

5. What is Foreign Key?

A Foreign Key is a column or combination of columns in one table that refers to the Primary Key in another table. It is used to create a relationship between two tables and enforce referential integrity.

Key Features:

- Establishes a relationship between two tables
- Can contain duplicate values
- Can accept NULL values
- Helps maintain data consistency

Example:

```
CREATE TABLE Departments (  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(50)  
);  
  
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    DepartmentID INT,  
    FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID)  
);
```

Here, DepartmentID in the Employees table is a foreign key that references the DepartmentID column in the Departments table.

6. What is UNIQUE Key?

A UNIQUE Key is a constraint that ensures all values in a column or combination of columns are distinct across all rows in the table. It prevents duplicate values but allows NULL values (only one NULL value in most databases).

Key Features:

- Ensures uniqueness of each record in the column
- Allows one NULL value (depending on the database)
- Multiple UNIQUE keys can be defined in a table
- Helps maintain data integrity

Example:

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    Email VARCHAR(100) UNIQUE,  
    Name VARCHAR(50)  
);
```

Here, the Email column has a UNIQUE constraint, ensuring no two employees can have the same email address.

7. What is the Difference Between Primary Key and UNIQUE Key?

Primary Key	UNIQUE Key
Uniquely identifies each row in a table	Ensures all values in the column are unique
Does not allow NULL values	Allows one NULL value (in most databases)
Only one primary key is allowed per table	Multiple UNIQUE keys can be defined in a table
Automatically creates a unique clustered index	Creates a unique non-clustered index
Used to uniquely identify a record	Used to enforce uniqueness in a column without being a primary identifier

8. What is NOT NULL Constraint?

The NOT NULL constraint ensures that a column cannot have NULL values. It is used to enforce that every row must have a value in that column.

Key Features:

- Prevents insertion of NULL values
- Ensures mandatory fields have data
- Can be applied to one or more columns

Example:

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    Email VARCHAR(100)  
);
```

In this example, the Name column cannot have NULL values, while the Email column can accept NULL values.

9. What is Default Constraint?

The Default Constraint provides a default value for a column when no value is specified during the insertion of a new record.

Key Features:

- Automatically assigns a default value if no value is provided
- Helps avoid NULL values in specific columns
- Can be applied to any data type

Example:

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Salary DECIMAL(10, 2) DEFAULT 5000  
);
```

In this example, if no salary is provided while inserting a record, the Salary column will automatically be set to 5000.

10. What is the Difference Between DELETE, TRUNCATE, and DROP?

Command	Function	Can Rollback	Affects Structure	Speed
DELETE	Removes specific rows based on a condition using the WHERE clause	Yes (with COMMIT/ROLLBACK)	No	Slow (Row-by-row deletion)
TRUNCATE	Removes all rows from the table without a condition	No	No	Faster than DELETE
DROP	Deletes the entire table including data and structure	No	Yes (Removes table structure)	Fastest

Example:

```
DELETE FROM Employees WHERE EmployeeID = 101; -- Deletes specific row
TRUNCATE TABLE Employees; -- Deletes all rows
DROP TABLE Employees; -- Deletes table completely
```

11. What is the Difference Between WHERE and HAVING?

Clause	Purpose	Used With	Filter Type	Execution Order
WHERE	Filters rows before grouping	SELECT, UPDATE, DELETE	Row-level filter	Applied before GROUP BY
HAVING	Filters groups after grouping	SELECT with GROUP BY	Group-level filter	Applied after GROUP BY

Example:

```
-- WHERE Example
SELECT Name, Salary
FROM Employees
WHERE Salary > 5000;

-- HAVING Example
SELECT Department, AVG(Salary) AS AvgSalary
FROM Employees
GROUP BY Department
HAVING AVG(Salary) > 5000;
```

In the WHERE clause, filtering is applied before grouping, while HAVING filters the aggregated result.

12. What are Joins in SQL?

Joins in SQL are used to combine data from two or more tables based on a related column between them.

Types of Joins:

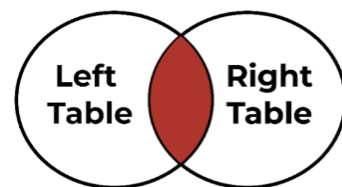
1. **INNER JOIN** – Returns only matching rows from both tables.
2. **LEFT JOIN** – Returns all rows from the left table and matching rows from the right table.
3. **RIGHT JOIN** – Returns all rows from the right table and matching rows from the left table.
4. **FULL JOIN** – Returns all rows from both tables (matching and non-matching).
5. **SELF JOIN** – Joins a table with itself.
6. **CROSS JOIN** – Returns the Cartesian product of both tables (all possible combinations).

13. What is INNER JOIN?

INNER JOIN is used to combine rows from two or more tables based on a matching condition between them. It returns only those records where the specified condition is true in both tables.

Key Features:

- Returns matching rows from both tables
- Ignores unmatched rows
- Most commonly used type of join



Inner Join

Syntax:

```
SELECT table1.column1, table2.column2
FROM table1
INNER JOIN table2 ON table1.common_column = table2.common_column;
```

Example:

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
INNER JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

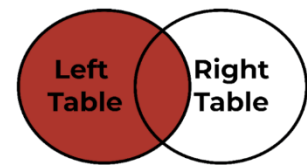
This query returns the employee names along with their department names where the DepartmentID is common in both tables.

14. What is LEFT JOIN?

LEFT JOIN is used to return all records from the left table and the matching records from the right table. If no match is found, the result will contain NULL values from the right table.

Key Features:

- Returns all rows from the left table
- Returns matching rows from the right table
- Displays NULL for non-matching rows from the right table



Left Join

Syntax:

```
SELECT table1.column1, table2.column2
FROM table1
LEFT JOIN table2 ON table1.common_column = table2.common_column;
```

Example:

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
LEFT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

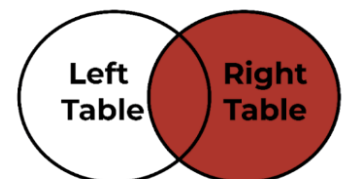
This query returns all employee names from the Employees table, and if the department is not assigned, the Department Name will be displayed as NULL.

15. What is RIGHT JOIN?

RIGHT JOIN is used to return all records from the right table and the matching records from the left table. If no match is found, the result will contain NULL values from the left table.

Key Features:

- Returns all rows from the right table
- Returns matching rows from the left table
- Displays NULL for non-matching rows from the left table



Right Join

Syntax:

```
SELECT table1.column1, table2.column2
FROM table1
RIGHT JOIN table2 ON table1.common_column = table2.common_column;
```

Example:

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
RIGHT JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

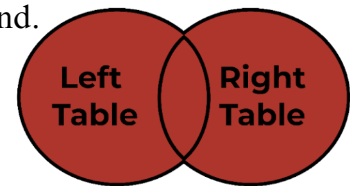
This query returns all department names from the Departments table, and if no employee is assigned to a department, the Employee Name will be displayed as NULL.

16. What is FULL JOIN?

FULL JOIN combines the results of both LEFT JOIN and RIGHT JOIN. It returns all records from both tables, with matching rows from both sides where available. If there is no match, the result will contain NULL values on the side where no match was found.

Key Features:

- Returns all rows from both tables
- Displays NULL where there is no match
- Useful to find unmatched records in both tables



Full Join

Syntax:

```
SELECT table1.column1, table2.column2
FROM table1
FULL JOIN table2 ON table1.common_column = table2.common_column;
```

Example:

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
FULL JOIN Departments ON Employees.DepartmentID = Departments.DepartmentID;
```

This query returns all employee names and department names, including those where there is no matching DepartmentID between the two tables.

17. What is Self Join?

Self Join is a type of join where a table is joined with itself to compare rows within the same table. It is used when a table contains a hierarchical relationship or when comparing values in the same table.

Key Features:

- Joins a table with itself
- Requires table aliases to differentiate table instances
- Used to compare rows within the same table

Syntax:

```
SELECT A.column1, B.column2
FROM table_name A
JOIN table_name B ON A.common_column = B.common_column;
```

Example:

```
SELECT E1.EmployeeName AS Employee, E2.EmployeeName AS Manager
FROM Employees E1
JOIN Employees E2 ON E1.ManagerID = E2.EmployeeID;
```

In this example, the Employees table joins with itself to show the employee's name along with their manager's name based on the ManagerID column.

18. What is Cross Join?

Cross Join returns the Cartesian product of two tables, meaning it combines every row from the first table with every row from the second table. It does not require any condition.

Key Features:

- Combines all rows from both tables
- Number of rows in the result = (Rows in Table 1) × (Rows in Table 2)
- Can produce large result sets if tables have many rows

Syntax:

```
SELECT table1.column1, table2.column2
FROM table1
CROSS JOIN table2;
```

Example:

```
SELECT Employees.Name, Departments.DepartmentName
FROM Employees
CROSS JOIN Departments;
```

This query returns all possible combinations of Employees and Departments, with every employee paired with every department.

19. What is Union and Union All?

UNION and UNION ALL are used to combine the result sets of two or more SELECT statements.

UNION	UNION ALL
Removes duplicate rows	Includes duplicate rows
Slower due to duplicate removal	Faster as no duplicate removal
Automatically sorts the result set	Does not sort the result set
Syntax: UNION	Syntax: UNION ALL

Syntax:

```
SELECT column1 FROM table1
UNION
SELECT column1 FROM table2;
```

Example (UNION):

```
SELECT Name FROM Employees
UNION
SELECT Name FROM Managers;
```

This query combines employee and manager names without duplicates.

Example (UNION ALL):

```
SELECT Name FROM Employees
UNION ALL
SELECT Name FROM Managers;
```

This query combines employee and manager names, including duplicates.

20. What is the difference between UNION and UNION ALL?

Criteria	UNION	UNION ALL
Duplicates	Removes duplicate rows	Includes all duplicate rows
Performance	Slower (due to duplicate removal)	Faster (no duplicate removal)
Sorting	Automatically sorts the result set	Does not sort the result set
Usage	Used when duplicate data is not required	Used when duplicate data needs to be preserved
Syntax	SELECT column FROM table1 UNION SELECT column FROM table2;	SELECT column FROM table1 UNION ALL SELECT column FROM table2;

21. What is Normalization?

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It involves dividing large tables into smaller related tables and defining relationships between them.

Key Features:

- Reduces data redundancy
- Improves data consistency
- Simplifies data maintenance
- Increases data integrity

Types of Normalization:

1. **1NF (First Normal Form)** – Eliminates duplicate columns and ensures each column contains atomic values.
2. **2NF (Second Normal Form)** – Ensures no partial dependency by making all non-key attributes fully dependent on the primary key.
3. **3NF (Third Normal Form)** – Removes transitive dependencies where non-key columns depend on other non-key columns.
4. **BCNF (Boyce-Codd Normal Form)** – Ensures that every determinant is a candidate key.

Example:

Unnormalized Table:

EmployeeID	Employee Name	Department	DepartmentLocation
101	Vinay	IT	Bangalore
102	Awadhesh	IT	Bangalore

Normalized Table (1NF & 2NF):

Employee Table:

EmployeeID	EmployeeName	DepartmentID
101	Vinay	1
102	Awadhesh	1

Department Table:

DepartmentID	DepartmentName	Location
1	IT	Bangalore

Normalization improves the efficiency and consistency of the database.

22. What is Denormalization?

Denormalization is the process of combining tables or adding redundant data into a database to improve read performance at the cost of data redundancy.

It is the opposite of Normalization, used when fast data retrieval is more important than maintaining data integrity.

Key Features:

- Improves data retrieval speed
- Increases data redundancy
- Reduces the number of joins required
- Used in data warehouses and reporting systems

Example:**Normalized Tables:****Employee Table:**

EmployeeID	EmployeeName	DepartmentID
101	Vinay	1
102	Awadhesh	2

Department Table:

DepartmentID	DepartmentName
1	IT
2	HR

Denormalized Table:

EmployeeID	EmployeeName	DepartmentName
101	Vinay	IT
102	Awadhesh	HR

In this example, the DepartmentName column is directly stored in the Employee table, making data retrieval faster but increasing redundancy.

23. What is the Difference Between CHAR and VARCHAR?

Criteria	CHAR	VARCHAR
Full Form	Character	Variable Character
Storage	Fixed-length	Variable-length
Memory Usage	Always uses the specified length, even if fewer characters are stored	Uses only the space required for the actual data plus one or two bytes for length
Performance	Faster for fixed-size data	Slower for varying data sizes
Padding	Pads extra spaces to match the fixed length	Does not pad spaces
Use Case	When data length is consistent (like PIN codes or gender)	When data length varies (like names or addresses)

Example:

```
CREATE TABLE Employees (
  EmpID INT,
  Name CHAR(10),
  Address VARCHAR(50)
);
```

In this example, Name will always take 10 bytes, while Address will use only the necessary space based on the actual data length.

24. What is the difference between SQL and MySQL?

Criteria	SQL	MySQL
Definition	Structured Query Language used to manage and manipulate databases	Relational Database Management System (RDBMS) that uses SQL
Type	Language	Software
Usage	Used to write queries to interact with databases	Used to store, manage, and retrieve data
Developer	Standard language developed by ANSI	Developed by Oracle Corporation
Platform	Universal (used by many databases like MySQL, Oracle, SQL Server)	Specific to MySQL
Purpose	Query language to communicate with databases	Database system to store and manage data

Example:

- SQL:

```
SELECT * FROM Employees;
```

- MySQL:

MySQL stores the Employees table and processes the SQL query to retrieve data.

25. What is Auto Increment in SQL?

Auto Increment is a property in SQL that automatically generates a unique sequential number whenever a new row is inserted into a table. It is typically used to create unique identifiers like primary keys.

Key Features:

- Automatically generates unique numbers
- Commonly used with Primary Key columns
- Starts from a defined value (default is 1)
- Automatically increments by 1 for each new row

Syntax (MySQL):

```
CREATE TABLE Employees (  
    EmployeeID INT AUTO_INCREMENT PRIMARY KEY,  
    Name VARCHAR(50),  
    Salary DECIMAL(10, 2)  
);
```

Example (Insert Records):

```
INSERT INTO Employees (Name, Salary) VALUES ('Vinay', 50000);  
INSERT INTO Employees (Name, Salary) VALUES ('Raj', 40000);
```

Output:

EmployeeID	Name	Salary
1	Vinay	50000
2	Awadhesh	40000

The EmployeeID column automatically increments without user input.

Intermediate Level

26. What is Subquery?

A Subquery is a query nested inside another query in SQL.

It is used to fetch data that will be used by the main query as a condition to filter or manipulate the result.

Key Points:

- Also called Inner Query or Nested Query.
- Always executes before the main query.
- The result of the subquery is used by the Outer Query.
- Can be used with SELECT, INSERT, UPDATE, or DELETE statements.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name = (SELECT column_name FROM another_table WHERE condition);
```

Types of Subqueries:

1. **Single Row Subquery:** Returns only one value.

Example:

```
SELECT name, salary
FROM employees
WHERE salary = (SELECT MAX(salary) FROM employees);
```

Explanation: It selects the employee with the highest salary.

2. **Multiple Row Subquery:** Returns multiple rows of data.

Example:

```
SELECT name
FROM employees
WHERE department_id IN (SELECT department_id FROM departments WHERE location = 'Bangalore');
```

Explanation: It selects employees working in the Bangalore location.

3. **Correlated Subquery:** Uses each row of the outer query to execute the subquery.

Example:

```
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE department_id = e.department_id);
```

Explanation: It selects employees whose salary is higher than the average salary of their own department.

27. What is Nested Query?

A Nested Query is a query written inside another query to retrieve data based on the result of the inner query.

It helps break down complex queries into smaller, more manageable parts.

Key Points:

- Also known as Inner Query or Subquery.
- The Inner Query executes first, and its result is passed to the Outer Query.
- Used in SELECT, INSERT, UPDATE, or DELETE statements.
- Can be used with WHERE, HAVING, and FROM clauses.

Syntax:

```
SELECT column_name
FROM table_name
WHERE column_name = (SELECT column_name FROM another_table WHERE condition);
```

Example:

Find employees who work in the 'Sales' department.

```
SELECT name
FROM employees
WHERE department_id = (SELECT department_id FROM departments WHERE department_name = 'Sales');
```

Explanation:

- The inner query finds the department_id for the 'Sales' department.
- The outer query selects employees based on that department_id.

28. What is Correlated Subquery?

A Correlated Subquery is a subquery that depends on the values from the outer query to execute.

It is executed repeatedly for each row of the outer query, making it slower compared to regular subqueries.

Key Points:

- The Inner Query uses columns from the Outer Query.
- Executes row by row for each result of the outer query.
- Cannot be executed independently without the outer query.
- Used for row-by-row comparisons.

Syntax:

```
SELECT column_name
FROM table_name t1
WHERE column_name = (SELECT column_name FROM table_name t2 WHERE t1.column = t2.column);
```

Example:

Find employees whose salary is higher than the average salary of their department.

```
SELECT e.name, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(salary) FROM employees WHERE department_id = e.department_id);
```

Explanation:

- The inner query calculates the average salary of each department.
- The outer query checks if the employee's salary is higher than their department's average salary.

29. What is GROUP BY in SQL?

The GROUP BY clause in SQL is used to group rows that have the same values into summary rows. It is typically used with aggregate functions to perform calculations on each group of data.

Key Points:

- Groups data based on one or more columns.
- It is used to summarize data.
- Always used after the WHERE clause and before the ORDER BY clause.
- Commonly used with aggregate functions like:
 - COUNT() – Counts the number of rows.
 - SUM() – Calculates the total sum.
 - AVG() – Calculates the average value.
 - MAX() – Returns the maximum value.
 - MIN() – Returns the minimum value.

Syntax:

```
SELECT column_name, aggregate_function(column_name)
FROM table_name
WHERE condition
GROUP BY column_name;
```

Example:

Find the total sales amount for each product category.

```
SELECT category, SUM(sales_amount) AS Total_Sales
FROM sales
GROUP BY category;
```

Conclusion:

The GROUP BY clause is essential for data summarization and helps in analyzing data patterns efficiently.

30. What is the Difference Between GROUP BY and ORDER BY in SQL?

Both GROUP BY and ORDER BY are SQL clauses used to organize query results, but they serve different purposes.

GROUP BY	ORDER BY
Used to group rows based on the same values in one or more columns.	Used to sort the result set in ascending or descending order.
Always works with aggregate functions like COUNT(), SUM(), AVG(), etc.	Does not require aggregate functions.
Syntax: Comes before ORDER BY.	Syntax: Always comes after GROUP BY.
Groups the result into summary rows.	Sorts the entire result set.
Example: Group sales by product category.	Example: Sort sales by highest to lowest amount.

Example with GROUP BY:

Find the total sales for each product category.

```
SELECT category, SUM(sales_amount) AS Total_Sales
FROM sales
GROUP BY category;
```

Example with ORDER BY:

Sort products by price in descending order.

```
SELECT product_name, price
FROM products
ORDER BY price DESC;
```

Conclusion:

- GROUP BY is used to group data and perform aggregate calculations.
- ORDER BY is used to sort the final result.

31. What is the Use of LIMIT in SQL?

LIMIT is used to restrict the number of rows returned by a query.

Syntax:

```
SELECT column_name
FROM table_name
LIMIT number_of_rows;
```

Example:

Fetch top 3 highest salaries:

```
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```

Conclusion:

- LIMIT helps to fetch limited data and is commonly used for Top N records or Pagination.

32. How to Find the Second Highest Salary in SQL?

Using Subquery:

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Explanation:

- The inner query gets the highest salary.
- The outer query finds the highest salary below the top salary.

33. How to find Duplicate Records in a table?

You can find Duplicate Records using the GROUP BY clause with the HAVING keyword.

Syntax:

```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name
HAVING COUNT(*) > 1;
```

Explanation:

- GROUP BY groups the same names.
- HAVING COUNT(*) > 1 filters only duplicate names.

34. What is CTE (Common Table Expression)?

CTE (Common Table Expression) is a temporary result set that is defined within the execution of a single SQL statement.

Syntax:

```
WITH cte_name AS (  
  SELECT column_name  
  FROM table_name  
  WHERE condition  
)  
SELECT * FROM cte_name;
```

Example:

Find employees who work in the IT department and have a salary greater than 50000.

```
WITH IT_Employees AS (  
  SELECT name, department, salary  
  FROM employees  
  WHERE department = 'IT'  
)  
SELECT name, salary  
FROM IT_Employees  
WHERE salary > 50000;
```

Explanation:

- The CTE IT_Employees selects all employees from the IT department.
- The SELECT query fetches employees with salary above 50000.

35. What is Temporary Table in SQL?

A Temporary Table in SQL is used to store temporary data during the session.

Key Points:

- Automatically deleted when the session ends.
- Used to store intermediate results.
- Prefixed with # in SQL Server or TEMP in MySQL.

Syntax:

MySQL:

```
CREATE TEMPORARY TABLE temp_table (  
  id INT,  
  name VARCHAR(50)  
);
```

SQL Server:

```
CREATE TABLE #temp_table (  
    id INT,  
    name VARCHAR(50)  
);
```

Example:

Create a temporary table to store employees with salary above 50000.

```
CREATE TEMPORARY TABLE temp_emp AS  
SELECT name, salary  
FROM employees  
WHERE salary > 50000;  
  
SELECT * FROM temp_emp;
```

Explanation:

- The table holds filtered data temporarily.
- It is automatically deleted after the session ends.

36. What is Window Function in SQL?

A Window Function performs calculations across a set of table rows related to the current row without collapsing the result into a single value.

Key Points:

- Works with OVER() clause.
- Does not group rows like aggregate functions.
- Commonly used for ranking, running totals, and moving averages.

Syntax:

```
SELECT column_name,  
       window_function() OVER (PARTITION BY column_name ORDER BY column_name)  
FROM table_name;
```

Types of Window Functions:

1. ROW_NUMBER()

Assigns a unique sequential number to each row in a partition.

Syntax:

```
ROW_NUMBER() OVER (PARTITION BY column ORDER BY column)
```

2. RANK()

Assigns a rank to each row with the same values having the same rank, but skips ranks for duplicate values.

Syntax:

```
RANK() OVER (PARTITION BY column ORDER BY column)
```

3. DENSE_RANK()

Similar to RANK(), but does not skip ranks for duplicate values.

Syntax:

```
DENSE_RANK() OVER (PARTITION BY column ORDER BY column)
```

4. NTILE(n)

Divides the result set into n equal parts and assigns a group number to each row.

Syntax:

```
NTILE(n) OVER (PARTITION BY column ORDER BY column)
```

5. SUM()

Calculates the cumulative total of a column within a partition.

Syntax:

```
SUM(column) OVER (PARTITION BY column ORDER BY column)
```

6. AVG()

Calculates the average value of a column within a partition.

Syntax:

```
AVG(column) OVER (PARTITION BY column ORDER BY column)
```

7. MAX() & MIN()

Returns the maximum or minimum value in a partition.

Syntax:

```
MAX(column) OVER (PARTITION BY column)  
MIN(column) OVER (PARTITION BY column)
```

8. LEAD()

Returns the next row's value in the result set.

Syntax:

```
LEAD(column, offset) OVER (PARTITION BY column ORDER BY column)
```

9. LAG()

Returns the previous row's value in the result set.

Syntax:

```
LAG(column, offset) OVER (PARTITION BY column ORDER BY column)
```

Summary Table:

Function	Purpose	Duplicates	Skips Numbers	Example Usage
ROW_NUMBER	Unique Rank	No	No	Employee Ranking
RANK	Rank with Skips	Yes	Yes	Competition Rank
DENSE_RANK	Rank without Skips	Yes	No	Class Rank
NTILE	Divide Rows	No	No	Quartiles
SUM	Running Total	No	No	Salary Analysis
AVG	Running Average	No	No	Salary Average
LEAD	Next Row Value	No	No	Price Trends
LAG	Previous Row Value	No	No	Sales Trends

Conclusion:

Window Functions help in performing complex calculations across result sets without grouping them into a single row, making them essential for ranking, running totals, and trend analysis.

37. What is the Difference Between ROW_NUMBER(), RANK(), and DENSE_RANK()?

These three Window Functions are used to assign numbers to rows based on their order.

Key Differences:

Function	Purpose	Duplicates	Skips Numbers	Example
ROW_NUMBER()	Assigns unique sequential numbers to each row	No	No	1, 2, 3, 4
RANK()	Assigns rank to each row with the same values having the same rank	Yes	Yes	1, 2, 2, 4
DENSE_RANK()	Assigns rank without skipping numbers for duplicate values	Yes	No	1, 2, 2, 3

38. What is CASE Statement in SQL?

The CASE Statement is used to apply conditional logic in SQL queries, similar to IF-ELSE statements.

Syntax:

```
SELECT column,  
CASE  
    WHEN condition1 THEN result1  
    WHEN condition2 THEN result2  
    ELSE result3  
END AS alias_name  
FROM table_name;
```

Example:

```
SELECT name, salary,  
CASE  
    WHEN salary > 6000 THEN 'High Salary'  
    WHEN salary BETWEEN 4000 AND 6000 THEN 'Medium Salary'  
    ELSE 'Low Salary'  
END AS Salary_Category  
FROM employees;
```

39. What is COALESCE in SQL?

COALESCE returns the first non-null value from a list of expressions.

Syntax:

```
COALESCE(expr1, expr2, ..., exprN)
```

Example:

```
SELECT name, COALESCE(email, phone, 'No Contact') AS Contact_Info  
FROM students;
```

Key Points:

- Returns the first non-null value.
- Used to handle NULL values.
- Can accept multiple expressions.

40. What is NVL Function in SQL?

NVL function replaces NULL values with a specified value.

Syntax:

```
NVL(expr1, expr2)
```

Example:

```
SELECT name, NVL(email, 'No Email') AS Email  
FROM students;
```

41. What is Indexing in SQL?

Indexing improves the speed of data retrieval from a table by creating a lookup structure.

Syntax:

```
CREATE INDEX index_name ON table_name(column_name);
```

Example:

```
CREATE INDEX idx_name ON students(name);
```

Key Points:

- Speeds up SELECT queries.
- Automatically maintained by the database.
- Can be created on one or more columns.
- Slows down INSERT, UPDATE, DELETE operations.

42. What is Clustered Index in SQL?

A Clustered Index sorts and stores the data physically in the table based on the indexed column.

Syntax:

```
CREATE CLUSTERED INDEX index_name ON table_name(column_name);
```

Example:

```
CREATE CLUSTERED INDEX idx_name ON students(id);
```

Key Points:

- Only one clustered index is allowed per table.
- Faster for data retrieval.
- Automatically created on Primary Key by default.
- Rearranges table rows physically.

43. What is Non-Clustered Index in SQL?

A Non-Clustered Index creates a separate structure from the table data, storing pointers to the actual rows.

Syntax:

```
CREATE INDEX index_name ON table_name(column_name);
```

Example:

```
CREATE INDEX idx_name ON students(name);
```

Key Points:

- Multiple Non-Clustered Indexes can be created on a table.
- Improves search performance.
- Does not affect the physical order of data.
- Stores pointers to the actual data.

44. Difference between Clustered and Non-Clustered Index

Clustered Index	Non-Clustered Index
Stores data physically sorted.	Stores pointers to data.
Only one per table.	Multiple indexes allowed.
Faster for data retrieval.	Slower than Clustered Index.
Automatically created on Primary Key.	Manually created on any column.
Affects physical order of table.	Does not affect physical order.

45. What is View in SQL?

A View is a virtual table based on the result of a SQL query.

Syntax:

```
CREATE VIEW view_name AS
SELECT column1, column2
FROM table_name
WHERE condition;
```

Example:

```
CREATE VIEW high_salary AS
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

Key Points:

- Does not store data physically.
- Simplifies complex queries.
- Provides data security.
- Can be used like a table in SELECT queries.

46. Difference between View and Table

View	Table
Virtual table.	Physical table.
Does not store data physically.	Stores data physically.
Based on SQL queries.	Contains raw data.
Provides data security by restricting access to certain columns.	No data restriction unless applied.
Automatically updates when base table data changes.	Needs manual updates.

47. What is Stored Procedure ?

A Stored Procedure is a group of predefined SQL statements stored in the database that can be executed multiple times.

Syntax:

```
CREATE PROCEDURE procedure_name
AS
BEGIN
    SQL_Statements;
END;
```

Example:

```
CREATE PROCEDURE GetEmployees
AS
BEGIN
    SELECT * FROM employees;
END;
```

Key Points:

- Improves code reusability.
- Increases performance.
- Supports input and output parameters.
- Provides security by hiding SQL code.

48. What is the difference between Function and Stored Procedure?

Function	Stored Procedure
Returns a single value or table.	May or may not return a value.
Can be used in SELECT statements.	Cannot be used in SELECT statements.
Allows only input parameters.	Allows input and output parameters.
Cannot modify database state.	Can modify database state (INSERT, UPDATE, DELETE).
Always returns a value.	Does not always return a value.

49. What is Trigger in SQL?

A Trigger is an automatic action executed when a specified event occurs in a table.

Syntax:

```
CREATE TRIGGER trigger_name
AFTER INSERT
ON table_name
FOR EACH ROW
BEGIN
    SQL_Statements;
END;
```

Example:

```
CREATE TRIGGER after_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO logs(message) VALUES('New employee added');
END;
```

Key Points:

- Automatically executes on INSERT, UPDATE, or DELETE.
- Used for data validation and logging.
- Cannot be called manually.
- Improves data integrity.

50. What is Cursor in SQL?

A Cursor is a database object used to retrieve, manipulate, and navigate row-by-row through the result set.

Syntax:

```
DECLARE cursor_name CURSOR FOR
SELECT column_name FROM table_name;

OPEN cursor_name;

FETCH NEXT FROM cursor_name INTO variable;

CLOSE cursor_name;
DEALLOCATE cursor_name;
```

Example:

```
DECLARE emp_cursor CURSOR FOR
SELECT name FROM employees;

OPEN emp_cursor;

FETCH NEXT FROM emp_cursor INTO @emp_name;

WHILE @@FETCH_STATUS = 0
BEGIN
    PRINT @emp_name;
    FETCH NEXT FROM emp_cursor INTO @emp_name;
END;

CLOSE emp_cursor;
DEALLOCATE emp_cursor;
```

Key Points:

- Used to process row-by-row results.
- Slower than set-based operations.
- Helps in complex data manipulation.
- Not recommended for large datasets.

Advanced Level

51. What is the ACID Property in SQL?

The ACID properties in SQL define the key principles to ensure that database transactions are processed reliably without affecting data integrity.

ACID Stands for:

Property	Description
A - Atomicity	Transaction should be all or nothing. If one part of the transaction fails, the entire transaction fails, and the database remains unchanged.
C - Consistency	The database must be in a consistent state before and after the transaction. It ensures that data remains correct and valid.
I - Isolation	Transactions should be executed independently, without interfering with each other.
D - Durability	Once a transaction is committed, the changes must be permanent in the database, even if the system crashes.

Example:

```
BEGIN TRANSACTION;  
UPDATE Accounts SET Balance = Balance - 1000 WHERE AccountNo = 101; -- Debit Amount  
UPDATE Accounts SET Balance = Balance + 1000 WHERE AccountNo = 102; -- Credit Amount  
COMMIT;
```

Explanation:

1. **Atomicity:** If one of the two queries fails, both updates will be rolled back.
2. **Consistency:** The total amount in both accounts will remain the same.
3. **Isolation:** If another transaction is trying to access the same account, it will wait until this transaction completes.
4. **Durability:** After COMMIT, changes will be saved permanently even in case of a power failure.

52. What is a Transaction in SQL?

A Transaction in SQL is a group of SQL operations that are executed as a single unit to perform a specific task on the database.

It follows ACID Properties to maintain data integrity.

Key ACID Properties:

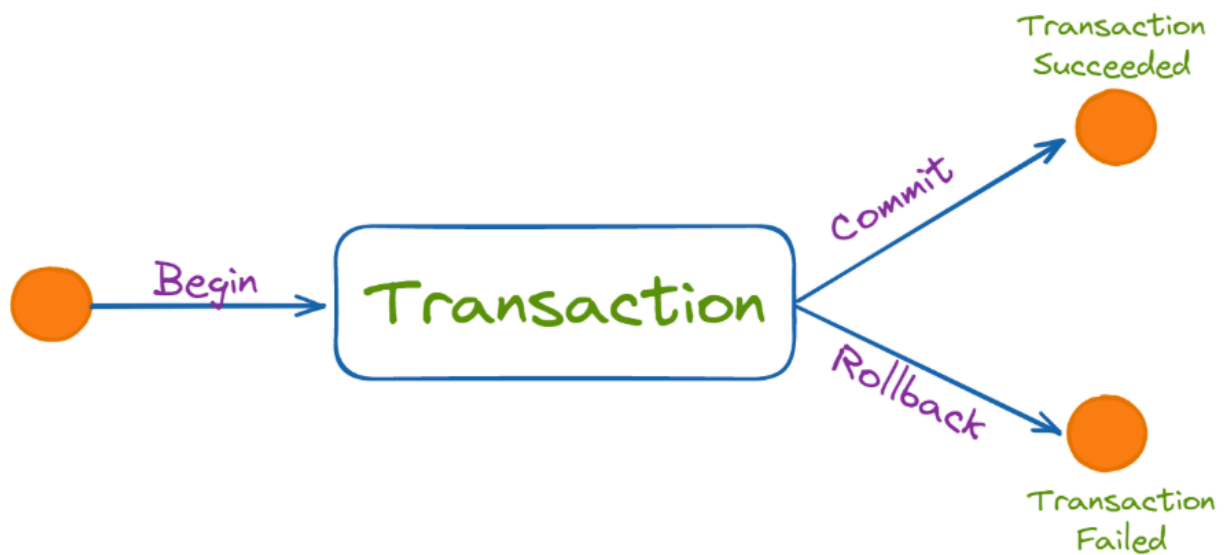
Property	Description
Atomicity	All operations must succeed or none will happen.
Consistency	The database must remain in a valid state before and after the transaction.
Isolation	Transactions execute independently without affecting each other.
Durability	Once committed, changes are permanent even after system failure.

Transaction Commands:

Command	Description
BEGIN TRANSACTION	Starts a new transaction.
COMMIT	Saves the changes permanently.
ROLLBACK	Undo changes if any error occurs.
SAVEPOINT	Sets a point to roll back to partially.

Example:

```
BEGIN TRANSACTION;  
UPDATE Employees SET Salary = Salary + 5000 WHERE ID = 101;  
DELETE FROM Employees WHERE ID = 102;  
  
-- If all queries are successful  
COMMIT;  
  
-- If any query fails  
ROLLBACK;
```



Transactions in SQL

53. What is the difference between COMMIT and ROLLBACK?

COMMIT	ROLLBACK
Saves the changes made by the transaction permanently into the database.	Undo all changes made by the transaction.
Once executed, changes cannot be undone.	Restores the database to its previous state.
Syntax: COMMIT;	Syntax: ROLLBACK;
Used when all operations are successful.	Used when any error occurs during the transaction.
Improves data durability.	Helps to maintain data consistency.

Example:

```

BEGIN TRANSACTION;
UPDATE Employees SET Salary = 6000 WHERE ID = 101;

-- If no error occurs
COMMIT;

-- If error occurs
ROLLBACK;
  
```

54. What is Savepoint in SQL?

Savepoint in SQL is used to temporarily save a transaction at a specific point, allowing you to rollback only part of the transaction without affecting the entire transaction.

Key Points:

- Allows setting multiple points in a transaction.
- Helps in partial rollback.
- Improves error handling.
- Used with ROLLBACK.

Syntax:

```
BEGIN TRANSACTION;
UPDATE Employees SET Salary = 5000 WHERE ID = 101;
SAVEPOINT SP1; -- Savepoint 1
UPDATE Employees SET Salary = 7000 WHERE ID = 102;
ROLLBACK TO SP1; -- Rollback to Savepoint 1
COMMIT; -- Final Commit
```

Explanation:

- The first update will be saved.
- The second update will be rolled back.
- Remaining changes will be committed.

55. What is the difference between IN and EXISTS?

IN	EXISTS
Compares values from the main query with a list of values.	Checks if subquery returns any rows.
Works with static values or subqueries.	Only works with subqueries.
Slower with large datasets.	Faster for large datasets.
Returns all matching rows.	Stops checking after finding the first match.

56. What is the difference between DELETE and TRUNCATE?

DELETE	TRUNCATE
Removes specific rows based on a condition using the WHERE clause.	Removes all rows from the table without any condition.
Can be rolled back using ROLLBACK if inside a transaction.	Cannot be rolled back once executed.
Slower because it logs each row deletion.	Faster because it does not log individual row deletions.
Maintains table structure and identity column values.	Resets identity column values to the initial seed.

57. What is Index Fragmentation?

Index Fragmentation occurs when the logical order of index pages in the database does not match the physical order of data on disk, making data retrieval slower.

Types of Index Fragmentation:

1. **Internal Fragmentation** – Unused space inside index pages due to data deletion or updates.
2. **External Fragmentation** – Index pages are stored in non-sequential order, causing slower data access.

How to Check Index Fragmentation in MySQL:

```
SHOW INDEX FROM Employees;
```

How to Fix Index Fragmentation:

- Use OPTIMIZE TABLE to reorganize index pages.

```
OPTIMIZE TABLE Employees;
```

Conclusion:

Index fragmentation slows down query performance and should be fixed regularly to maintain database efficiency.

58. What is the difference between RANK() and DENSE_RANK()?

RANK()	DENSE_RANK()
Assigns a unique rank to each row, but skips the next rank if there are duplicate values.	Assigns a unique rank to each row without skipping ranks if there are duplicate values.
Gaps are created in ranking sequence.	No gaps in ranking sequence.
Slower in performance compared to DENSE_RANK().	Faster than RANK() because it doesn't skip ranks.

59. How to fetch common records from two tables?

To fetch common records from two tables, you can use the INNER JOIN clause in SQL.

Syntax:

```
SELECT table1.column, table2.column
FROM table1
INNER JOIN table2 ON table1.common_column = table2.common_column;
```

60. What is the difference between UNION and JOIN?

UNION	JOIN
Combines result sets vertically (rows) from two or more tables.	Combines result sets horizontally (columns) based on common columns.
Removes duplicates by default (UNION), or includes duplicates with UNION ALL.	Does not remove duplicates.
Tables should have the same number of columns and data types.	Tables can have different numbers of columns.
Used when tables have similar data.	Used when tables have related data through a common column.

61. What is Pivot Table in SQL?

A Pivot Table in SQL is used to transform row data into column data to provide a summary report of the dataset.

It is commonly used to perform data aggregation and present data in a more readable format.

Key Points:

- Converts rows into columns.
- Used to generate summary reports.
- Helps in data analysis.
- Commonly used with aggregate functions like SUM(), AVG(), COUNT(), etc.

Example: Pivot Table Query

```
SELECT Product,  
SUM(CASE WHEN Month = 'Jan' THEN Sales ELSE 0 END) AS Jan_Sales,  
SUM(CASE WHEN Month = 'Feb' THEN Sales ELSE 0 END) AS Feb_Sales  
FROM Sales  
GROUP BY Product;
```

Conclusion:

Pivot Tables help to summarize large datasets and present them in a structured format.

62. What is Case Sensitivity in SQL?

Case Sensitivity in SQL refers to whether the database treats uppercase and lowercase letters as different or same when performing queries.

Key Points:

- **Column Names:** Most databases are not case-sensitive (MySQL, SQL Server).
- **Table Names:** Case sensitivity depends on the database and operating system.
- **String Values:** By default, MySQL is case-insensitive for string comparisons.

Example in MySQL:

```
CREATE TABLE Employees (  
    EmpID INT,  
    Name VARCHAR(50)  
);  
  
INSERT INTO Employees VALUES (101, 'Vinay'), (102, 'VINAY');  
  
SELECT * FROM Employees WHERE Name = 'vinay';
```

Output:

EmplID	Name
101	Vinay
102	VINAY

How to Make MySQL Case-Sensitive:

```
SELECT * FROM Employees WHERE BINARY Name = 'vinay';
```

This query will only return exact case-sensitive matches.

Conclusion:

- Case Insensitivity is default in MySQL for string comparisons.
- Use the BINARY keyword to perform case-sensitive searches.

63. How to find the Nth Highest Salary?

To find the Nth Highest Salary in SQL, you can use the LIMIT with OFFSET method or Subquery with ORDER BY.

Method 1: Using LIMIT with OFFSET (MySQL)

```
SELECT DISTINCT Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 1 OFFSET N-1;
```

Example: Example (3rd Highest Salary):

```
SELECT DISTINCT Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 1 OFFSET 2;
```

Explanation:

- **ORDER BY Salary DESC** → Sorts the salaries in descending order.
- **OFFSET 2** → Skips the top 2 salaries.
- **LIMIT 1** → Selects the next salary as the 3rd highest.

Method 2: Using Subquery with LIMIT

```
SELECT Name, Salary
FROM Employees
WHERE Salary = (SELECT DISTINCT Salary
                FROM Employees
                ORDER BY Salary DESC
                LIMIT 1 OFFSET 2);
```

Conclusion:

- Use LIMIT with OFFSET for faster results in MySQL.
- This method is commonly asked in interviews.
- Always use DISTINCT to remove duplicate salaries.

64. How to get First 3 Maximum Salaries?

To fetch the First 3 Maximum Salaries from a table, you can use the DISTINCT, ORDER BY, and LIMIT clauses.

Method : Using LIMIT (MySQL)

```
SELECT DISTINCT Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 3;
```

Explanation:

- **DISTINCT** → Removes duplicate salaries.
- **ORDER BY Salary DESC** → Sorts salaries in descending order.
- **LIMIT 3** → Fetches the top 3 salaries.

65. What is the difference between Drop, Delete, and Truncate?

Command	Purpose	Can Rollback	Speed	Structure Affected	Condition-Based
DROP	Deletes the entire table with its structure	No	Fastest	Yes (Removes table structure)	No
DELETE	Removes specific rows based on condition	Yes (if within Transaction)	Slow	No	Yes (With WHERE Clause)
TRUNCATE	Removes all rows from the table	No	Fast	No (Keeps structure)	No

Conclusion:

- Use DELETE for removing specific rows with conditions.
- Use TRUNCATE for removing all rows quickly without rollback.
- Use DROP to delete both data and table structure completely.

66. How to calculate Age from Date of Birth in SQL?

You can calculate the Age from the Date of Birth using the DATEDIFF() or YEAR() functions depending on the database.

Method 1: Using DATEDIFF() (MySQL)

```
SELECT Name,
FLOOR(DATEDIFF(CURDATE(), DOB) / 365) AS Age
FROM Employees;
```

Explanation:

- **CURDATE()** → Returns the current date.
- **DATEDIFF()** → Calculates the difference between the current date and date of birth in days.
- **FLOOR()** → Converts the result into whole years.

Method 2: Using YEAR() (MySQL)

```
SELECT Name,
YEAR(CURDATE()) - YEAR(DOB) AS Age
FROM Employees;
```

Explanation:

This method calculates the difference between the current year and the birth year.

Conclusion:

- Use DATEDIFF() for accurate age calculation.
- Use YEAR() for simple year-based age calculation.

67. What is Recursive Query in SQL?

A Recursive Query in SQL is a query that refers to itself to perform repetitive operations until a specific condition is met. It is commonly used to process hierarchical data such as employee-manager relationships or organizational structures.

Key Points:

- Used to handle hierarchical data.
- Implemented using Common Table Expressions (CTE).
- The recursion continues until the termination condition is satisfied.

Example:

Employee Table:

EmpID	Name	ManagerID
101	Vinay	NULL
102	Awadhesh	101
103	Nevendra	102

Recursive Query:

```
WITH EmployeeCTE AS (
    SELECT EmpID, Name, ManagerID FROM Employees WHERE ManagerID IS NULL -- Anchor Query
    UNION ALL
    SELECT E.EmpID, E.Name, E.ManagerID FROM Employees E
    JOIN EmployeeCTE C ON E.ManagerID = C.EmpID -- Recursive Query
)
SELECT * FROM EmployeeCTE;
```

68. What is the difference between Temporary Table and CTE?

Temporary Table	CTE (Common Table Expression)
Stores data physically in temporary memory.	Stores data logically without physical storage.
Needs to be explicitly created and dropped.	Automatically disappears after query execution.
Can be used multiple times within a session.	Can be used only once in the same query.
Supports Indexing and DDL operations.	Does not support Indexing or DDL operations.
Slower for small datasets.	Faster for small datasets.

Conclusion:

- Use Temporary Tables when data needs to be reused multiple times.
- Use CTE for short-term data manipulation and improved readability.

69. How to find Odd and Even records in SQL?

You can find Odd and Even records in SQL using the MOD() or ROW_NUMBER() functions.

Method 1: Using MOD() Function (MySQL)

```
SELECT * FROM Employees WHERE MOD(EmpID, 2) = 0; -- Even Records
SELECT * FROM Employees WHERE MOD(EmpID, 2) = 1; -- Odd Records
```

Explanation:

- MOD(EmpID, 2) → Returns the remainder when EmpID is divided by 2.
- If the remainder is 0, the record is Even.
- If the remainder is 1, the record is Odd.

Method 2: Using ROW_NUMBER() (SQL Server, PostgreSQL)

```
WITH RowCTE AS (
    SELECT Name, ROW_NUMBER() OVER (ORDER BY EmpID) AS RowNum FROM Employees
)
SELECT Name FROM RowCTE WHERE RowNum % 2 = 0; -- Even
SELECT Name FROM RowCTE WHERE RowNum % 2 = 1; -- Odd
```

Conclusion:

- Use MOD() for databases like MySQL.
- Use ROW_NUMBER() for databases that support Window Functions.

70. What is JSON in SQL?

JSON (JavaScript Object Notation) in SQL is used to store, retrieve, and manipulate data in a structured, text-based format within relational databases.

Key Points:

- Stores data in key-value pairs.
- Lightweight and easy to read.
- Commonly used for semi-structured data.
- Supported in MySQL, SQL Server, and PostgreSQL.

MySQL Example:**Create Table with JSON Column:**

```
CREATE TABLE Employees (  
    EmpID INT,  
    Name VARCHAR(50),  
    Details JSON  
);
```

Insert JSON Data:

```
INSERT INTO Employees VALUES (101, 'Vinay', '{"City": "Bangalore", "Age": 25}');
```

Retrieve JSON Data:

```
SELECT Name, Details->>'$.City' AS City FROM Employees;
```

Conclusion:

JSON helps to handle semi-structured data within relational databases without the need for separate NoSQL databases.

71. What is XML in SQL?

XML (Extensible Markup Language) in SQL is used to store, retrieve, and manipulate structured data in a text-based format within relational databases.

Key Points:

- Stores data in hierarchical format using tags.
- Commonly used for data exchange between applications.
- Supported in SQL Server, MySQL, and Oracle.
- Helps to store semi-structured data.

MySQL Example:

Create Table with XML Data:

```
CREATE TABLE Employees (  
    EmpID INT,  
    Name VARCHAR(50),  
    Details TEXT  
);
```

Insert XML Data:

```
INSERT INTO Employees VALUES (101, 'Vinay', '<Employee><City>Bangalore</City><Age>25</Age></Employee>');
```

Retrieve XML Data:

```
SELECT Name, Details FROM Employees WHERE EmpID = 101;
```

Conclusion:

XML is used to store and transfer hierarchical data in relational databases, making it easier to exchange data between applications.

72. How to handle NULL values in SQL?

NULL represents missing or unknown data in SQL.

Methods to Handle NULL Values:

1. IS NULL – To check if a column contains NULL.

```
SELECT * FROM Employees WHERE Salary IS NULL;
```

2. IS NOT NULL – To check if a column does not contain NULL.

```
SELECT * FROM Employees WHERE Salary IS NOT NULL;
```

3. COALESCE() – Replaces NULL with a default value.

```
SELECT Name, COALESCE(Salary, 0) AS Salary FROM Employees;
```

4. IFNULL() (MySQL) – Replaces NULL with a specified value.

```
SELECT Name, IFNULL(Salary, 0) AS Salary FROM Employees;
```

5. NULLIF() – Returns NULL if two expressions are equal.

```
SELECT NULLIF(Salary, 5000) AS Result FROM Employees;
```

Conclusion:

Use COALESCE() or IFNULL() to replace NULL values and ensure data consistency in queries.

73. What is Dynamic SQL?

Dynamic SQL is a method of constructing and executing SQL statements at runtime instead of writing static queries.

Key Points:

- Allows flexible query creation based on user input or conditions.
- Used for complex queries with varying conditions.
- Helps in Parameterized Queries and stored procedures.
- Increases security risks if not handled properly.

Example (MySQL):

```
SET @query = 'SELECT * FROM Employees WHERE Department = "IT";  
PREPARE stmt FROM @query;  
EXECUTE stmt;  
DEALLOCATE PREPARE stmt;
```

Conclusion:

Dynamic SQL provides flexibility in query execution but should always be used with parameterized queries to prevent SQL injection attacks.

74. How to calculate Percentage in SQL?

You can calculate Percentage in SQL using arithmetic expressions and aggregate functions.

Syntax:

```
SELECT (PartValue * 100.0) / TotalValue AS Percentage  
FROM TableName;
```

Example: Query to Calculate Percentage of Employees in Each Department:

```
SELECT Department,  
(Employees * 100.0) / (SELECT SUM(Employees) FROM Employee) AS Percentage  
FROM Employee;
```

75. How to find the Employees who earn more than their Manager?

To find employees who earn more than their manager, you need to join the Employee table with itself using Self Join.

Example Table:

Employee Table

EmpID	Name	Salary	ManagerID
101	Vinay	50000	NULL
102	Nevendra	40000	101
103	Awadhesh	60000	101
104	Rituraj	45000	102

Query:

```
SELECT E1.Name AS Employee, E1.Salary AS EmployeeSalary, E2.Name AS Manager, E2.Salary AS ManagerSalary
FROM Employees E1
JOIN Employees E2 ON E1.ManagerID = E2.EmpID
WHERE E1.Salary > E2.Salary;
```

Output:

Employee	EmployeeSalary	Manager	ManagerSalary
Awadhesh	60000	Vinay	50000

Conclusion: Use Self Join with a condition to compare employee salaries against their managers' salaries. This query helps in hierarchical data analysis.

Real-Time Scenarios

76. How to find Duplicate Emails in the Employee Table?

To find duplicate emails in SQL, you can use the GROUP BY clause with the HAVING condition.

Syntax:

```
SELECT Email, COUNT(Email) AS DuplicateCount
FROM Employees
GROUP BY Email
HAVING COUNT(Email) > 1;
```

Explanation:

- GROUP BY groups the records based on the Email column.
- COUNT(Email) counts how many times each email appears.
- HAVING COUNT(Email) > 1 filters only those emails that have more than one occurrence.

77. How to get the Highest Salary in each Department?

You can find the Highest Salary in Each Department using the GROUP BY clause with the MAX() aggregate function.

Syntax:

```
SELECT Department, MAX(Salary) AS HighestSalary
FROM Employees
GROUP BY Department;
```

78. How to find Employees joined in the last 3 months?

To find employees who joined in the last 3 months, you can use the DATE_ADD() or DATEDIFF() function along with the WHERE clause.

Method 1: Using DATEDIFF() (MySQL)

```
SELECT Name, JoiningDate
FROM Employees
WHERE DATEDIFF(CURDATE(), JoiningDate) <= 90;
```

Method 2: Using DATE_ADD() (MySQL)

```
SELECT Name, JoiningDate
FROM Employees
WHERE JoiningDate >= DATE_ADD(CURDATE(), INTERVAL -3 MONTH);
```

79. How to Display the First 5 Records in SQL?

You can display the First 5 Records using the LIMIT or TOP clause depending on the database.

Method 1: Using LIMIT (MySQL)

```
SELECT * FROM Employees  
LIMIT 5;
```

Method 2: Using TOP (SQL Server)

```
SELECT TOP 5 * FROM Employees;
```

80. How to find the Number of Employees in each Department?

You can find the Number of Employees in each department using the COUNT() function with the GROUP BY clause.

Syntax:

```
SELECT Department, COUNT(EmpID) AS EmployeeCount  
FROM Employees  
GROUP BY Department;
```

81. How to find the Last 3 Records in SQL?

To fetch the Last 3 Records in SQL, you can use the ORDER BY clause along with LIMIT.

Method 1: Using ORDER BY with LIMIT (MySQL)

```
SELECT * FROM Employees  
ORDER BY EmpID DESC  
LIMIT 3;
```

Method 2: Using ROW_NUMBER() (SQL Server, PostgreSQL)

```
WITH CTE AS (  
    SELECT *, ROW_NUMBER() OVER (ORDER BY EmpID DESC) AS RowNum  
    FROM Employees  
)  
SELECT * FROM CTE WHERE RowNum <= 3;
```

82. How to find Employees without Managers?

To find employees without managers, you need to filter records where the ManagerID column is NULL or does not exist in the table.

Method 1: Using IS NULL (MySQL, SQL Server, Oracle)

```
SELECT Name, EmpID
FROM Employees
WHERE ManagerID IS NULL;
```

Method 2: Using LEFT JOIN (MySQL, SQL Server, PostgreSQL)

```
SELECT E1.Name AS Employee
FROM Employees E1
LEFT JOIN Employees E2 ON E1.ManagerID = E2.EmpID
WHERE E1.ManagerID IS NULL OR E2.EmpID IS NULL;
```

83. How to find the First Name starting with 'A'?

You can find employee names starting with the letter 'A' using the LIKE operator.

Syntax:

```
SELECT Name
FROM Employees
WHERE Name LIKE 'A%';
```

Explanation:

- **LIKE 'A%'** → Finds names that start with 'A'.
- **%** → Represents any number of characters after 'A'.

84. How to fetch Alternate Rows from a table?

You can fetch Alternate Rows using the MOD() or ROW_NUMBER() functions based on row position.

Method 1: Using MOD() (MySQL)

Fetch Even Rows:

```
SELECT *
FROM Employees
WHERE MOD(EmpID, 2) = 0;
```

Fetch Odd Rows:

```
SELECT *
FROM Employees
WHERE MOD(EmpID, 2) = 1;
```

Method 2: Using ROW_NUMBER() (SQL Server, PostgreSQL)

Fetch Odd Rows:

```
WITH RowCTE AS (  
    SELECT *, ROW_NUMBER() OVER (ORDER BY EmpID) AS RowNum  
    FROM Employees  
)  
SELECT * FROM RowCTE  
WHERE RowNum % 2 = 1;
```

Fetch Even Rows:

```
WITH RowCTE AS (  
    SELECT *, ROW_NUMBER() OVER (ORDER BY EmpID) AS RowNum  
    FROM Employees  
)  
SELECT * FROM RowCTE  
WHERE RowNum % 2 = 0;
```

85. How to swap two columns in SQL?

You can swap the values of two columns using the UPDATE statement with the TEMPORARY variable technique.

Syntax:

```
UPDATE TableName  
SET Column1 = Column2,  
    Column2 = Column1;
```

Correct Way Using Temporary Variable:

```
UPDATE Employees  
SET FirstName = @temp := FirstName,  
    FirstName = LastName,  
    LastName = @temp;
```

86. How to display the Duplicate Records with their Count?

To display duplicate records along with their occurrence count, use the GROUP BY clause with the HAVING clause.

Syntax:

```
SELECT ColumnName, COUNT(ColumnName) AS Count  
FROM TableName  
GROUP BY ColumnName  
HAVING COUNT(ColumnName) > 1;
```

87. How to find the Highest Salary without using MAX()?

You can find the Highest Salary without using the MAX() function by using the ORDER BY clause with the LIMIT or TOP keyword.

Method 1: Using ORDER BY with LIMIT (MySQL)

```
SELECT Salary
FROM Employees
ORDER BY Salary DESC
LIMIT 1;
```

Method 2: Using Subquery (MySQL, SQL Server)

```
SELECT Salary
FROM Employees E1
WHERE Salary >= ALL (SELECT Salary FROM Employees E2);
```

Method 3: Using NOT IN (MySQL, SQL Server)

```
SELECT Salary
FROM Employees
WHERE Salary NOT IN (SELECT Salary FROM Employees WHERE Salary < (SELECT Salary FROM Employees));
```

Conclusion:

- Use ORDER BY with LIMIT for simple queries.
- Use Subqueries or ALL for advanced scenarios.
- This method works in all RDBMS.

88. How to fetch common records from two tables without JOIN?

You can fetch common records from two tables without using JOIN by using the IN or INTERSECT operators.

Method 1: Using IN (MySQL, SQL Server)

```
SELECT Name
FROM Employees
WHERE Name IN (SELECT Name FROM Managers);
```

Method 2: Using INTERSECT (SQL Server, PostgreSQL)

```
SELECT Name FROM Employees
INTERSECT
SELECT Name FROM Managers;
```

89. How to delete Duplicate Records from a table?

You can delete duplicate records using CTE, ROW_NUMBER(), or GROUP BY methods.

Method 1: Using ROW_NUMBER() (SQL Server, PostgreSQL)

```
WITH CTE AS (  
    SELECT EmpID, Name,  
           ROW_NUMBER() OVER (PARTITION BY Name ORDER BY EmpID) AS RowNum  
    FROM Employees  
)  
DELETE FROM Employees WHERE EmpID IN (  
    SELECT EmpID FROM CTE WHERE RowNum > 1  
);
```

Method 2: Using GROUP BY with MIN() (MySQL)

```
DELETE E1 FROM Employees E1  
JOIN Employees E2  
ON E1.Name = E2.Name AND E1.EmpID > E2.EmpID;
```

Method 3: Using DISTINCT INTO Temporary Table (MySQL)

```
CREATE TEMPORARY TABLE Temp AS  
SELECT DISTINCT * FROM Employees;  
  
TRUNCATE TABLE Employees;  
INSERT INTO Employees SELECT * FROM Temp;  
DROP TEMPORARY TABLE Temp;
```

Conclusion:

- Use ROW_NUMBER() for advanced databases.
- Use GROUP BY for simple queries.
- Always backup data before deleting duplicates.

90. How to find the Department with the highest Employee Count?

You can find the Department with the Highest Employee Count using the GROUP BY clause along with the ORDER BY and LIMIT clauses.

Method 1: Using GROUP BY with ORDER BY (MySQL)

```
SELECT Department, COUNT(EmpID) AS EmployeeCount
FROM Employees
GROUP BY Department
ORDER BY EmployeeCount DESC
LIMIT 1;
```

Method 2: Using Subquery (MySQL, SQL Server)

```
SELECT Department, EmployeeCount
FROM (
    SELECT Department, COUNT(EmpID) AS EmployeeCount
    FROM Employees
    GROUP BY Department
) AS DeptCount
ORDER BY EmployeeCount DESC
LIMIT 1;
```

Conclusion:

- Use GROUP BY with ORDER BY for a simple approach.
- Use Subqueries for better performance with large datasets.

Optimization Techniques

91. How to Optimize SQL Queries?

Optimizing SQL queries improves performance and execution speed while handling large datasets.

Best Practices to Optimize SQL Queries:

1. **Use Indexes:** Create indexes on columns used in WHERE, JOIN, and ORDER BY clauses.

```
CREATE INDEX idx_name ON Employees(Name);
```

2. **Avoid SELECT:** Select only required columns instead of using SELECT

```
SELECT Name, Salary FROM Employees;
```

3. **Use Joins Efficiently:** Use INNER JOIN instead of OUTER JOIN whenever possible.

4. **Use EXISTS Instead of IN:** EXISTS performs better with large datasets.

```
SELECT Name FROM Employees WHERE EXISTS (SELECT 1 FROM Managers WHERE Employees.EmpID = Managers.EmpID);
```

5. **Use LIMIT or TOP:** Fetch only required rows using LIMIT or TOP.

```
SELECT Name FROM Employees LIMIT 10;
```

6. **Avoid Functions in WHERE Clause:**

Instead of:

```
SELECT * FROM Employees WHERE UPPER(Name) = 'VINAY';
```

Use:

```
SELECT * FROM Employees WHERE Name = 'Vinay';
```

7. **Optimize Subqueries:** Use JOIN instead of subqueries whenever possible.

8. **Partition Large Tables:** Split large tables into smaller partitions.

9. **Use CTEs and Temp Tables:** Store temporary results for better performance.

10. **Analyze Execution Plan:** Use EXPLAIN or Query Plan to check how queries are executed.

```
EXPLAIN SELECT * FROM Employees;
```

92. What is Query Execution Plan?

A Query Execution Plan is a detailed roadmap used by the database engine to execute SQL queries efficiently.

Key Points:

- Shows how the database will retrieve data.
- Helps to analyze performance and optimize queries.
- Displays the order of execution for each query operation like joins, scans, and sorting.
- Provides information about indexes, table scans, and filtering methods.

How to View Execution Plan:

MySQL:

```
EXPLAIN SELECT Name, Salary FROM Employees WHERE Department = 'IT';
```

SQL Server:

```
SET SHOWPLAN_ALL ON;  
SELECT Name, Salary FROM Employees WHERE Department = 'IT';  
SET SHOWPLAN_ALL OFF;
```

Why Use Execution Plan?

- Identify performance bottlenecks.
- Check if the query is using Indexes or Table Scans.
- Understand how Joins and Filters are applied.

93. How to Improve Query Performance?

Improving Query Performance ensures faster data retrieval and better database efficiency, especially for large datasets.

Best Practices to Improve Query Performance:

1. **Use Indexes:** Create indexes on columns used in WHERE, JOIN, and ORDER BY clauses.

```
CREATE INDEX idx_name ON Employees(Name);
```

2. **Avoid SELECT:** Select only required columns instead of using SELECT

```
SELECT Name, Salary FROM Employees;
```

3. **Use Joins Efficiently:** Prefer INNER JOIN over OUTER JOIN when possible.

4. **Use WHERE Instead of HAVING:** Filter rows early using WHERE.

```
SELECT Name FROM Employees WHERE Salary > 5000;
```

5. **Limit Results:** Fetch only the required number of rows.

```
SELECT Name FROM Employees LIMIT 10;
```

6. **Use EXISTS Instead of IN:** EXISTS performs better with subqueries.

```
SELECT Name FROM Employees  
WHERE EXISTS (SELECT 1 FROM Managers WHERE Employees.EmpID = Managers.EmpID);
```

7. **Avoid Functions in WHERE Clause:**

Instead of:

```
SELECT * FROM Employees WHERE UPPER(Name) = 'VINAY';
```

Use:

```
SELECT * FROM Employees WHERE Name = 'Vinay';
```

8. **Partition Large Tables:** Split large tables into smaller partitions.

9. **Use Temporary Tables or CTEs:** Store temporary results for better performance.

10. **Analyze Execution Plans:** Use EXPLAIN or SHOWPLAN to understand query execution.

94. What is Indexing?

Indexing in SQL is a technique used to improve the speed of data retrieval from a database by creating a lookup table for faster access.

Key Points:

- Works like a book index to find data quickly.
- Reduces the time required for SELECT queries.
- Automatically updated when data is inserted, updated, or deleted.
- Indexes are created on columns frequently used in WHERE, JOIN, and ORDER BY clauses.

Types of Indexes:

1. **Primary Index** – Automatically created on Primary Key columns.
2. **Unique Index** – Ensures that values in a column are unique.
3. **Clustered Index** – Sorts data rows physically based on key values (Only one per table).
4. **Non-Clustered Index** – Stores pointers to data rows (Multiple indexes allowed).

Conclusion: Indexing improves query performance by quickly locating data but may increase the time for INSERT, UPDATE, and DELETE operations.

95. What is Table Partitioning?

Table Partitioning is a technique used to divide large tables into smaller, more manageable pieces without changing the table structure.

Key Points:

- Improves query performance on large datasets.
- Simplifies data management.
- Helps in faster data retrieval.
- Each partition is stored separately.
- Data can be partitioned by range, list, hash, or composite methods.

Types of Partitioning:

1. **Range Partitioning** – Divides data based on value ranges.
2. **List Partitioning** – Divides data based on specific column values.
3. **Hash Partitioning** – Distributes data evenly using a hash function.
4. **Composite Partitioning** – Combination of Range and Hash partitioning.

Syntax (MySQL Range Partitioning):

```
CREATE TABLE Employees (
    EmpID INT,
    Name VARCHAR(50),
    JoiningDate DATE
)
PARTITION BY RANGE (YEAR(JoiningDate)) (
    PARTITION p1 VALUES LESS THAN (2022),
    PARTITION p2 VALUES LESS THAN (2023),
    PARTITION p3 VALUES LESS THAN (2024)
);
```

Example Query:

```
SELECT * FROM Employees
PARTITION (p2);
```

Conclusion: Table Partitioning improves performance and data management by splitting large datasets into smaller, more manageable sections.

96. How to Avoid Deadlocks in SQL?

A Deadlock occurs when two or more transactions block each other by holding locks on resources that the other transactions need.

Ways to Avoid Deadlocks:

1. Access Tables in the Same Order:

- Always access tables in a consistent sequence across transactions.

2. Minimize Lock Time:

- Keep transactions short and fast to reduce lock holding time.

3. Use Lower Isolation Levels:

- Use READ COMMITTED instead of SERIALIZABLE isolation level if possible.

4. Avoid User Interaction Inside Transactions:

- Do not wait for user input during a transaction.

5. Use NOLOCK or Read Uncommitted:

- Allow non-blocking reads for read-only operations.

```
SELECT * FROM Employees WITH (NOLOCK);
```

6. Proper Indexing:

- Use Indexes to minimize the number of rows locked.

7. Break Large Transactions:

- Split large transactions into smaller batches.

Example:

Without Deadlock Prevention:

```
BEGIN TRANSACTION;  
UPDATE Employees SET Salary = 60000 WHERE EmpID = 101;  
UPDATE Departments SET Budget = 500000 WHERE DeptID = 1;  
COMMIT;
```

With Deadlock Prevention:

```
BEGIN TRANSACTION;  
UPDATE Departments SET Budget = 500000 WHERE DeptID = 1;  
UPDATE Employees SET Salary = 60000 WHERE EmpID = 101;  
COMMIT;
```

Conclusion:

Follow consistent table access patterns, minimize lock times, and use proper indexing to avoid deadlocks in SQL transactions.

97. What is the use of EXISTS in SQL?

EXISTS in SQL is used to check whether a subquery returns any rows. It returns:

- TRUE if the subquery returns at least one row.
- FALSE if the subquery returns no rows.

Syntax:

```
SELECT column_name  
FROM table1  
WHERE EXISTS (SELECT 1 FROM table2 WHERE table1.id = table2.id);
```

Conclusion: EXISTS is faster than IN for large datasets and is commonly used in subqueries to check data existence.

98. What is Query Optimization?

Query Optimization is the process of improving the efficiency and performance of SQL queries to retrieve data faster while using minimal system resources.

Key Points:

- Helps in reducing query execution time.
- Improves database performance.
- Minimizes CPU usage, memory usage, and I/O operations.
- Automatically performed by the Query Optimizer in most RDBMS.

Techniques for Query Optimization:

1. Use Indexes to speed up searches.
2. Avoid SELECT and select only necessary columns.
3. Use JOINS instead of subqueries when possible.
4. Use EXISTS instead of IN for large datasets.
5. Avoid Functions in WHERE clause.
6. Use LIMIT or TOP to fetch only required rows.
7. Analyze the Execution Plan using tools like EXPLAIN.

Conclusion:

Query Optimization improves execution speed, reduces resource usage, and enhances overall database performance.

99. What is the Difference Between Stored Procedure and Function in SQL?

Stored Procedure	Function
Can return multiple values.	Returns only one value (scalar or table).
Can perform DML operations like INSERT, UPDATE, DELETE.	Cannot perform DML operations.
Supports input and output parameters.	Only supports input parameters.
Can call Functions inside it.	Cannot call Stored Procedures inside it.
Used for business logic and complex operations.	Used for calculations and returning values.

100. What is Query Optimization?

OLTP (Online Transaction Processing)	OLAP (Online Analytical Processing)
Used for day-to-day transactional operations.	Used for data analysis and reporting.
Focuses on data consistency and speed.	Focuses on data aggregation and analysis.
Stores detailed transactional data.	Stores historical and summarized data.
Supports INSERT, UPDATE, DELETE operations.	Supports SELECT operations with complex queries.
Small amounts of data processed per transaction.	Large amounts of data processed at once.
Example: Banking systems, E-commerce websites.	Example: Data Warehouses, Business Intelligence Tools.

SOURCE 12

Java Coding Interview Questions

Java coding questions with sample answers



Java Coding Interview Questions



1. Write a Java program to check if a number is prime.

```
public class PrimeCheck {
    public static void main(String[] args) {
        int num = 29;
        boolean isPrime = true;

        for (int i = 2; i <= num / 2; i++) {
            if (num % i == 0) {
                isPrime = false;
                break;
            }
        }

        if (isPrime)
            System.out.println(num + " is a prime number.");
        else
            System.out.println(num + " is not a prime
number.");
    }
}
```

Output: 29 is a prime number.

2. Write a Java program to sort an array using the bubble sort algorithm.

```
import java.util.Arrays;

public class BubbleSort {
    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        bubbleSort(arr);
        System.out.println("Sorted array: " +
Arrays.toString(arr));
    }
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
```

```

        // swap arr[j] and arr[j+1]
        int temp = arr[j];
        arr[j] = arr[j + 1];
        arr[j + 1] = temp;
    }
}
}
}
}
}

```

Output: Sorted array: [11, 12, 22, 25, 34, 64, 90]

3. Write a Java program to reverse a string.

```

public class ReverseString {
    public static void main(String[] args) {
        String str = "Java";
        String reversed = new
        StringBuilder(str).reverse().toString();
        System.out.println("Reversed string: " + reversed);
    }
}

```

Output: Reversed string: avaJ

4. Write a Java program to check if a string is a rotation of another string.

```

public class StringRotation {
    public static void main(String[] args) {
        String str1 = "abcd";
        String str2 = "cdab";
        boolean isRotation = areRotations(str1, str2);
        System.out.println(str1 + " and " + str2 + " are
        rotations: " + isRotation);
    }
    public static boolean areRotations(String str1, String
    str2) {
        if (str1.length() != str2.length()) return false;
        String temp = str1 + str1;
        return temp.contains(str2);
    }
}

```

Output: **abcd and cdab are rotations: true**

5. Write a Java program to check for balanced parentheses in an expression.

```
import java.util.Stack;

public class BalancedParentheses {
    public static void main(String[] args) {
        String expression = "{[()]}" ;
        boolean isBalanced = checkBalanced(expression);
        System.out.println("Is the expression balanced? " +
isBalanced;
    }

    public static boolean checkBalanced(String expr) {
        Stack<Character> stack = new Stack<>();
        for (char ch: expr.toCharArray()) {
            if (ch == '{' || ch == '(' || ch == '[') {
                stack.push(ch);
            } else if (ch == '}' || ch == ')' || ch == ']') {
                if (stack.isEmpty()) return false;
                char top = stack.pop();
                if (!isMatchingPair(top, ch)) return false;
            }
        }
        return stack.isEmpty();
    }

    public static boolean isMatchingPair(char open, char close)
    {
        return (open == '{' && close == '}') ||
            (open == '(' && close == ')') ||
            (open == '[' && close == ']');
    }
}
```

Output: **Is the expression balanced? true**

6. Write a Java program to find the largest number in an array.

```
public class LargestNumber {
    public static void main(String[] args) {
        int[] arr = {10, 20, 5, 40, 25};
        int max = arr[0];

        for (int num : arr) {
            if (num > max) {
                max = num;
            }
        }

        System.out.println("The largest number is: " + max);
    }
}
```

Output: The largest number is: 40

7. Write a Java program to print the Fibonacci series up to n terms.

```
public class Fibonacci {
    public static void main(String[] args) {
        int n = 10, a = 0, b = 1;
        System.out.print("Fibonacci Series: " + a + " " + b);

        for (int i = 2; i < n; i++) {
            int next = a + b;
            System.out.print(" " + next);
            a = b;
            b = next;
        }
    }
}
```

Output: Fibonacci Series: 0 1 1 2 3 5 8 13 21 34

8. Write a Java program to find the sum of digits of a number.

```
public class SumOfDigits {  
    public static void main(String[] args) {  
        int num = 1234, sum = 0;  
  
        while (num != 0) {  
            sum += num % 10;  
            num /= 10;  
        }  
  
        System.out.println("Sum of digits: " + sum);  
    }  
}
```

Output: Sum of digits: 10

9. Write a Java program to print all prime numbers up to n.

```
public class PrimeNumbersUpToN {  
    public static void main(String[] args) {  
        int n = 20;  
        System.out.print("Prime numbers up to " + n + ": ");  
  
        for (int i = 2; i <= n; i++) {  
            if (isPrime(i)) {  
                System.out.print(i + " ");  
            }  
        }  
  
        public static boolean isPrime(int num) {  
            for (int i = 2; i <= num / 2; i++) {  
                if (num % i == 0) {  
                    return false;  
                }  
            }  
            return true;  
        }  
    }  
}
```

Output: Prime numbers up to 20: 2 3 5 7 11 13 17 19

10. Write a Java program to find the length of the longest substring without repeating characters.

```
import java.util.HashSet;

public class LongestSubstring {
    public static void main(String[] args) {
        String str = "abcabcbb";
        int length = lengthOfLongestSubstring(str);
        System.out.println("Length of longest substring: " +
length);
    }
    public static int lengthOfLongestSubstring(String s) {
        HashSet<Character> set = new HashSet<>();
        int maxLength = 0, left = 0;
        for (int right = 0; right < s.length(); right++) {
            while (set.contains(s.charAt(right))) {
                set.remove(s.charAt(left++));
            }
            set.add(s.charAt(right));
            maxLength = Math.max(maxLength, right - left + 1);
        }
        return maxLength;
    }
}
```

Output: Length of longest substring: 3

11. Write a Java program to count the number of vowels in a string.

```
public class CountVowels {
    public static void main(String[] args) {
        String str = "Hello World";
        int count = countVowels(str);
        System.out.println("Number of vowels: " + count);
    }

    public static int countVowels(String str) {
        int count = 0;
        for (char c : str.toLowerCase().toCharArray()) {
```

```

        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' ||
c == 'u') {
            count++;
        }
    }
    return count;
}
}

```

Output: Number of vowels: 3

12. Write a Java program to implement a simple banking system (deposit and withdrawal).

```

import java.util.Scanner;

public class SimpleBanking {
    private double balance;

    public SimpleBanking() {
        this.balance = 0.0;
    }

    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
    }

    public void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance.");
        }
    }

    public double getBalance() {
        return balance;
    }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        SimpleBanking account = new SimpleBanking();

        while (true)

```

```
{
    System.out.println("\n1. Deposit\n2. Withdraw\n3. Check
Balance\n4. Exit");
    System.out.print("Choose an option: ");
    int choice = scanner.nextInt();
    switch (choice)
    {
case 1:
    System.out.print("Enter amount to deposit: ");
    double depositAmount = scanner.nextDouble();
    account.deposit(depositAmount);
        Break;

case 2:
    System.out.print("Enter amount to withdraw: ");
    double withdrawAmount = scanner.nextDouble();
    account.withdraw(withdrawAmount);
        Break;

Case 3:
    System.out.println("Current balance: " +
account.getBalance());
        Break;

Case 4:
    System.out.println("Exiting...");
    scanner.close();
    return;
Default:
    System.out.println("Invalid choice. Try again.");
        }
    }
}
```

Output:

- 1. Deposit**
- 2. Withdraw**
- 3. Check Balance**
- 4. Exit**

Choose an option: 1

Enter the amount to deposit: 100

Deposited: 100.0

13. Write a Java program to implement bubble sort.

```
import java.util.Arrays;

public class BubbleSort {
    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        bubbleSort(arr);
        System.out.println("Sorted array: " +
Arrays.toString(arr));
    }

    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    // swap arr[j] and arr[j+1]
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

Output: **Sorted array: [11, 12, 22, 25, 34, 64, 90]**

14. Write a Java program to count the occurrences of each character in a string.

```
import java.util.HashMap;
public class CharCount {
    public static void main(String[] args) {
        String str = "java";
        HashMap<Character, Integer> charCount = new
HashMap<>();
        for (char c: str.toCharArray()) {
            charCount.put(c, charCount.getOrDefault(c, 0) + 1);
        }
        System.out.println("Character count: " + charCount);
    }
}
```

```

    }
}

```

Output: Character count: {a=2, v=1, j=1}

15. Write a Java program to find the length of the longest palindrome in a string.

```

public class LongestPalindrome {
    public static void main(String[] args) {
        String str = "babad";
        System.out.println("Longest palindrome: " +
        longestPalindrome(str));
    }

    public static String longestPalindrome(String s) {
        int maxLength = 1, start = 0, len = s.length();

        for (int i = 0; i < len; i++) {
            for (int j = i; j < len; j++) {
                int flag = 1;

                for (int k = 0; k < (j - i + 1) / 2; k++)
                    if (s.charAt(i + k) != s.charAt(j - k))
                        flag = 0;

                if (flag != 0 && (j - i + 1) > maxLength) {
                    start = i;
                    maxLength = j - i + 1;
                }
            }
        }

        return s.substring(start, start + maxLength);
    }
}

```

Output: Longest palindrome: bab

16. Write a Java program to find the maximum element in an array.

```
public class MaxElement {
    public static void main(String[] args) {
        int[] arr = {10, 20, 4, 45, 99};
        int max = findMax(arr);
        System.out.println("Maximum element: " + max);
    }

    public static int findMax(int[] arr) {
        int max = arr[0];
        for (int num: arr) {
            if (num > max) {
                max = num;
            }
        }
        return max;
    }
}
```

Output: Maximum element: 99

17. Write a Java program to calculate the sum of all elements in a 2D array.

```
public class Sum2DArray {
    public static void main(String[] args) {
        int[][] arr = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        int sum = 0;

        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr[i].length; j++) {
                sum += arr[i][j];
            }
        }
        System.out.println("Sum of all elements: " + sum);
    }
}
```

Output: Sum of all elements: 45

18. Write a Java program to find the transpose of a matrix.

```
public class TransposeMatrix {
    public static void main(String[] args) {
        int[][] matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
        int[][] transpose = new
int[matrix[0].length][matrix.length];

        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[0].length; j++) {
                transpose[j][i] = matrix[i][j];
            }
        }

        System.out.println("Transpose of the matrix:");
        for (int i = 0; i < transpose.length; i++) {
            for (int j = 0; j < transpose[0].length; j++) {
                System.out.print(transpose[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Output: Transpose of the matrix:

1 4 7

2 5 8

3 6 9

19. Write a Java program to convert a binary number to decimal.

```
public class BinaryToDecimal {
    public static void main(String[] args) {
        String binary = "1010";
        int decimal = Integer.parseInt(binary, 2);
        System.out.println("Decimal of " + binary + " is: " +
decimal);
    }
}
```

Output: The decimal of 1010 is: 10

20. Write a Java program to find the power of a number using recursion.

```
public class PowerRecursion {
    public static void main(String[] args) {
        int base = 2, exp = 5;
        System.out.println(base + "^" + exp + " = " +
power(base, exp));
    }

    public static int power(int base, int exp) {
        if (exp == 0)
            return 1;
        return base * power(base, exp - 1);
    }
}
```

Output: $2^5 = 32$

21. Write a Java program to implement a simple calculator (addition, subtraction, multiplication, division).

```
import java.util.Scanner;

public class SimpleCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter first number: ");
        double num1 = scanner.nextDouble();
        System.out.print("Enter second number: ");
        double num2 = scanner.nextDouble();
        System.out.print("Choose operation (+, -, *, /): ");
        char operation = scanner.next().charAt(0);

        double result;
        switch (operation) {
            case '+':
                result = num1 + num2;
                break;
            case '-':
                result = num1 - num2;
                break;
        }
    }
}
```

```
        case '*':
            result = num1 * num2;
            break;
        case '/':
            result = num1 / num2;
            break;
        default:
            System.out.println("Invalid operation");
            return;
    }
    System.out.println("Result: " + result);
    scanner.close();
}
}
```

Output:

Enter the first number: 10

Enter the second number: 5

Choose operation (+, -, *, /): +

Result: 15.0

22. What is the output of the following code?

```
public class Main {
    public static void main(String[] args) {
        String str1 = "Hello";
        String str2 = "Hello";
        String str3 = new String("Hello");

        System.out.println(str1 == str2);
        System.out.println(str1 == str3);
        System.out.println(str1.equals(str3));
    }
}
```

Output:

True

False

True

23. Write a Java program to find the intersection of two arrays.

```
import java.util.HashSet;

public class IntersectionArrays {
    public static void main(String[] args) {
        int[] arr1 = {1, 2, 3, 4, 5};
        int[] arr2 = {4, 5, 6, 7, 8};

        HashSet<Integer> set = new HashSet<>();
        for (int i: arr1) {
            set.add(i);
        }

        System.out.print("Intersection: ");
        for (int i: arr2) {
            if (set.contains(i)) {
                System.out.print(i + " ");
            }
        }
    }
}
```

Output: Intersection: 4 5

24. Write a Java program to find an array's maximum and minimum number.

```
public class MaxMinInArray {
    public static void main(String[] args) {
        int[] arr = {10, 20, 5, 25, 15};
        int max = arr[0], min = arr[0];

        for (int num : arr) {
            if (num > max) {
                max = num;
            }
            if (num < min) {
                min = num;
            }
        }

        System.out.println("Maximum number is: " + max);
    }
}
```

```

        System.out.println("Minimum number is: " + min);
    }
}

```

Output:

Maximum number is: 25

Minimum number is: 5

25. Write a Java program to find the sum of the first n natural numbers.

```

public class SumNaturalNumbers {
    public static void main(String[] args) {
        int n = 10;
        int sum = n * (n + 1) / 2;
        System.out.println("Sum of first " + n + " natural
numbers: " + sum);
    }
}

```

Output: Sum of first 10 natural numbers: 55

26. Write a Java program to find the common elements between two arrays.

```

import java.util.HashSet;

public class CommonElements {
    public static void main(String[] args) {
        int[] arr1 = {1, 2, 3, 4, 5};
        int[] arr2 = {3, 4, 5, 6, 7};

        HashSet<Integer> set = new HashSet<>();
        for (int i: arr1) {
            set.add(i);
        }

        System.out.print("Common elements: ");
        for (int i: arr2) {
            if (set.contains(i)) {
                System.out.print(i + " ");
            }
        }
    }
}

```

```

    }
}
}
}

```

Output: Common elements: 3 4 5

27. Write a Java program to find the largest element in each row of a 2D array.

```

public class LargestInRow {
    public static void main(String[] args) {
        int[][] arr = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

        for (int i = 0; i < arr.length; i++) {
            int max = arr[i][0];
            for (int j = 1; j < arr[i].length; j++) {
                if (arr[i][j] > max) {
                    max = arr[i][j];
                }
            }
            System.out.println("Largest element in row " + (i +
1) + ": " + max);
        }
    }
}

```

Output:

Largest element in row 1: 3

Largest element in row 2: 6

Largest element in row

28. Write a Java program to remove duplicates from an array.

```

import java.util.Arrays;
import java.util.LinkedHashSet;

public class RemoveDuplicates {
    public static void main(String[] args) {
        Integer[] arr = {1, 2, 2, 3, 4, 4, 5};
        LinkedHashSet<Integer> set = new

```

```

LinkedHashSet<>(Arrays.asList(arr));

        Integer[] newArr = set.toArray(new Integer[0]);
        System.out.println("Array after removing duplicates: "
+ Arrays.toString(newArr));
    }
}

```

Output: Array after removing duplicates: [1, 2, 3, 4, 5]

29. Write a Java program to convert a decimal number to binary.

```

public class DecimalToBinary {
    public static void main(String[] args) {
        int decimal = 10;
        String binary = Integer.toBinaryString(decimal);
        System.out.println("Binary of " + decimal + " is: " +
binary);
    }
}

```

Output: Binary of 10 is: 1010

30. Write a Java program to check if a number is a power of two.

```

public class PowerOfTwo {
    public static void main(String[] args) {
        int num = 16;
        boolean result = isPowerOfTwo(num);
        if (result) {
            System.out.println(num + " is a power of two.");
        } else {
            System.out.println(num + " is not a power of
two.");
        }
    }

    public static boolean isPowerOfTwo(int num) {
        if (num <= 0) {
            return false;
        }
        return (num & (num - 1)) == 0;
    }
}

```

```
}
```

Output: 16 is a power of two.

31. Write a Java program to implement binary search.

```
import java.util.Arrays;

public class BinarySearch {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};
        int target = 30;
        Arrays.sort(arr); // Binary search requires sorted
array
        int index = binarySearch(arr, target, 0, arr.length -
1);
        if (index == -1) {
            System.out.println("Element not found.");
        } else {
            System.out.println("Element found at index: " +
index);
        }
    }

    public static int binarySearch(int[] arr, int target, int
low, int high) {
        if (low > high) {
            return -1;
        }
        int mid = (low + high) / 2;

        if (arr[mid] == target) {
            return mid;
        } else if (arr[mid] > target) {
            return binarySearch(arr, target, low, mid - 1);
        } else {
            return binarySearch(arr, target, mid + 1, high);
        }
    }
}
```

Output: Element found at index: 2

32. Write a Java program to swap two numbers without using a temporary variable.

```
public class SwapNumbers {  
    public static void main(String[] args) {  
        int a = 10, b = 20;  
  
        System.out.println("Before swapping: a = " + a + ", b =  
" + b);  
        a = a + b;  
        b = a - b;  
        a = a - b;  
        System.out.println("After swapping: a = " + a + ", b =  
" + b);  
    }  
}
```

Output:

Before swapping: a = 10, b = 20

After swapping: a = 20, b = 10

33. Write a Java program to find the second largest number in an array.

```
public class SecondLargest {  
    public static void main(String[] args) {  
        int[] arr = {10, 20, 4, 45, 99};  
        int first = Integer.MIN_VALUE, second =  
Integer.MIN_VALUE;  
  
        for (int num: arr) {  
            if (num > first) {  
                second = first;  
                first = num;  
            } else if (num > second && num != first) {  
                second = num;  
            }  
        }  
  
        if (second == Integer.MIN_VALUE) {  
            System.out.println("No second largest element.");  
        }  
    }  
}
```

```
        } else {  
            System.out.println("Second largest number: " +  
second);  
        }  
    }  
}
```

Output: Second largest number: 45

34. Write a Java program to find the factorial of a number using recursion.

```
public class FactorialRecursion {  
    public static void main(String[] args) {  
        int num = 5;  
        System.out.println("Factorial of " + num + " is: " +  
factorial(num));  
    }  
  
    public static int factorial(int num) {  
        if (num == 0) {  
            return 1;  
        }  
        return num * factorial(num - 1);  
    }  
}
```

Output: Factorial of 5 is: 120

35. Write a Java program to check if a number is a perfect square.

```
public class PerfectSquare {  
    public static void main(String[] args) {  
        int num = 16;  
        if (isPerfectSquare(num)) {  
            System.out.println(num + " is a perfect square.");  
        } else {  
            System.out.println(num + " is not a perfect  
square.");  
        }  
    }  
}
```

```
public static boolean isPerfectSquare(int num) {  
    int sqrt = (int) Math.sqrt(num);  
    return sqrt * sqrt == num;  
}  
}
```

Output: 16 is a perfect square.

36. Write a Java program to implement selection sort.

```
import java.util.Arrays;  
  
public class SelectionSort {  
    public static void main(String[] args) {  
        int[] arr = {29, 10, 14, 37, 13};  
        selectionSort(arr);  
        System.out.println("Sorted array: " +  
Arrays.toString(arr));  
    }  
  
    public static void selectionSort(int[] arr) {  
        for (int i = 0; i < arr.length - 1; i++) {  
            int minIdx = i;  
            for (int j = i + 1; j < arr.length; j++) {  
                if (arr[j] < arr[minIdx]) {  
                    minIdx = j;  
                }  
            }  
            int temp = arr[minIdx];  
            arr[minIdx] = arr[i];  
            arr[i] = temp;  
        }  
    }  
}
```

Output: Sorted array: [10, 13, 14, 29, 37]

37. Write a Java program to check if two strings are anagrams.

```
import java.util.Arrays;

public class AnagramCheck {
    public static void main(String[] args) {
        String str1 = "listen";
        String str2 = "silent";

        if (areAnagrams(str1, str2)) {
            System.out.println(str1 + " and " + str2 + " are
anagrams.");
        } else {
            System.out.println(str1 + " and " + str2 + " are
not anagrams.");
        }
    }

    public static boolean areAnagrams(String str1, String str2)
{
    if (str1.length() != str2.length()) {
        return false;
    }
    char[] arr1 = str1.toCharArray();
    char[] arr2 = str2.toCharArray();
    Arrays.sort(arr1);
    Arrays.sort(arr2);
    return Arrays.equals(arr1, arr2);
}
}
```

Output: listen and silent are anagrams.

38. Write a Java program to find the missing number in an array of consecutive numbers.

```
public class MissingNumber {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 5, 6};
        int n = arr.length + 1;
        int totalSum = n * (n + 1) / 2;
        int arrSum = 0;
```

```
        for (int num : arr) {
            arrSum += num;
        }

        int missingNumber = totalSum - arrSum;
        System.out.println("The missing number is: " +
missingNumber);
    }
}
```

Output: The missing number is: 4

39. Write a Java program to check if a string is a palindrome.

```
public class PalindromeCheck {
    public static void main(String[] args) {
        String str = "madam";
        if (isPalindrome(str)) {
            System.out.println(str + " is a palindrome.");
        } else {
            System.out.println(str + " is not a palindrome.");
        }
    }

    public static boolean isPalindrome(String str) {
        int left = 0;
        int right = str.length() - 1;

        while (left < right) {
            if (str.charAt(left) != str.charAt(right)) {
                return false;
            }
            left++;
            right--;
        }
        return true;
    }
}
```

Output: madam is a palindrome.

40. Write a Java program to merge two sorted arrays.

```
import java.util.Arrays;

public class MergeSortedArrays {
    public static void main(String[] args) {
        int[] arr1 = {1, 3, 5, 7};
        int[] arr2 = {2, 4, 6, 8};
        int[] mergedArr = mergeArrays(arr1, arr2);
        System.out.println("Merged array: " +
Arrays.toString(mergedArr));
    }

    public static int[] mergeArrays(int[] arr1, int[] arr2) {
        int[] mergedArr = new int[arr1.length + arr2.length];
        int i = 0, j = 0, k = 0;

        while (i < arr1.length && j < arr2.length) {
            if (arr1[i] < arr2[j]) {
                mergedArr[k++] = arr1[i++];
            } else {
                mergedArr[k++] = arr2[j++];
            }
        }

        while (i < arr1.length) {
            mergedArr[k++] = arr1[i++];
        }

        while (j < arr2.length) {
            mergedArr[k++] = arr2[j++];
        }

        return mergedArr;
    }
}
```

Output: Merged array: [1, 2, 3, 4, 5, 6, 7, 8]

41. Write a Java program to find the GCD (Greatest Common Divisor) of two numbers.

```
public class GCD {
    public static void main(String[] args) {
        int num1 = 56;
        int num2 = 98;
        System.out.println("GCD of " + num1 + " and " + num2 +
" is: " + findGCD(num1, num2));
    }

    public static int findGCD(int num1, int num2) {
        if (num2 == 0) {
            return num1;
        }
        return findGCD(num2, num1 % num2);
    }
}
```

Output: GCD of 56 and 98 are: 14

42. Write a Java program to find the LCM (Least Common Multiple) of two numbers.

```
public class LCM {
    public static void main(String[] args) {
        int num1 = 12;
        int num2 = 18;
        System.out.println("LCM of " + num1 + " and " + num2 +
" is: " + findLCM(num1, num2));
    }

    public static int findLCM(int num1, int num2) {
        return (num1 * num2) / findGCD(num1, num2);
    }

    public static int findGCD(int num1, int num2) {
        if (num2 == 0) {
            return num1;
        }
        return findGCD(num2, num1 % num2);
    }
}
```

```
}
```

Output: LCM of 12 and 18 is: 36

43. Write a Java program to find the first non-repeating character in a string.

```
import java.util.LinkedHashMap;
import java.util.Map;

public class FirstNonRepeatingCharacter {
    public static void main(String[] args) {
        String str = "swiss";
        System.out.println("First non-repeating character: " +
findFirstNonRepeating(str));
    }
    public static Character findFirstNonRepeating(String str) {
        Map<Character, Integer> charCountMap = new
LinkedHashMap<>();

        for (char c: str.toCharArray()) {
            charCountMap.put(c, charCountMap.getOrDefault(c, 0)
+ 1);
        }
        for (char c: str.toCharArray()) {
            if (charCountMap.get(c) == 1) {
                return c;
            }
        }
        return null;
    }
}
```

Output: First non-repeating character: w

44. Write a Java program to rotate an array to the right by a given number of steps.

```
import java.util.Arrays;

public class RotateArray {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        int k = 2;
        rotate(arr, k);
        System.out.println("Array after rotation: " +
Arrays.toString(arr));
    }

    public static void rotate(int[] arr, int k) {
        k = k % arr.length;
        reverse(arr, 0, arr.length - 1);
        reverse(arr, 0, k - 1);
        reverse(arr, k, arr.length - 1);
    }

    public static void reverse(int[] arr, int start, int end) {
        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }
}
```

Output: Array after rotation: [4, 5, 1, 2, 3]

45. Write a Java program to reverse the words in a sentence.

```
public class ReverseWords {
    public static void main(String[] args) {
        String sentence = "Hello World from Java";
        String reversed = reverseWords(sentence);
        System.out.println("Reversed sentence: " + reversed);
    }

    public static String reverseWords(String sentence) {
        String[] words = sentence.split(" ");
        StringBuilder reversed = new StringBuilder();

        for (int i = words.length - 1; i >= 0; i--) {
            reversed.append(words[i]).append(" ");
        }

        return reversed.toString().trim();
    }
}
```

Output: Reversed sentence: Java from World Hello

46. Write a Java program using recursion to find the nth Fibonacci number.

```
public class Fibonacci {
    public static void main(String[] args) {
        int n = 10;
        System.out.println("The " + n + "th Fibonacci number is: " + fib(n));
    }

    public static int fib(int n) {
        if (n <= 1) {
            return n;
        }
        return fib(n - 1) + fib(n - 2);
    }
}
```

Output: The 10th Fibonacci number is: 55